



System Design Document for R-ICMS: *Regional Integrated Corridor Management System*

Version: 1.0

Approval date: 10/30/2018



DOCUMENT CONTROL PANEL		
File Name & Location	P:\10-23368_FDOT_D5_ICMS\Deliverables\Software Detailed Design\R-ICMS-SDD-1.0.docx	
Version Number:	1.0	
	Name	Date
Created By:	Clay Weston, SwRI	7/23/2018
Reviewed By:	Joe Zingalli, Kapsch	8/8/2018
	Claudia Paskauskas, DFE review – InNovo Partners	8/13/2018
	Clay Packard, VHB	8/14/2018
	Aafiya Shah, Kapsch	8/9/2018
	Kevin Miller, Kapsch	8/7/2018
	Clay Packard, VHB	10/1/2018
Modified By:	David Vickers	8/27/2018
	Clay Weston	9/17/2018
	Clay Weston	10/3/2018
Approved By:	Tushar Patel, FDOT	10/30/2018

Table of Contents

1	Overview.....	1
1.1	<i>Identification</i>	<i>1</i>
1.2	<i>System Overview.....</i>	<i>1</i>
2	Reference Documents	2
3	Design.....	2
3.1	<i>System Context</i>	<i>2</i>
3.2	<i>Design Assumptions and Constraints [by Category].....</i>	<i>3</i>
	3.2.1 Operational Assumptions.....	4
	3.2.2 Non-operational Design	4
	3.2.3 System Inputs and Outputs.....	4
	3.2.4 Behavior and Performance	5
	3.2.5 User View of Data	6
	3.2.6 Safety, Security, and Privacy	6
	3.2.7 Third Party Components	7
3.3	System Architecture.....	15
3.4	System Components.....	17
	3.4.1 DFE Layers and Components.....	18
	3.4.1.1 Drivers.....	19
	3.4.1.2 Pipelines.....	59
	3.4.1.3 Data Store.....	63
	3.4.1.4 Data Services	71
	3.4.2 DSS/Business Services.....	84
	3.4.2.1 Session Authentication (SA-BS)	84
	3.4.2.2 User Personalization (UP-BS).....	84
	3.4.2.3 Admin (ADM-BS).....	84
	3.4.2.4 Alert (ALT-BS).....	84
	3.4.2.5 Orchestration (ORC-BS)	84
	3.4.2.6 Reporting (RPT-BS)	85
	3.4.2.7 Business Rules Engine (BRE-BS).....	85
	3.4.2.8 Predictive Engine (PRE-BS)	85
	3.4.2.9 Evaluation Engine (EVE-BS)	85
	3.4.2.10 Event (EM-BS).....	85
	3.4.2.11 Response Plan (RP-BS).....	86

	3.4.2.12 Signal Optimization Timing (SOT-BS)	86
3.4.3	User Interface	93
	3.4.3.1 Login (Login-UI)	94
	3.4.3.2 Map (MAP-UI)	95
	3.4.3.3 Admin (ADM-UI)	103
	3.4.3.4 Notifications (NOT-UI)	103
	3.4.3.5 Alerts (ALT-UI)	103
	3.4.3.6 Reporting (RP-UI).....	104
	3.4.3.7 Event Management (EM-UI)	104
	3.4.3.8 Response Plan Management (RPM-UI).....	104
	3.4.3.9 Signal Optimization Timing (SOT-UI).....	104
3.4.4	Common Core	105
	3.4.4.1 Authorization (Auth-CC)	105
	3.4.4.2 Logging (Log-CC).....	105
	3.4.4.3 Monitoring (Mon-CC)	106
	3.4.4.4 Alerting (AL-CC)	106
	3.4.4.5 Gateway (GW-CC).....	106
3.5	System Interfaces.....	107
	3.5.1 Data Source Drivers	107
	3.5.2 SunGuide Driver.....	107
	3.5.3 Third Party Access.....	107
	3.5.4 Signal Timing Files.....	107
	3.5.5 Bulk Load	107
	3.5.6 Modeling Engine	108
	3.5.7 Signal Timing Optimization Engine.....	108
3.6	System Resources	108
	3.6.1 Hardware Configuration	108
	3.6.2 Physical Deployment: Servers and Network.....	109
3.7	Concept of Execution.....	110
	3.7.1 Security.....	113
	3.7.1.1 Login	114
	3.7.1.2 User Data Request.....	115
	3.7.1.3 User Authorization Update.....	116

3.7.1.4	<i>User Personalization</i>	117
3.7.1.5	<i>GIS Use Cases</i>	118
3.7.2	Situational Awareness	122
3.7.2.1	<i>ITSIQA Ingestion</i>	123
3.7.2.2	<i>Map Update for Pre-Located Data</i>	124
3.7.2.3	<i>Map Update for Dynamically Located Data</i>	125
3.7.3	Event Diagrams	126
3.7.3.1	<i>SunGuide / R-ICMS Event</i>	127
3.7.3.2	<i>SunGuide / R-ICMS Event Flow</i>	128
3.7.3.3	<i>Business Rule Configuration</i>	129
3.7.3.4	<i>Response Plan Selection</i>	130
3.7.3.5	<i>Response Plan Approval</i>	131
3.7.3.6	<i>Response Plan Activation</i>	132
3.7.3.7	<i>Response Plan Monitor/Reevaluation</i>	133
3.7.4	Signal Optimization Timing Diagrams	134
3.7.4.1	<i>Periodic Optimization</i>	135
3.7.4.2	<i>On Demand Optimization</i>	136
3.7.4.3	<i>Modified Timing Plan</i>	137
3.7.4.4	<i>Detailed: Create Corridor</i>	138
3.7.4.5	<i>Detailed: SOT-DS Intersection Geometry</i>	139
3.7.4.6	<i>Detailed: Modify Corridor</i>	140
3.7.4.7	<i>Detailed: Corridor Optimization 1</i>	141
3.7.4.8	<i>Detailed: Corridor Optimization 2</i>	143
3.7.4.9	<i>Detailed: Deploy Corridor</i>	144
3.7.4.10	<i>Detailed: SOT-BS Logging</i>	145
3.7.4.11	<i>Iteration 1: Use Case Description</i>	146
3.7.5	Other Use Case Diagrams	148
3.7.5.1	<i>Other: External Access – Authentication</i>	149
3.7.5.2	<i>Other: External Access – Data Request</i>	150
3.7.5.3	<i>Other: External Access – Streaming</i>	151
3.7.5.4	<i>Other: External Access – Metadata</i>	152
3.7.5.5	<i>Other: Bulk Load – Business Rules</i>	153
3.7.5.6	<i>Other: Bulk Load – Response Plans</i>	154
3.7.5.7	<i>Other: Parking</i>	155
3.7.5.8	<i>Other: Alerts, Information, and Notification</i>	156

	3.7.5.9 Other: Response Plan Notification	157
	3.7.5.10 Other: Data Source Outage Alert	158
	3.7.5.11 Other: Logging	159
	3.7.5.12 Other: Monitoring	160
	3.7.5.13 Other: Timing Plan Data Service.....	161
3.8	System Deployment	162
	3.8.1 Service Host Deployment.....	162
	3.8.2 Containerized Service Orchestration.....	163
4	Notes.....	164
	4.1 Definitions	164

List of Tables

Table 1 - R-ICMS Fundamental Capabilities	5
Table 2 - Third Party Components	8
Table 3 - ITSQA Ingestion	26
Table 4 – Transit Location Specification.....	46
Table 5 – Basemap Specification	54
Table 6 – RCI # Lanes at Intersection Specification.....	54
Table 7 – School Zones Specification	55
Table 8 – School Zones Specification	56
Table 9 – Emergency Responder Specification	58
Table 10 – Data Source Storage	64
Table 11 - SQL vs NoSQL.....	67
Table 12 - Generic Data Services	72
Table 13 - ITSQA API Details.....	73
Table 14 - GIS Data Services.....	77
Table 15 - Stream Service Details.....	79
Table 16 - ArcGIS Feature Service API Details.....	80
Table 17 - ArcGIS Vector Layer Server API	82
Table 18 - R-ICMS Data Services	83
Table 19 - Hardware Configuration	108
Table 20 - Use Case View Map Details.....	118
Table 21 - View GIS Layers Use Case Details.....	119
Table 22 - View Details Use Case Details.....	120
Table 23 - Receive Statis Data Use Case Details	121
Table 24 - Receive Dynamic Data Use Case Details	122
Table 25 - Optimize Corridor Use Case Details	146

List of Figures

Figure 1 - R-ICMS Context Diagram	3
Figure 2 - R-ICMS High-Level Diagram.....	16
Figure 3 - Color Scheme	18
Figure 4 – Ingestion Flow	20
Figure 5 – FTP Workflow.....	23
Figure 6 – SunGuide Data Ingestion Process	25
Figure 7 – SunGuide Data Flow	25
Figure 8 – ITSQA Data Flow	31
Figure 9 – R-ICMS Data Streaming Workflow.....	63
Figure 10 – GIS Data Store.....	69
Figure 11 - Geodata Shape File Data Flow	70
Figure 12 - Map Layer Shape File.....	71
Figure 13 – GeoEvent Server Flow	78
Figure 14 – ITSQA GeoEvent Data Flow	79
Figure 15 – Angular MVC.....	94
Figure 16 – User Interface Navigation Hierarchy	94

Figure 17 - Login Screen.....	95
Figure 18 - Login Flow.....	95
Figure 19 - Map Data Flow.....	97
Figure 20 - Main Screen.....	98
Figure 21 - Search Screen	98
Figure 22 - Zoom features.....	99
Figure 23 - Home button	99
Figure 24 - Feature Details.....	100
Figure 25 - Layer Selection Screen.....	101
Figure 26 - Traffic Condition Layer Selection	101
Figure 27 - Basemap Selection	102
Figure 28 - Legend Selection	102
Figure 29 - Navigation	103
Figure 30 - Navigation	103
Figure 31 – Log Design Pattern	106
Figure 32 – Physical Deployment: Servers and Network Diagram.....	110
Figure 33 - R-ICMS Coloring Scheme Diagram	112
Figure 34 - Login Sequence Diagram.....	114
Figure 35 - User Data Request Sequence Diagram.....	115
Figure 36 - User Authorization Update Sequence Diagram	116
Figure 37 - User Personalization Sequence Diagram	117
Figure 38 – GIS Use Cases	118
Figure 39 - ITSQA Ingestion Sequence Diagram	123
Figure 40 - Map Update for Pre-Located Data Sequence Diagram.....	124
Figure 41 - Map Update for Dynamically Located Data Sequence Diagram	125
Figure 42 - SunGuide / R-ICMS Event Flow Diagram.....	127
Figure 43 - SunGuide / R-ICMS Event Flow Sequence Diagram.....	128
Figure 44 - Business Rule Configuration Sequence Diagram	129
Figure 45 - Response Plan Selection Sequence Diagram	130
Figure 46 - Response Plan Approval Sequence Diagram.....	131
Figure 47 - Response Plan Activation Sequence Diagram.....	132
Figure 48 - Response Plan Monitor/Reevaluation Sequence Diagram	133
Figure 49 - SOT - Periodic Optimization Sequence Diagram	135
Figure 50 - SOT On Demand Optimization Sequence Diagram.....	136
Figure 51 - SOT Modified Timing Plan Sequence Diagram.....	137
Figure 52 – SOT Detailed Create Corridor Sequence Diagram	138
Figure 53 – SOT Detailed SOT-DS Intersection Geometry Diagram	139
Figure 54 – SOT Detailed Modify Corridor Sequence Diagram.....	140
Figure 55 – SOT Detailed Corridor Optimization 1.....	142
Figure 56 – SOT Detailed Corridor Optimization 2.....	143
Figure 57 – SOT Detailed Deploy Corridor.....	144
Figure 58 – SOT Detailed SOT-BS Logging.....	145
Figure 59 – SOT Use Cases	146
Figure 60 – SOT Operational Database Diagram	148
Figure 61 - External Access - Authentication Sequence Diagram.....	149
Figure 62 - External Access - Data Request Sequence Diagram.....	150

Figure 63 - External Access - Streaming Sequence Diagram	151
Figure 64 - External Access - Metadata Sequence Diagram.....	152
Figure 65 - Bulk Load – Business Rules Sequence Diagram	153
Figure 66 - Bulk Load - Response Plans Sequence Diagram	154
Figure 67 - Parking Sequence Diagram	155
Figure 68 - Alerts, Information, and Notification Flow Chart Diagram	156
Figure 69 - Response Plan Notification Sequence Diagram	157
Figure 70 – Data Source Outage Alert Sequence Diagram.....	158
Figure 71 - Logging Sequence Diagram.....	159
Figure 72 - Monitoring Sequence Diagram.....	160
Figure 73 – Timing Plan Data Service Diagram	161
Figure 74 – Service Host Deployment Diagram	162
Figure 75 – Containerized Service Orchestration Diagram	163

List of Acronyms and Abbreviations

AAM	Active Arterial Management
AM	Ante Meridiem
API	Application Program Interface
AST	Agency for State Technology
ATMS	Advanced Traffic Management System
AVL	Automatic Vehicle Location
AWS	Amazon Web Services
CCTV	Closed Circuit Television
CLI	Command Line Interface
COTS	Commercial Off the Shelf
CSV	Comma Separated Variable
DFE	Data Fusion Environment
DMS	Dynamic Message Signs
DOT	Department of Transportation
DSS	Decision Support System
ERD	Entity Relationship Diagram
ETL	Extract, Transform, Load
FCS	Florida Cybersecurity Standards
FDOT	Florida Department of Transportation
FTP/SFTP	File Transport Protocol / Secure File Transport Protocol
GIS	Geographic Information System
GTFS	General Transit Feed Specification
GTFS-RT	General Transit Feed Specification – Real Time
HDFS	Hadoop Distributed File System
HTTPS	Hyper Text Transfer Protocol Secure
ICD	Interface Control Document
ID	Identifier
IEN	Information Exchange Network
IMC	Intersection Movement Counts
IT	Information Technology
ITS	Intelligent Transportation System
ITS IQA	Intelligent Transportation System Input Quality Assurance
JSON	JavaScript Object Notation
JWT	JSON Web Tokens
LDAP	Lightweight Directory Access Protocol
ME	Modeling Engine
MOE	Measure of Effectiveness
MS SQL	Microsoft SQL
MVC	Model View Controller
OAS	OpenAPI Specification
PD	Preliminary Design
PDF	Portable Document Format
PDR	Preliminary Design Review
PM	Post Meridiem
RCI	Roadway Characteristics Inventory
RDBMS	Relational DataBase Management System
REST	Representational State Transfer
R-ICMS	Regional Integrated Corridor Management System
RP	Response Plan

RPE.....	Response Plan Element
SDD	System Design Document
SHS	State Highway System
SLES	SUSE Linux Enterprise Server
SOT.....	Signal Optimization Tool
SQL	Structured Query Language
SSL	Secure Sockets Layer
TBD	To Be Determined
TGDC.....	Time Grouped Demand Cluster
TLS.....	Transport Layer Security
TSMO	Transportation Systems Management and Operations
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
XML	Extensible Markup Language

Acronyms are used as Suffixes to Components

BS	Business Service
CC.....	Common Core
DR	Driver layer
DS.....	Data Service
GW.....	Gateway
PL	Pipeline layer
ST	Data Store
UI	User Interface

Component Abbreviations

ADM-BS.....	Admin – Business Service
ADM-UI	Admin – User Interface
AL-CC	Alerting – Common Core
ALT-BS.....	Alert – Business Service
ALT-UI	Alerts – User Interface
AUTH-CC	Authorization – Common Core
BM-DR.....	Basemap – Driver
BRE-BS.....	Business Rules Engine – Business Service
CT5-PL	Traffic Conditions 5-Minute Average – Pipeline
CTC-PL	Current Traffic Conditions – Pipeline
EM-BS	Event – Business Service
EM-DS	Event Management – Data Service
EM-UI	Event Management – User Interface
ERL-DR	Emergency Responder Locations (GIS) – Driver
EVE-BS.....	Evaluation Engine – Business Service
EVT-PL	Events – Pipeline
FS-ST	File Store – Data Store
GEO-DS	Geographic Data Service
GIS-ST.....	GIS Database – Data Store
GW-CC	Gateway – Common Core
HCS7.....	Highway Capacity Manual Version 7

IDS-PL.....	ITS Device Status – Pipeline
IMC-DR.....	Intersection Movement Counts – Driver
IMC-PL.....	Intersection Movement Counts – Pipeline
LOG-CC.....	Logging – Common Core
Login-UI.....	Login – User Interface
MAP-UI	Map – User Interface
MD-DS.....	Metadata – Data Service
MON-CC.....	Monitoring – Common Core
NOS-ST	NoSQL – Data Store
NOT-UI	Notifications – User Interface
ORC-BS.....	Orchestration – Business Service
PK-PL.....	Parking – Pipeline
PRE-BS.....	Predictive Engine – Business Service
RCI-DR.....	RCI Number of Lanes at Intersection (GIS) – Driver
RP-BS.....	Response Plan (RP-BS) – Business Service
RPM-UI.....	Response Plan Management – User Interface
RPT-BS.....	Reporting – Business Service
RP-UI	Reporting – User Interface
SA-BS.....	Session Authentication – Business Service
SE-DR	Special Event – Driver
SG-DR.....	SunGuide – Driver
SPM-DR.....	Signal Performance Measures – Driver
SOT-BS	Signal Optimization Timing – Business Service
SOT-DS	Signal Optimization Timing – Data Service
SOT-UI.....	Signal Optimization Timing – User Interface
SQL-ST.....	SQL Database – Data Store
ST-PL	Signal Timing – Pipeline
SZ-DR	School Zones (GIS) – Driver
TA-DR.....	Transit AVL – Driver
TAVL-PL.....	Transit AVL – Pipeline
TL-DR.....	Transit Locations – Driver
TRF-DR	Traffic – Driver
UP-BS	User Personalization – Business Service
VID-DR	Video – Driver
WEA-DR	Weather – Driver

1 Overview

This System Design Document (SDD) establishes the detailed design for iteration 1 of 4 for the Regional – Integrated Corridor Management System (R-ICMS). This document includes all of the Preliminary Design elements as well as the iteration 1 Critical Design elements. Specific areas addressed in Iteration 1 include:

- Static data ingestion, storage, and display on a map.
- ~~Intelligent Transportation System Input Quality Assurance (ITS IQA) traffic data ingestion, storage, and display on a map.~~
- Dynamic Message Sign (DMS) data ingestion, storage, and display on a map.
- The intended mechanism used to wrap the HCS Tool for system use.

This document enables a system developer to understand how to further design and implement the R-ICMS, and this document is a communication tool for stakeholder concurrence and a record of how the system will meet system requirements.

1.1 Identification

The project for which this document was created is identified by the following:

Project Name: Central Florida Regional Integrated Corridor Management System.
Agreement Number: BE521
Financial Project Identification: 436328-1-82-01
Federal Aid Project Number: Not Applicable.

1.2 System Overview

The R-ICMS is intended to be an initial implementation of a multi-modal regional transportation management system. The R-ICMS will integrate freeway, arterial, transit, and rail transportation management for the I4 corridor, including management of transportation system components owned and operated by the state, as well as the county, city, and regional transportation agencies.

The R-ICMS will consist of, but not be limited to; commercial off-the-shelf (COTS) modeling software (provided by the DEPARTMENT), a custom-built Decision Support System (DSS), a custom-built Information Exchange Network (IEN) subsystem that includes dashboards and other user interfaces to the system, and a Data Fusion Environment (DFE) to host data sources for both the R-ICMS and other external users and applications.

This project is funded and managed by District 5 of the Florida Department of Transportation (FDOT). It is intended for the use of District personnel, as well as personnel from the cities, counties, and transportation agencies located within the District. The initial deployment of the R-ICMS will be to the Transportation Management Center being built in District 5 by the FDOT.

2 Reference Documents

The following documents, of the exact issue shown, form a part of this document to the extent specified herein. In the event of a conflict between the documents referenced herein and the contents of this document, this document shall be considered the superseding requirement.

System and Subsystem Requirements
Specification for R-ICMS for: Regional
Integrated Corridor Management System:
R-ICMS-REQ-0.2

BE521 - Executed Contract

*Data Sets Needed by ICMS - ICMS
Requirements Table 7

SunGuide Documentation

Southwest Research Institute
FDOT R-ICMS Project SharePoint Site

Florida Department of Transportation
D5prcustodian@dot.state.fl.us
Southwest Research Institute
FDOT R-ICMS Project SharePoint Site

<http://www.sunguidesoftware.com/document-library>

*This spreadsheet originated from the requirements, was modified in the negotiations and project requirements definition processes and remains a part of the SharePoint project website until the completion of the project where the final version will be put into the design document as a revision at the final iteration.

3 Design

This section documents the system design.

3.1 System Context

The R-ICMS will be integrated into the FDOT District 5 Transportation Systems Management and Operations (TSMO) systems. It will collect data from FDOT and other agency systems. It will store the data in a DFE, which will also provide the data to other components of the R-ICMS and to external users. Via a user interface referred to as the IEN, the R-ICMS will support FDOT and agency users in conducting traffic management operations. The system will support the development, storage, and use of response plans to be used to manage the transportation infrastructure of the corridor in response to periodic and non-recurring congestion. The R-ICMS will support the development, storage and use of the set of business rules that determine when response plans should be suggested to users. The R-ICMS will include a DSS which will support the business functions of the users. The system will execute the business rules and determine when a set of response plans should be recommended to the users. The R-ICMS will support the selection of a response plan, its approval by affected agency users, and the implementation of the response plan. The R-ICMS will support the management of parking resources in the corridor and the optimization of signal timing plans for linear groups of signals. The R-ICMS will also

support basic access to the data stored in the DFE both by other R-ICMS subsystems and by external users.

Figure 1 - R-ICMS Context Diagram provides an overview of the TSMO environment and the place of the R-ICMS and its DFE in that environment.

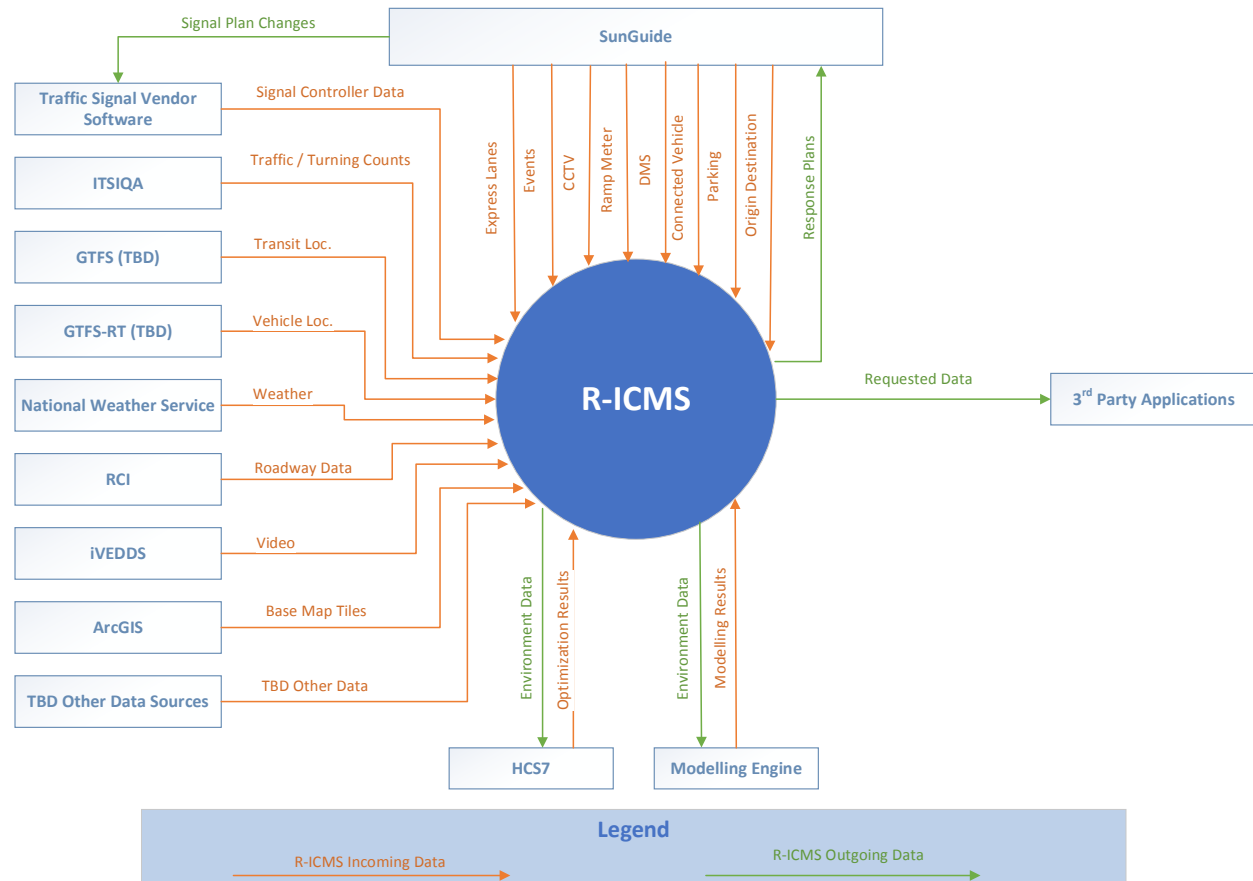


Figure 1 - R-ICMS Context Diagram

3.2 Design Assumptions and Constraints [by Category]

The R-ICMS design philosophy is centered around the use of architecturally significant use cases to drive an architecture intended to satisfy the R-ICMS requirements. Architecturally significant use cases are those which have structural or performance impacts on the architecture needed to satisfy the users' needs. This document, in addition to the high-level design information, contains more detailed design for those use cases selected as being architecturally significant and those use cases intended to be implemented in Iteration 1. Use cases not detailed in this document are not less important and are not being ignored. They were just not the specific ones selected to drive the architecture. Once the architecture has been designed, it is anticipated that there should be relatively few changes to the high-level design as the other use cases are added. In general, UML sequence diagrams have been used to illustrate the interactions between architectural components. Occasionally other diagram types have been used to illustrate specific portions of the architecture or the design.

The following paragraphs describe high-level design assumptions and constraints considered in developing the architecture.

3.2.1 Operational Assumptions

A number of operational assumptions drove the design. Those assumptions include:

- The R-ICMS will consist of a combination of open source, commercial and custom software (with the custom software owned by FDOT)
- The DFE will make use of Structured Query Language (SQL) databases, noSQL databases, Geographic Information System (GIS) data, File Stores, and streaming data pipelines
- The R-ICMS will operate in a web services environment and the IEN will operate within a browser on the user's system
- In the future, the R-ICMS will provide access to transportation-related data for external systems developed or purchased by the District or other agencies
- In the future, the DFE will be integrated into the Smart Cities infrastructure of the region and provide data and analytic access to applications used to integrate Corridor systems
- Many of the actions initiated by the R-ICMS will be executed by SunGuide
- FDOT will provide hosting and operational environment (Traffic Management Center & Data Center)
- A to-be-selected commercial modeling engine will be used by the Predictive Engine.
- Authentication will be based on FDOT's Active Directory, but fine-grained authorization will be based on a combination of Active Directory roles and an internal data store of permissions associated with those roles

3.2.2 Non-operational Design

Non-operational Design assumptions and constraints include:

- The R-ICMS is intended to operate within the FDOT D5 TSMO Information Technology (IT) system of systems
- The decisions documented in the remaining subsections of this section are based on that intent

3.2.3 System Inputs and Outputs

Selection of the set of inputs to be addressed during the initial phase of the implementation of the R-ICMS was based on several fundamental capabilities needed by the initial phase of the system. Those are listed and described in Table 1 - R-ICMS Fundamental Capabilities.

Table 1 - R-ICMS Fundamental Capabilities

Name	Description
Situational Awareness	The capability of providing users with an overall understanding of the current traffic and parking conditions within the core traffic network covered by the R-ICMS (System Requirement 1.1.1 and System Requirement 1.1.23)
Response Plan Definition	The capability of providing users the ability to define response plans for non-recurring congestion occurring within the network (System Requirement 6.1) • Response plan • Response plan element • Response plan element types will be enumerated at 90% design.
Response Plan Execution	The capability of providing users the ability to respond to non-recurring events generated by SunGuide by providing the users suggested response plans (System Requirement 5.1) and allowing the appropriate authorized users to select (System Requirement 5.1.3) and approve (System Requirement 10.1.2.1) the response plans for implementation
Response Plan Monitoring	The capability of monitoring the traffic network while response plans are active to determine if the response plan should be replaced or deactivated (System Requirement 16.1)
Signal Timing Plan Optimization	The capability of providing authorized users the ability to develop signal timing plan sets based on collected traffic data (System Requirement 20.1)

The Data Sources table (see Appendix A) provides the specification of the set of inputs to be received. The contract provides notional representations of the user interaction screens for some of the user interactions. Additional details of the interactions are being provided as a part of the detailed design. Additional information specifying the formats and protocols of the system interactions are being provided on an ongoing basis by FDOT.

3.2.4 Behavior and Performance

This section provides the behavior and performance assumptions and constraints.

For performance and architectural clarity, a layered architecture has been defined for the system. The set of layers and their functionality is provided in Section 3.3 System Architecture.

Since one of the primary capabilities to be provided by the R-ICMS is an understanding of the current traffic network situation, a map-based interface to the real-time (i.e., 1-minute updates)

data from the traffic network sensors is a major focus of the system and therefore a major influence on the architectural decisions. As proposed, ArcGIS is being used as the basis of the map user interface. The use of ArcGIS has significant impact on the overall architecture of the system.

The contract provides a base set of behavioral and performance requirements. Updates to those requirements are included in the SDD. One of the architecturally significant performance requirements is the requirement to provide data on the users' screens within 3 seconds of receiving the data (System Requirement 1.42). Based on that requirement, ArcGIS functionality that allows the addition of attributes (such as colors of polygons and appearance of icons) will be separated from the generation of the base tiles of the map and done using functionality available in the ArcGIS Software Development Kit to knit together the basemap tiles and the attributes derived from user layer selection and real-time data updates.

To ensure that real-time updates are consistently available to both the business services and the data services that provide updates to the user interfaces, a pipeline architecture has been selected for delivering real-time updates from the external systems to the update consumers including the data stores.

To facilitate performance, especially in data updates, it is anticipated that the data services will provide a "publish-subscribe" mechanism for access to streaming data.

3.2.5 User View of Data

As per the requirements (System Requirement 1.2.3), the user interface for the R-ICMS will be via a browser. Angular has been chosen as the framework in which the browser-based user interface will be implemented.

The R-ICMS will provide several types of user interfaces. Interactive user interfaces will be provided to facilitate user understanding and actions needed to accomplish the primary purposes of the initial version of the R-ICMS. A map-based user interface will be provided to facilitate understanding of the current traffic network situation. Additional user interfaces will be provided to facilitate understanding of specific aspects of the traffic network conditions. Interfaces will also be provided to allow users to evaluate, select, approve, activate, and terminate response plans. Interfaces will be provided to allow users to configure the system. Interfaces will also be provided to allow users to request periodic and on-demand generation, modification, evaluation, and signing of signal timing plans.

3.2.6 Safety, Security, and Privacy

The R-ICMS will be integrated into the security environment of FDOT D5. R-ICMS so users will be required to have active accounts in the FDOT D5 domain or in a domain trusted by the FDOT D5 domain. Those accounts will be created, managed, and terminated by standard FDOT D5 processes.

Fine-grained authorization of user capabilities will be controlled within the R-ICMS. Administrative R-ICMS users will be provided the capability to control which data sources users can access, which actions users can perform, and which response plan elements users can approve based on the devices contained in the response plan element. Permissions for access to data sources and system capabilities can be provided to users by assigning users to specific roles. Response plan element approval capability can be provided to users by assigning users to agencies which have the relevant element assignments.

Access to the R-ICMS from outside the FDOT network will be provided through secured communications channels (System Requirement 3.1). It is anticipated that these secure channels will be based on Secure Sockets Layer (SSL) to the browser.

3.2.7 Third Party Components

Table 2 - Third Party Components contains the initial list of known third-party components intended for use as part of the R-ICMS. Other libraries and third-party components, identified later, will be addressed in the appropriate 90% design documentation.

Table 2 - Third Party Components

Component	License	Type	Features	Reason Chosen
HCS7	TBD	SOT	Optimization of corridors	A decision was made during the negotiations to utilize the capabilities of HCS7, a signal timing plan optimization tool-set developed by the University of Florida, as the basis of the Signal Optimization Tool (SOT) that is an integral part of the initial version of the R-ICMS. HCS7 generates proposed signal timing plans for intersections and for corridors. The optimization is based on using intersection and corridor parameters combined with calculations based on the federal Highway Capacity Manual and a genetic approach to optimization. Investigation of the match between user needs, the capabilities of HCS7, and the ability of HCS7 to be integrated into the R-ICMS is still being conducted. However, it is anticipated that much of the base capabilities of the SOT will be provided by the HCS7 tool-set. Additional user needs not met by the basic HCS7 tool-set will be provided by the portions of the R-ICMS wrapping the HCS7 tool-set.
Modeling Engine	TBD	TBD	TBD	A modeling engine is being selected by FDOT. Ramifications of the modeling engine selection will be dealt with after the selection is completed.
<u>ElastAlert</u>	<u>Apache 2.0</u>	Alerting	Monitoring and alerting based on data in ElasticSearch	ElastAlert is used to watch for various conditions in ElasticSearch that are of interest for monitoring the health of the R-ICMS system and notifying an operator if something appears out of the ordinary. This helps catch any anomalies in the system early so that they can be addressed.

Component	License	Type	Features	Reason Chosen
NSwag	MIT	Code generator / Library	OpenAPI code generator for ASP.NET Core and TypeScript	Specifying internal and external Application Program Interfaces (API) in terms of specifications and then using code generators to create clients and server stubs greatly reduces the time spent implementing and debugging wire protocols. NSwag supports generating both ASP.NET Core and TypeScript bindings from a common specification, allowing the front-end and back-end to easily communicate with each other.
Logstash	Apache 2.0 / Elastic	Data flow	ingestor for ElasticSearch	Logstash can take data from many sources, transform and filter it, and pass it on to many other services - the primary one being ElasticSearch. Logstash was chosen for its integration with ElasticSearch.
Beats	Apache 2.0 / Elastic	Data flow	Data and metrics shipper	Beats are small applications that collect and send data over the network to various services. It's lighter-weight than Logstash and they are often used in conjunction. Beats would allow us to get logs and basic system metrics from systems without the memory and processor overhead of Logstash on machines that don't require the full capabilities of Logstash.
Kafka	Apache 2.0	Data flow	Distributed, persistent, streaming message queue	Kafka is a messaging system that works to ensure that every message sent to it is received by its intended recipients by saving events to disk and distributing them among multiple instances for high availability. It is used for fanning out events to multiple services and will be the primary carrier of data from the drivers to the rest of the system. The intent is to use the Cloudera distribution of Kafka.

Component	License	Type	Features	Reason Chosen
<u>Redis</u>	<u>3-Clause BSD</u>	Data store	In-memory key-value data structure store	Redis is a widely used, stable, fast key-value store often used to cache shared application state. In this case, the intent is to use it to store authorization data. Storing this data in Redis allows all requests for authorization to use an always up service with a well-defined API to do so instead of relying on a hand-rolled service with a custom API that will likely be updated more frequently. Additionally, Redis supports replication for high availability.
<u>MongoDB</u>	<u>AGPL3 or proprietary</u>	Data store	NoSQL database	A popular data store for document-oriented, semi-structured information. It is a high-performance data store that supports sharing and replication.
<u>ElasticSearch</u>	<u>Apache 2.0 / Elastic</u>	Data store	search engine	ElasticSearch is a tool for indexing and querying text-heavy information. It was selected it because it's supports semi-structured data such as logs, high availability with automatic recovery, and has a suite of tools for ingestion and visualization of data.
<u>HDFS</u> (part of Hadoop)	<u>Apache 2.0</u>	Data store	Distributed File System	The Hadoop Distributed File System (HDFS) is a scalable, distributed file system. All input data and some generated data, such as signal optimization runs, will be stored in HDFS. This will give future users of the system all the raw data that the system used to perform analytics and big data analyses. HDFS allows adding nodes to a running cluster, allowing the storage to scale without requiring the cluster to be restarted. The Cloudera distribution of Hadoop will be used, which includes HDFS.

Component	License	Type	Features	Reason Chosen
<u>Microsoft SQL Server</u>	<u>proprietary</u>	Data store	Relational Database System	Microsoft SQL (MS SQL) Server will be used for smaller volume, highly relational data. MS SQL Server will be used for SOT operation and inputs, permissions, agency and device configuration and defaults, a possible back-end for the ArcGIS database, managed events, and response plans. FDOT will provide a Microsoft SQL Server instance.
<u>ArcGIS Enterprise (Advanced License)</u>	<u>proprietary, one time, or subscription</u>	Data store	Geographic data store	Much of the data for the R-ICMS project has locations and geographical shapes associated with it, so the data will be stored in a data store with mapping and first-class support for queries on these properties. The Advanced license was chosen to enable the use of the GeoEvent server for real-time streaming geographic information.
<u>Kubernetes</u>	<u>Apache 2.0</u>	Deployment	Container orchestration for deployment, load balancing, hot failover, canary and rolling deployments	A container orchestrator handles scheduling and running containers across multiple computers and routing traffic among them. This simplifies management of applications running on clusters of computers. It works to ensure that applications stay running and will automatically restart them (on different machines if needed). It also supports rolling deployments to ensure uptime, rolling back deployments to previous states, and partial deployments to test stability of new code before doing a full roll out (canary deployments). These are all desirable features in a system that values high uptime. Kubernetes is a community developed system created by former Google engineers. Additionally, major cloud providers (Amazon Web Services (AWS), Google Cloud, Azure) all support Kubernetes clusters as a paid service.

Component	License	Type	Features	Reason Chosen
<u>ASP.NET Core</u>	<u>Apache 2.0</u>	Framework	Web framework for .NET Core	ASP.NET Core is the next version of the ASP.NET MVC framework. It runs on both .NET Core and .NET Framework. It boasts a more modular design than its predecessors and is actively developed.
<u>Angular</u>	<u>MIT</u>	Framework	Front-end development framework	Angular provides ways to build modular interfaces in the browser and encourages efficient and extensible data flow patterns within an application. It is popular and maintained by Google.
<u>System.IdentityModel.Tokens.Jwt</u>	<u>MIT</u>	Library	.NET library for creating and validating JavaScript Object Notation (JSON) web tokens	JSON Web Tokens (JWTs) will be used to store user authorizations. JWTs support storing data and are cryptographically signed, making them tamper-proof and safe to send to a user's device. JWTs also have standard fields for usernames and expiration dates.
<u>Serilog</u>	<u>Apache 2.0</u>	Library	Structured logging library for .NET Core	Logging is essential for debugging and metrics. Serilog supports multiple log levels and data sinks including console and files. It supports logging in JSON formats (that's the "structured" in "structured logging"), which simplifies ingestion via tools like Logstash.
<u>StackExchange.Redis</u>	<u>MIT</u>	Library	.NET library for interacting with Redis	This library is used to access Redis from a .NET application. It is maintained by Stack Exchange, the company that runs many large question and answer sites, including Stack Overflow, a well-known developer resource.
<u>MongoDB C# driver</u>	<u>Apache 2.0</u>	Library	.NET Core library for interacting with MongoDB	This library is used to access MongoDB from .NET Framework and .NET Core applications. We chose this driver because it is officially maintained and supported by MongoDB.

Component	License	Type	Features	Reason Chosen
<u>confluent-kafka-dotnet</u>	<u>Apache 2.0</u>	Library	.NET library for interacting with Kafka	This library is for interfacing with Kafka from .NET applications and supports all basic Kafka operations. This was selected because it appears to be the most up to date and maintained library for interfacing Kafka with .NET.
<u>SUSE Linux Enterprise Server (SLES)</u>	GNU General Public License and various	Operating system	Linux-based Operating System	Much of the software selected prefers running on a Linux operating system, and a few only run on Linux. SLES is one of the supported operating systems of Cloudera that has an option for paid support and supports running Docker and Kubernetes.
<u>Windows Server</u>	<u>proprietary</u>	Operating system	Windows operating system	Some of the software selected only runs on Windows. The primary one is HCS7. In order to run this software, a Windows server is required.
<u>Docker</u>	<u>Apache 2.0</u>	Packaging / Deployment	Container runtime for bundling and codifying application dependencies and running them in repeatable, isolated environments	Using containers makes dependency management simpler and deployment easier, as only the Docker runtime needs to be installed to run the applications. This simplifies system setup and configuration management, helps to be confident that the application behaves the same in multiple environments (development, testing, and production), ensures that there aren't missing dependencies required to run applications, and documents the creation of these environments in a source control friendly way. Containers are also used industry-wide; as an example, the major public clouds - Amazon Web Services (AWS), Google Cloud, and Microsoft Azure - all support running containers as a paid service.
<u>Cloudera</u>	<u>proprietary subscription</u>	Platform	Data warehousing software and support	Cloudera provides a managed and supported data warehousing and analytics platform based on Kafka and Hadoop with additional management features. Cloudera was chosen for its ease of administration and support.

Component	License	Type	Features	Reason Chosen
<u>.NET Core</u>	<u>MIT</u>	Runtime	.NET runtime	.NET Core is an open source runtime from Microsoft for .NET applications that runs on Windows, Linux, and macOS. It is different than .NET Framework, which is Windows only. (Both implement the .NET Standard; Framework includes additional Windows-specific libraries.) .NET Core was chosen because it is stable, developed by Microsoft, is open source and cross-platform.
<u>Kibana</u>	<u>Apache 2.0 / Elastic</u>	Visualization	dashboard and analytics on top of ElasticSearch	Kibana is a tool for querying and visualizing aggregations of data from ElasticSearch. Kibana was selected it for its integration with ElasticSearch.
<u>Tableau</u>	<u>proprietary subscription</u>	Visualization / Reporting	Dashboards and reporting	Tableau is primarily a tool for creating dashboards and exploring data stored in various data stores that can render a snapshot of a dashboard to PDF. Tableau was selected because it is the emerging standard for data visualization for business intelligence.
<u>Nginx</u>	<u>2-Clause BSD</u>	Web server	Web server for gateway for Transport Layer Security (TLS) termination and request routing/proxying	A widely deployed web server that supports TLS and reverse proxying of applications behind different routes. This will be the gateway through which all requests into the system pass.

3.3 System Architecture

The overall high-level architecture of the R-ICMS is shown in Figure 2 - R-ICMS High-Level Diagram. Figure 3 - Color Scheme shows the meaning of the colors in Figure 2 - R-ICMS High-Level Diagram. The architecture is depicted as a set of layers of types of functionality which are described below. Source systems are shown on the left. Updates to input data streams will be ingested through a set of drivers within the DFE. Those drivers will be responsible for maintaining and reporting the status of the link with the external system. R-ICMS will provide a common set of functionalities to support monitoring and logging of status information throughout the system. The drivers will also be used by the R-ICMS to send messages to SunGuide to request modifications to external systems as a part of the activation of response plans.

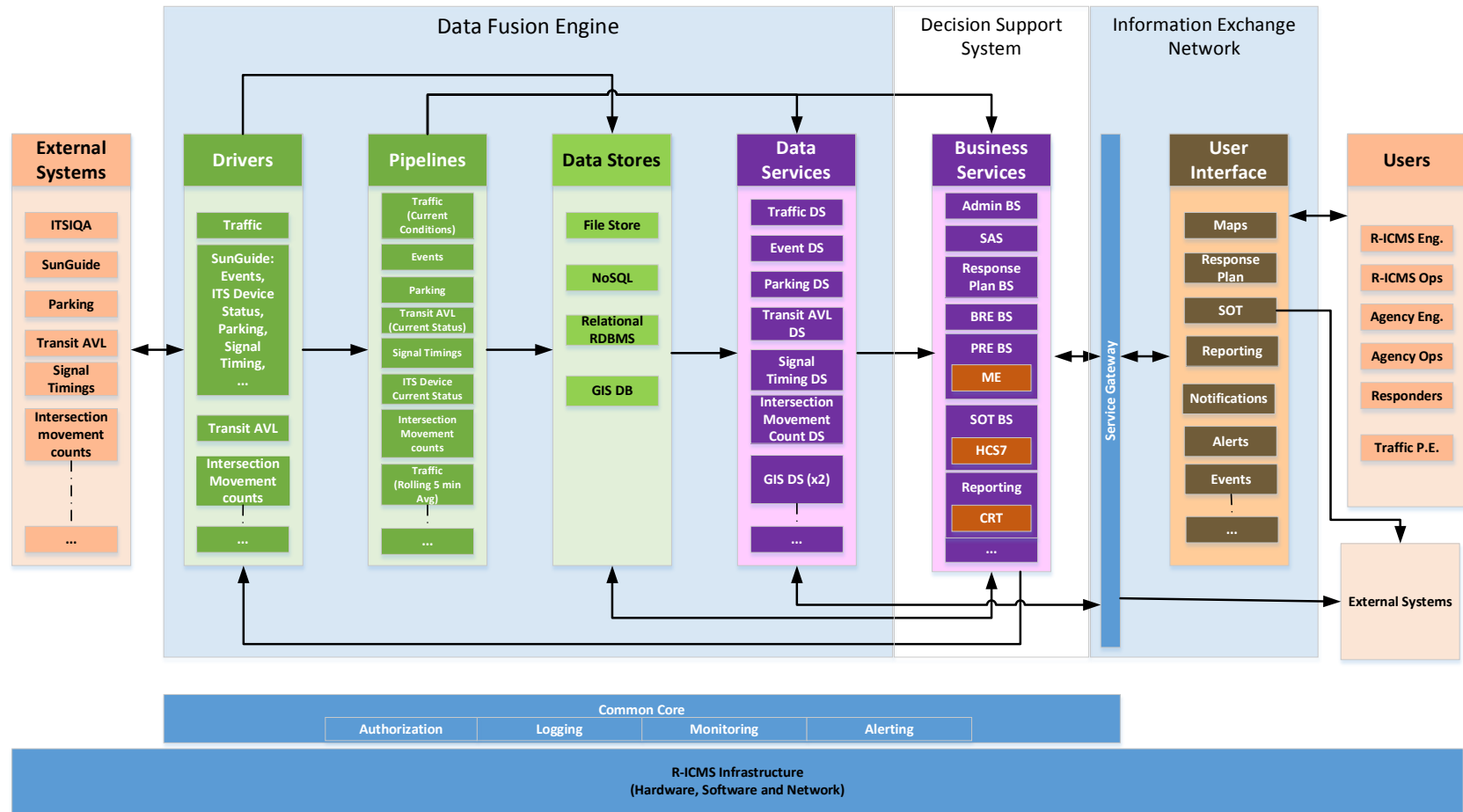


Figure 2 - R-ICMS High-Level Diagram

Streaming data will pass from the drivers into the pipeline layer of the architecture. The pipeline layer will be used to facilitate rapid throughput of streaming updates throughout the system. Streaming data pipelines will be consumed by data services, as well as the business services. The data services will publish the data to consumers who have been authenticated, authorized, and who have subscribed to the data. Business services will consume the data from the pipelines to make the real-time decisions needed to respond to events and evaluate the status of the traffic network. The streaming data pipelines will also be consumed by the data stores so that the incoming data is stored for later use and analysis.

Non-streaming data, otherwise known as batch, static or, more appropriately, slowly changing data, will not be entered through the pipelines. Non-streaming data will be entered into the system using processes executed by a system administrator. Where the data is available in files with standard formats and available via a standard protocol such as File Transport Protocol / Secure File Transport Protocol (FTP/SFTP), a standardized generated script process will be used to retrieve the data. Where the data can be acquired through standardized interfaces, those interfaces will be used. However, non-streaming data sources that do not have standard interfaces for collecting the data will require additional customization. Once the data has been collected, it will be placed into the data stores using scripts. The data services will access the data stores to provide data to business services.

Data will be stored in different types of data stores depending on its user, volume, age, and nature. Response plan elements, response plans, user actions, system recommendations, signal timing plans, and other transactional data will be stored in a MS SQL database. Streaming data, modeling results, and other large, complex sets of data will be stored in NoSQL databases such as HDFS, Mongo, or Elastic Search. Data to be used to display maps and the data on maps will be stored in an ArcGIS Database.

3.4 System Components

As described in the original invitation to negotiate, the R-ICMS consists of three Subsystems:

- The DFE, which provides a “big data” environment for the ingestion, collection transformation, storage and publishing of the data;
- The DSS, which provides the business layer for the R-ICMS
- The IEN, which provides the user interface for the R-ICMS.

The design architecture of the R-ICMS has been broken down into seven layers. Four of those layers consist primarily of functionality attributed to the DFE. The DFE layers include:

- Drivers, which handle the connections to external systems, will collect or receive data from external systems and send data to external systems.
- Pipelines, which provide access to and transformation of streaming data being ingested from external systems
- Data Stores, which provide persistence of data from both internal and external sources

- Data Services, which provide filtered access to data for authorized users and provide caching of data so that users can initialize access to data streams

The Business Services layer contains much of the functionality of the DSS and the User Interface layer contains much of the functionality of the IEN.

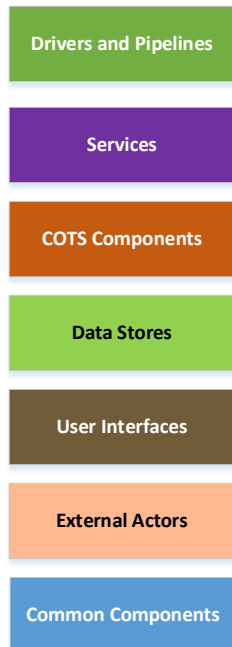


Figure 3 - Color Scheme

The colors in the high-level diagram are shown in Figure 3 - Color Scheme and indicate current expectation for the locations within the physical architectures of the layers and components illustrated. The layers and components shown in green, i.e., the Drivers, Pipelines, and Data Stores, are expected to reside in the “Big Data” environment. The layers and components shown in purple, i.e., the Data Services and the Business Services, are expected to reside in the standard FDOT services physical servers. The common components shown in blue are also expected to primarily reside in the standard FDOT services physical servers. The components shown in brown are expected to reside in the user’s systems.

As mentioned above, the DSS is made up of the Business Services layer. Likewise, the IEN is made up of the User Interface layer.

In addition to names used where they provide better understanding, R-ICMS components are assigned an identifier based on the name and the layer in which they reside. The suffix of the component IDs is based on the layer the component been assigned to as follows:

- Driver layer component names end in “-DR”
- Pipeline layer component names end in “-PL”
- Data Store layer components end in “-ST”
- Data Service layer components end in “-DS”
- Business Service Layer components end in “-BS”
- User Interface Layer components end in “-UI”
- Common Core components end in “-CC”

The following subsections list and describe the components identified for inclusion in the R-ICMS.

3.4.1 DFE Layers and Components

Several specific, common big data design patterns will be used in the design and implementation of the DFE including the following:

- Multisource Extractor Pattern is used to ingest multiple data source types in an efficient manner

- Multi-destination Pattern is used to transport data to multiple components like HDFS, GIS, IN-memory data service (for real-time), NoSQL
- Real-Time Streaming Pattern is used for real-time streaming of data to the web user interface
- Just-in-Time Transformation Pattern is used to upload large files (GIS Shape files)/large quantities of data in batch mode or using traditional Extract, Transform, Load (ETL) methods only when required to save compute time.

The following subsections list the components in each of the layers of the DFE.

3.4.1.1 Drivers

The following components are assigned to the Driver layer of the architecture. Drivers are responsible for initiating, monitoring, and maintaining links to external systems. The drivers will be responsible for adding the “Received Time” to data received by the R-ICMS. If a connection to an external system is lost, the driver is responsible for reconnecting to the external system and, where feasible, requesting data missed while the connection was down. The drivers are also responsible for informing the monitoring service of connection status.

3.4.1.1.1 Driver Structure:

Data ingestion is the process of extracting data from external systems and loading the data into a data warehouse environment (aka ETL). The Driver performs the first step in ingestion. The most common steps involved in the ingestion phase are:

1. Establish a connection to the external data source (web service, FTP site, SQL database etc.)
2. Once the connection is successful, the process will make a request for the intended data. The method will vary depending on the source type (API Call, FTP get request, SQL query etc.)
3. The data will then get transferred to the processing system where the process will make additional inspection / adjustments / processing to make it more meaningful based on the intended target system usage.
 - a. Identification – Identification of various data source formats (identified during the design phase) will be handled at this step
 - b. Filtration – Remove any data that is not required. Sometimes, the sources provide more data than is required for the application and the extra information needs to be filtered at the very first level.
 - c. Validation– Identify and removal of any ill-formatted data which will cause exceptions downstream for the applications will be handled in this step. This will be done by validating the data values and data types coming from the source.
 - d. Transformation – This step will be used when the external source data format is not the intended format for the destination. Transformation will happen at the

entry level to suit the intended destinations. Also, same data will be structured in different ways depending on the needs of the R-ICMS application.

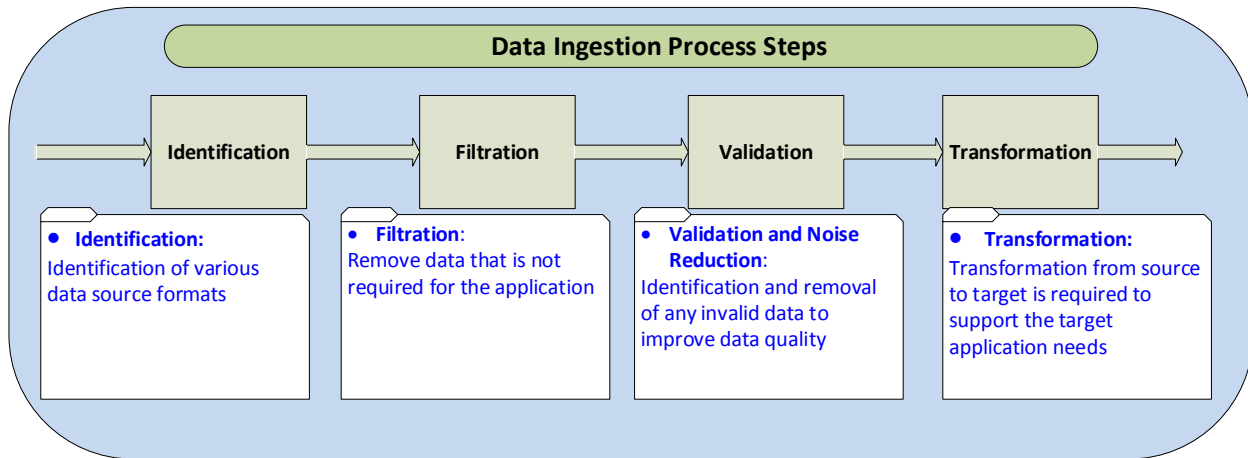


Figure 4 – Ingestion Flow

- When data is received/retrieved and when it is passed to other layers, a log entry is generated using the common logging service including begin, end timestamps, elapsed time, status of the process, and data size / volume. This information is needed for picking up from where the previous processes left off, as well as providing vital statistics about the data processed and resource usage.
- The process will also incorporate retry / recovery options when exceptions are encountered and will log such incidents through the logging mechanism.
- All these process steps will be encapsulated into script/(s) and the script execution will be automated using a job scheduler at pre-defined intervals depending on the data availability frequency.

In addition to the above steps, a separate process will be established to send notifications to the configured users about job failure. The process will be driven by a configuration mechanism mapping the data load jobs and the users to be notified. Metadata with the job information will be stored in the NoSQL to help map notifications with the intended users.

3.4.1.1.1.1 Configuration Based Design:

The R-ICMS data ingestion process will adopt the configuration-based approach which will facilitate much simpler, reusable, efficient and better maintainable code which follows the suite of best practices methodology. This approach encapsulates defining the required configuration parameters to accomplish the entire process of fetching data, storing and distributing to other processes. Each individual data source will have a manually entered and maintained configuration file consisting of parameters for various functions (Ex. source location, availability method, and credentials required, HDFS storage location, NoSQL storage location etc.). The main advantages of this approach is minimal code changes while parameters can be altered without impacting the code, which also benefits in less code builds and deployments. This approach is dependent on the following items.

- Define one configuration per individual data source with all necessary parameters required.
- Store the configuration in the NoSQL database using the JSON document model.
- Build a common utility script to fetch the configuration for the source.
- Create initial configuration, build scripts and make them part of code deployment.

3.4.1.1.1.2 Ingestion Methods

R-ICMS ingestion methods can be classified as either FTP/SFTP, API, or TCP socket based (SunGuide Databus). The process for ingesting data for either method follow the following steps.

1. Invoke the script with the data source name
2. Fetch the configuration for the data source from NoSQL database using the common utility function
3. Using the retrieved configuration data, access the data source (either API-call or FTP/SFTP)
4. Perform any specific processing related to that data set.
5. Call the common function to store the data into HDFS file system which will accomplish storing it in HDFS
6. Call the common function (or define its own process specific to the data set) to store the data into NoSQL database
7. If the data set is intended to go to the MS SQL database, an equivalent storage function will be created and utilized
8. Call the logging function (see below) to log the process; relevant information will be specified in the configuration
9. For errors, call the notification function to send notifications; relevant information will be specified in the configuration

The system will generate a log each time a data ingestion process is executed. The logs will be stored in the NoSQL database. Log entries will contain the following parameters:

- Data source and data type / sub-set (Build a list of all the data source and sub-set data types and make it as part of metadata)
- Process name (Reference the process name as part of the metadata referenced above)
- Process execution date and timestamp
- Process begin, end timestamps and time elapsed
- Volume (size) of the data transferred
- Number of records processed where applicable
- Process status (Success / Failure)
- Errors encountered during execution (Process exceptions)
- Errors received from the source when requesting data

These parameters shall provide the base for system performance analytics.

3.4.1.1.1.2.1 FTP/SFTP process for static data:

One of the common types of data transport is through FTP/SFTP. When data is available as static files which are made available at regular intervals, the FTP/SFTP process is the most suitable

methodology to get the data from the source location. To use the FTP/SFTP process the following criteria should be true for the data source.

- The source location where the file is located should be fixed
- The file should be available in the same location at all times
- The target location where the file will be stored should be fixed
- Credentials to the source system shall be provided if authentication is required

The FTP/SFTP process will incorporate a script to transfer the intended file from the source location to the target destination. The script will be generic in nature and will drive the process using a pre-defined configuration for the file transfer jobs. The same common processes can be reused for other FTP/SFTP jobs as well. The process will carry out the following instructions:

1. Read the configuration from the NoSQL metadata to get the details for the transfer (Source location, target location, login credentials)
2. Establish a connection to the source location
3. Perform authentication when required
4. Retrieve the file details (name and timestamp)
5. Retrieve the last retrieval details (date and timestamp) from the target location
6. Verify if the source file is newer
7. If the file is newer, continue with the transfer to the target location
8. Call the logging function to log the process; relevant information will be specified in the configuration (Begin and end times, elapsed time, volume of data transferred, volume of records where applicable, status and any exceptions / errors encountered)
9. If the file is not newer, call the logging function add a log entry for the job execution with appropriate status (This entry is needed for identifying any missing data when a scheduled job did not get the configured data)
10. If the file is supposed to contain periodic data and there are gaps in the data that meet the identified criteria, log the gap and call the notification function to send notifications
11. For errors, call the notification function to send notifications; relevant information will be specified in the configuration
12. Exit the process
13. Schedule a job to process the specific data file at configured intervals

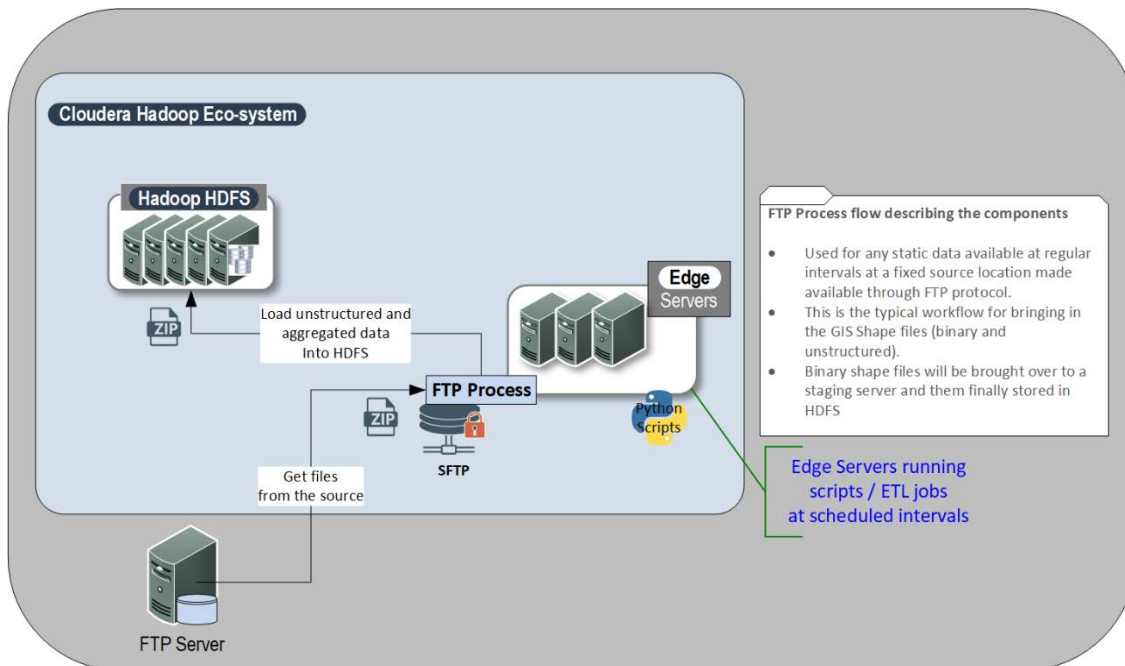


Figure 5 – FTP Workflow

3.4.1.1.1.2.2 API / Web Service Calls:

Another common type of data ingestion process is through API calls. When the data is made available through web services / API calls, the ingestion will adopt this method to fetch data from the external systems. This section describes the common applicable methodology, but the process will be different for each individual data source depending on the type / format of the data. A common utility will be used for making API calls and will wrap the functionality for each specific data source to process and distribute the data accordingly. In addition, more common functions can be defined depending on the context and need.

Design and create a script that can fetch configuration parameters, call the API, and return the content of the data received. The script will be a common function and can be used anywhere when data needs to be brought in through API calls.

Once the data is fetched from the source, the following steps will take place:

1. Validate the data for any missing data elements and add them if necessary – Ex. Update date and timestamp
2. Publish the data to the appropriate pipeline layer component (Kafka streaming server) for further distribution
3. When there are exceptions that the data doesn't need to go through the pipeline, the process for that individual dataset will extend to handle further processing and storage
4. Call the logging function to log the process

3.4.1.1.1.2.3 SunGuide Databus:

The RICMS will communicate with SunGuide to receive SunGuide related data including DMS, Events, CCTV, etc. Communication will occur over a single TCP socket connection with the

SunGuide Databus subsystem. Configuration data for specific data types (such as DMS) will be requested and subscribed from the representative subsystem through communication with Databus. Status data will be collected directly from Databus via status update messages. To receive data from the Databus, the RICMS is responsible for executing the following series of steps.

1. Initiate a socket connection to SunGuide Databus.
2. Authenticate to appropriate subsystems (DMS, CCTV, etc).
3. Subscribe to receive configuration data from the appropriate subsystems.
4. Retrieve data from appropriate subsystems to get current configuration and status information.
5. Subscribe to Databus to receive status information for appropriate subsystems.
6. From that point onwards, any updates from SunGuide will be pushed to the client which will then be forwarded to the appropriate destinations.
7. The process will also incorporate a mechanism to re-establish the connection in case of connectivity loss, performing necessary authentication, restart the subscription and continue to receive the data.
8. After receiving the data / updates, the client process will redirect the data flow to the appropriate destinations per the design.

Below are the steps describing the ingestion process implementation in general. There might be additional steps specific to individual data types and are documented with the respective data type ingestion process where applicable.

1. Build a process encapsulating the steps outlined above.
2. The driver / client process will also encapsulate the mechanism to re-establish the connectivity to Databus, re-authenticate, and resume data reception activity.
3. Each time a data update is received from the server, performs any additional process specific to the data set (Ex. validations / filtering etc.).
4. Publish the data to the Pipeline / streaming application (Kafka) after transforming to intended format. Each data set will have one or more Kafka topics (message queues) designed and the data forwarding will be aligned with the respective data set / topic(s)
 - a. Down the process flow, another process (Consumer) will retrieve the data from the streaming pipeline and sends to the NoSQL database storage.
5. Store a copy of the data received onto the HDFS.
 - a. Create an additional process to read the data files from the HDFS, consolidate / aggregate the data into a suitable format and store in the HDFS.
 - b. Remove the smaller XML files from HDFS
6. Add a log entry in to the log database using the common logging framework (Function or API).
7. This client process is deployed to run in background so that it will continuously receive the data updates from SunGuide and redirect the flow appropriately.

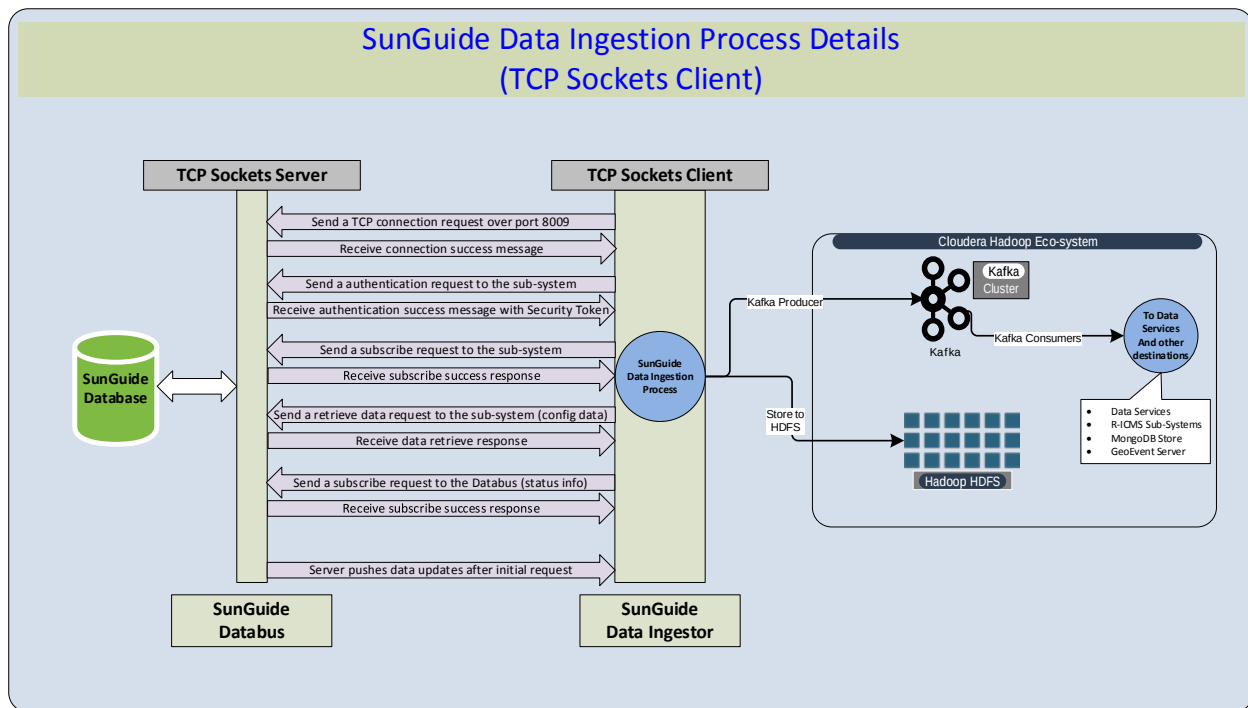


Figure 6 – SunGuide Data Ingestion Process

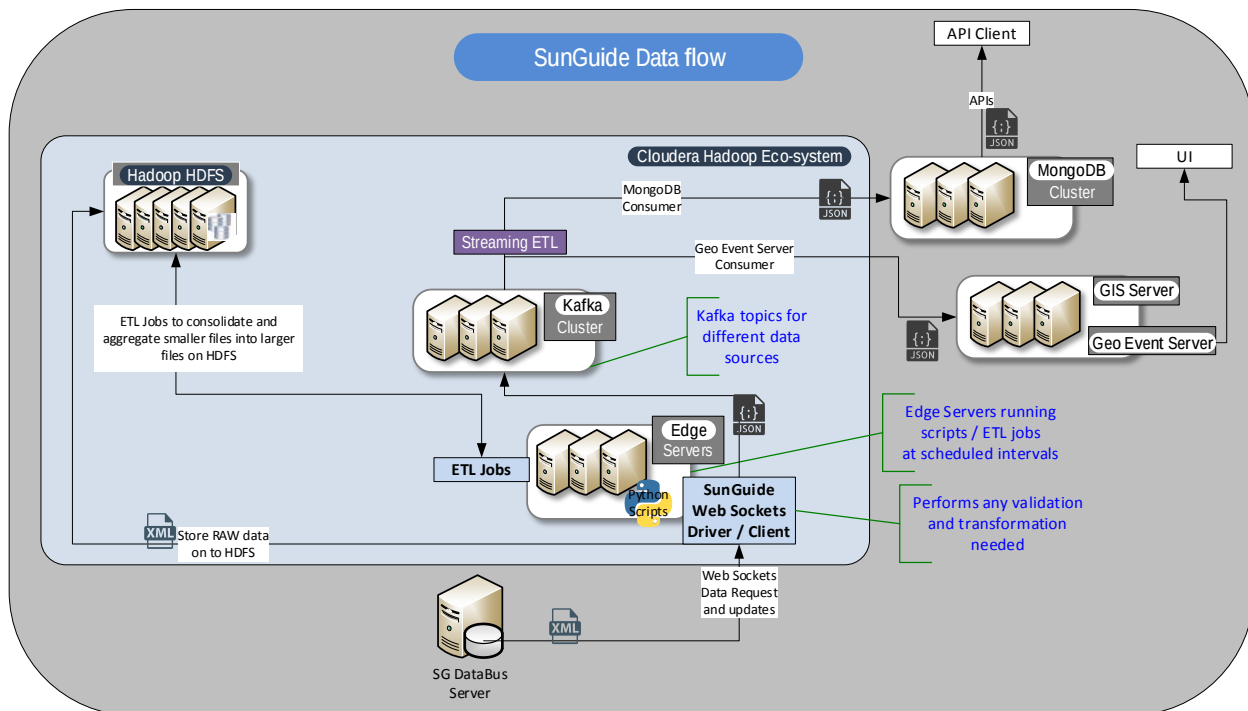


Figure 7 – SunGuide Data Flow

3.4.1.1.2 ~~Traffic (TRF-DR)~~

ITSIQA (Intelligent Transportation Systems Integration Quality and Analysis) is the source system for Traffic Conditions and is available from FDOT D5. The ITSQA system collects the data from ITS devices as well as utilizes commercially available traffic data from HERE / WAZE etc. to validate the integrity of the conditions data and publishes the data for Traffic Operations, analysis, and other uses.

ITSQA traffic conditions data is available through REpresentational State Transfer (REST) API calls from ITSQA processing server and available via the following streams.

- ~~Link Configuration~~ — Link configuration details
- ~~Traffic Conditions~~ — Dynamic traffic conditions including speed, volume, occupancy and travel times per link.
- ~~Classification Data~~ — Dynamic conditions of vehicle volumes by vehicle class.

Table 3 — ITSQA Ingestion

Data-Set Name	ITSQA Traffic Conditions
Source location	http://10.32.0.80:80/ITSQA/
API URLs	LinkConfig — http://10.32.0.80:80/ITSQA/LinkConfig-ITSQA-AllSources.xml TrafficData — http://10.32.0.80:80/ITSQA/TrafficData-ITSQA-AllSources.xml ClassificationData — http://10.32.0.80:80/ITSQA/ClassData-ITSQA-AllSources.xml
Source Format	Extensible Markup Language (XML)
Target Location	HDFS, Streaming pipeline and NoSQL database HDFS location: /user/dev/traffic_conditions/<year>/<month>/<day>/<feed_name>_<date>_<time>.xml Mongo-DB: Database: traffic_conditions Collections: link_config traffic_data classification_data
Target Format	JSON — MongoDB XML — HDFS (initially RAW XML files will be stored in HDFS. But storage of small size files is not efficient in HDFS and a process to merge and consolidate to larger files will be designed during the iteration — 1) JSON — Streaming for Geo Event Server
Data load mechanism into R-ICMS	Create script(s) which can make API calls to fetch data from the ITSQA feeds, store a copy of the XML files to the HDFS, load into NoSQL after converting to JSON format and publish to the Streaming Server. Automate the script execution at the intervals aligned with the source intervals using a job scheduler to load the data continuously.
Frequency	Link Configuration — Once every 24 hours Traffic Conditions — every 60 seconds Classification data — every 60 seconds
Process Notifications	<TBD: In later iteration>
Logging	The job execution history will be stored in the MongoDB database.

Notes	Currently, Classification Data is being published as a separate feed associated with Link Identifier (ID). If the classification data is going to be used for traffic analysis / evaluation by business services along with the speed / volume provided by TrafficData feed, then it will make more sense to combine this information into traffic data as a single message. This can be addressed when the data request requirements are clearly defined.
--------------	--

3.4.1.1.2.1 Ingestion process:

A script will be created with the data source name as the script name and will encapsulate the following processes:

1. Fetch the configuration specific to this data source from NoSQL using a common function
2. Use the common API call function to fetch the data from source location with the help of configuration parameters
3. Perform any additional process specific to the data set (Ex. validations / filtering etc.)
4. Publish the data to the Pipeline / streaming application (Kafka) after transforming to intended format
 - a. Down the process flow, another process will retrieve the data from the streaming pipeline and send to the NoSQL database storage
5. Store a copy of the data received onto the HDFS
 - a. Create an additional process to read the data files from the HDFS, consolidate / aggregate the data into a suitable format and store in the HDFS
 - b. Remove the smaller XML files from HDFS
6. Add a log entry in to the log database using the common logging framework (Function or API)
7. Schedule the jobs to run at frequent intervals using job scheduler
 - a. LinkConfig—Schedule to run once every 24 hours
 - b. TrafficData—Schedule to run once every 60 seconds
 - c. Classification Data—Schedule to run once every 60 seconds
 - d. Load into HDFS from staging server (consolidate and aggregate at specific intervals) —once a day

3.4.1.1.2.2 Data Model:

The ITSQA source provides the data in XML format for the feeds and is available in the data dictionary format published on the R-ICMS SharePoint portal hosted by FDOT (embedded in OneNote names ITSQA under "ICMS Notebook").

Data coming from external sources will be stored in a NoSQL MongoDB database in addition to publishing to streaming pipelines. The ITSQA Traffic Conditions data will be stored in MongoDB using a document based model described below.

MongoDB Storage Model:

JSON document model for LinkConfig data:

Database Name: itsqa

Collection Name: link_config

Note: Per the data dictionary, "Link_type" attribute does not currently exist, but this is coming in the feed. Need clarification on the validity of the data dictionary.

```

    "_id": {
      "link_id": "", /* String */
      "update_timestamp": "" /* DateTime */
    },
    "road": "", /* String */
    "direction": "", /* String */
    "cross_street": "", /* String */
    "county": "", /* String */
    "lane_count": , /* Integer */
    "speed_limit": , /* Integer */
    "length": , /* Float */
    "link_type": "", /* String */
    "start_location": { /* Sub-document */
      "latitude": , /* Double */
      "longitude": /* Double */
    },
    "end_location": { /* Sub-document */
      "latitude": , /* Double */
      "longitude": /* Double */
    },
    "mid_points": [ /* Array of sub-documents */
      {
        "latitude": , /* Double */
        "longitude": /* Double */
      }
    ],
    "upstream_link": "", /* String */
    "downstream_link": "" /* String */
  }

```

Indexes:

Default unique index on
_id.link_id and _id.update_timestamp

Additional indexes on
county
road

JSON model for TrafficData:

Database Name: itsiq

Collection Name: traffic_conditions

```

{
  "_id": {
    "link_id": "", /* String */
    "update_timestamp": "" /* DateTime */
  },
  "speed": , /* Integer */
  "speed_data_quality": , /* Integer */
  "volume": , /* Integer */
  "volume_data_quality": , /* Integer */
  "occupancy": , /* Integer */
  "occupancy_data_quality": , /* Integer */
}

```

```

    "travel_time": , /* Integer */
    "travel_time_data_quality": /* Integer */
  }

```

Indexes:

Default unique index on
_id.Link_id and _id.update_timestamp

JSON document model for ClassData:

Database Name: itsiq

Collection Name: classification

```

{
  "_id": {
    "Link_id": "", /* String */
    "update_timestamp": "" /* DateTime */
  },
  "class1": , /* Integer */
  "class2": , /* Integer */
  "class3": , /* Integer */
  "class4": , /* Integer */
  "class5": , /* Integer */
  "class6": , /* Integer */
  "class7": , /* Integer */
  "class8": , /* Integer */
}

```

Indexes:

Default unique index on
_id.Link_id and _id.update_timestamp

In addition to the feed specific collections, merged storage will support queries for the applications.

JSON document model for traffic data with link config:

Database Name: traffic_conditions

Collection Name: traffic_data_with_link_config

```

{
  "_id": {
    "Link_id": "", /* String */
    "update_timestamp": "" /* DateTime */
  },
  "dateparts": {
    "date": "", /* Date */
    "year": , /* Integer */
    "month": , /* Integer 1-12 */
    "day": , /* Integer 1-31 */
    "weekday": , /* Integer 1-7 (1 = Sunday) */
    "hour": , /* Integer 0-24 */
    "minute": , /* Integer 0-59 */
  },
  "speed": , /* Integer */
  "speed_data_quality": , /* Integer */
  "volume": , /* Integer */
}

```

```
"volume_data_quality": , /* Integer */
"occupancy": , /* Integer */
"occupancy_data_quality": , /* Integer */
"travel_time": , /* Integer */
"travel_time_data_quality": , /* Integer */
"Link_config": {
  "road": "", /* String */
  "direction": "", /* String */
  "cross_street": "", /* String */
  "county": "", /* String */
  "lane_count": , /* Integer */
  "speed_limit": , /* Integer */
  "length": , /* Float */
  "link_type": "", /* String */
  "start_location": { /* Sub-document */
    "latitude": , /* Double */
    "longitude": /* Double */
  },
  "end_location": { /* Sub-document */
    "latitude": , /* Double */
    "longitude": /* Double */
  },
  "mid_points": [ /* Array of sub-documents */
    {
      "latitude": , /* Double */
      "longitude": /* Double */
    }
  ],
  "upstream_link": "", /* String */
  "downstream_link": "" /* String */
}
```

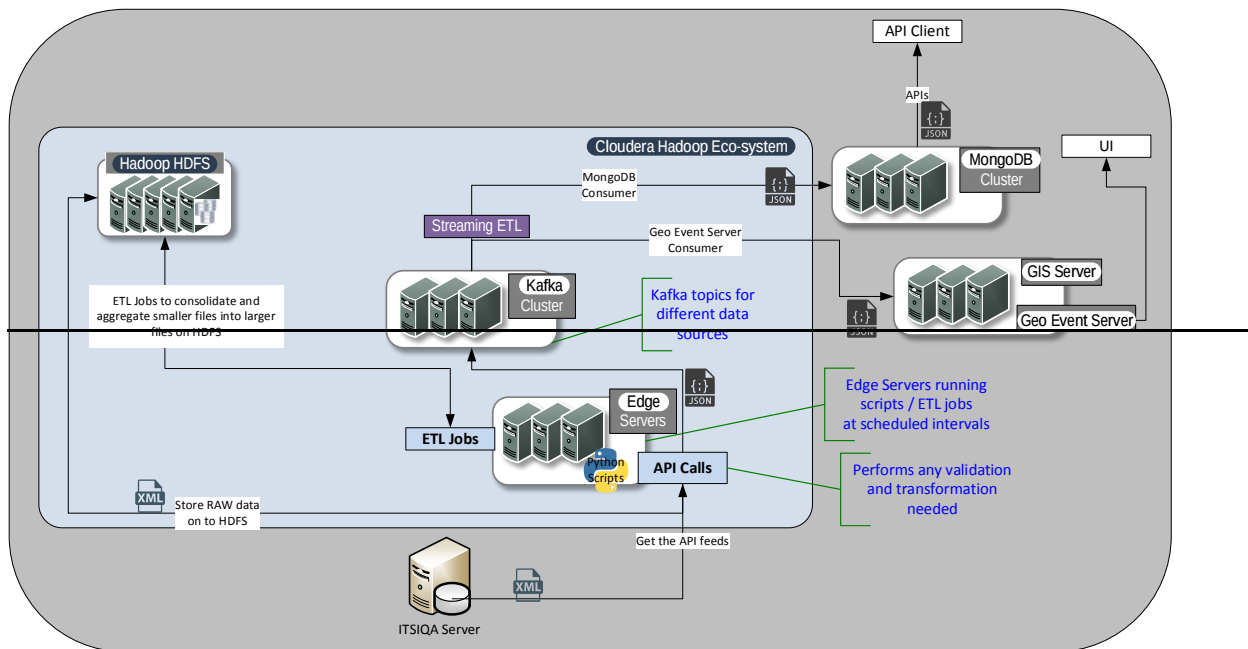


Figure 8 — ITSQA Data Flow

3.4.1.1.3 SunGuide (SG-DR)

A generic SunGuide (see the SunGuide reference documents in Section 2 Reference Documents) driver will provide access through the DataBus (a generic SunGuide interface) to streaming data sourced from SunGuide. The streaming data sources received from SunGuide include:

- Events
- Cameras
- Ramp Meters
- Dynamic Message Signs
- Connected Vehicle Roadside Units
- Origin / Destination
- Traffic Signal Status
- Express Lanes
- Parking Data

The overall SunGuide driver will be responsible for maintaining the connection with SunGuide as well as managing the connections to each of the subsystems within SunGuide from which data will be requested. The driver will be responsible for reconnecting, authenticating, and subscribing to each subsystem when the overall SunGuide connection is lost or when any individual subsystem is started/restarted.

Additionally, the SunGuide driver will also be used to send response plans to the District 5 SunGuide Incident Detection Subsystem for activation. Information received from SunGuide regarding response plan activation will be used to relate SunGuide events to R-ICMS events.

Multiple instances of the SunGuide driver may be used to interact with multiple SunGuide instances, as well as, potentially, to separate data collection from response plan activation.

Interface Control Documents (ICD) for SunGuide drivers and other SunGuide documents are available at: <http://sunguidesoftware.com/document-library>.

3.4.1.1.3.1 Dynamic Message Signs DMS (SG-DMS-DR)

DMS (Dynamic Message Signs) information is one of the data streams provided by the SunGuide Databus interface. DMSs are the electronic display boards placed across the roadways which display information about traffic and alerts. DMSs provide important information enabling travelers to make informed decisions about their commute (Travel times, road closure/ detour notifications, crash notifications etc.) as well as other emergency alerts. These devices include attributes like location (roadway, latitude, longitude, etc.), display characteristics and so on. Ingested data will include configuration data (location, description etc.) as well as status data (changing messages). While the configuration details are used for building a map layer, the status data for those DMS devices will show the current message and status displayed at that time on the map.

The generalized process details for the SunGuide data ingestion are documented under section 3.4.1.1.2.3 SunGuide Databus.

When a request is made to SunGuide for the data stream for the first time, it will provide the complete current state of the DMS devices including the configuration and current status. The subsequent updates are pushed to the client process within the subscription mechanism.

Table 4 – SunGuide DMS Ingestion

Data Set Name		SunGuide DMS (Dynamic Message Signs)	
Source location	SunGuide Server: 10.32.0.138 Port# 8009		
API URLs	N/A		
Source Format	XML		
Source ICD Location	ICD Document location (version 6.2): http://sunguidesoftware.com/sunguidesoftware/documentlibrary/ICD/6_2/SunGuide-General-ICD-6.2.pdf Schemas location: http://sunguidesoftware.com/sunguidesoftware/documentlibrary/ICD/7_1/SunGuide%207.1%20Schemas.zip Locations of the schemas within the ZIP folder		
	Schema	Folder Location	
	dms - authenticateReq	common/requests	
	dms - authenticateResp	common/responses	
	dms - subscribeReq	dms/requests	
	dms - subscribeResp dms - addDmsResp dms - modifyDmsResp dms - deleteDmsResp	dms/responses	

	dms - retrieveDataReq		dms/requests	
	dms - retrieveDataResp		dms/responses	
	databus – subscribeReq		databus/requests	
	databus – subscribeResp databus – statusUpdateMsg		databus/responses	
SunGuide Request / Responses	Subsystem	Request	Response	Parameters
	dms	authenticateReq	authenticateResp	refId = refId1 icdVersion = 1.0 providerName = dms username = <username> password = <password>
	dms	subscribeReq	subscribeResp addDmsResp modifyDmsResp deleteDmsResp	refId = refId1 icdVersion = 1.0 providerName = dms username = <username> securityToken = <securityToken> dmsData = true
	dms	retrieveDataReq	retrieveDataResp	refId = refId1 icdVersion = 1.0 username = <username> securityToken = <securityToken> dmsList = true
	databus	subscribeReq	subscribeResp statusUpdateMsg	refId = refId1 icdVersion = 1.0 providerName = databus dataReq = dms
Target Location	• HDFS, Streaming pipeline and NOSQL database • HDFS location: /user/dev/sunguide/dms/<year>/<month>/<day>/<feed_name>_<date>_<time>.xml • Mongo DB: - o Database: sunguide - o Collections: ▪ sunguide_dms_data ▪ sunguide_dms_status ▪ sunguide_dms_current			
Target Format	• JSON - MongoDB • XML - HDFS (initially RAW XML files will be stored in HDFS. But storage of small size files is not efficient in HDFS and a process to merge and consolidate to larger files will be designed during the iteration - 1) • JSON – Streaming for Geo Event Server			
Data load mechanism into R-ICMS	A TCP Sockets client process (SunGuide Driver) will be created and deployed onto the Edge server within the Cloudera Hadoop Eco-system. The client process / driver will establish the connection to the server, authenticate, subscribe to the data feed, receive initial data and subsequent updates. The client process will also redirect the incoming data and updates to the appropriate destinations per design (HDFS, Kafka Streaming, GeoEvent Server etc.)			
Frequency	Initial request sends all current data			

	Updates are pushed when status changes or when signs are polled by SunGuide Databus.
Process Notifications	<TBD: In later iteration>
Logging	All data receiving events will be logged into the NoSQL database with details about the time, and data being received.
Notes	Since there are multiple data sets from SunGuide that need to be tapped into, a single driver / client process will be deployed which will subscribe to all data streams that are intended to be brought over to R-ICMS. Each data set will be redirected to the appropriate destination per the flow design.

The source data attributes are available as part of SunGuide ICD documentation which are located at <http://sunguidesoftware.com/document-library>

3.4.1.1.3.2 Ingestion process:

To bring data from SunGuide to the R-ICMS system, the system will deploy a SunGuide Databus driver / client process with the help of a script, which will accomplish the process referenced in section - 3.4.1.1.1.2.3

3.4.1.1.3.3 Data Model:

Data coming from external feeds will be stored in the MongoDB NoSQL database in addition to publishing to streaming pipelines. The SunGuide DMS data will be stored in MongoDB using a document-based model described below.

MongoDB Storage Model:

JSON document model for DMS data (Schema definition):

The schemas are represented as JSON schemas

Database Name: sunguide

Collection Name:

1. sunguide_dms_data – This will store the DMS device config information history

```
{
  "$id": "http://example.com/example.json",
  "type": "object",
  "definitions": {},
  "$schema": "http://json-schema.org/draft-07/schema#",
  "properties": {
    "_id": {
      "$id": "/properties/_id",
      "type": "object",
      "properties": {
        "dms_id": {
          "$id": "/properties/_id/properties/dms_id",
          "type": "integer"
        },
        "update_timestamp": {
          "$id": "/properties/_id/properties/update_timestamp",
          "type": "string",
          "description": "Date and time when dmsData data was received from sunguide"
        }
      }
    },
    "dms_comm": {
      "$id": "/properties/dms_comm",
```

```

        "type": "object",
        "properties": {
            "dms_address": {
                "$id": "/properties/dms_comm/properties/dms_address",
                "type": "object",
                "properties": {
                    "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address",
                    "type": "object",
                    "properties": {
                        "port_server_udp_address": {
                            "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address",
                            "type": "object",
                            "properties": {
                                "address": {
                                    "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address/properties/address",
                                    "type": "integer"
                                },
                                "port_server_ip": {
                                    "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address/properties/port_server_ip",
                                    "type": "string"
                                },
                                "port_server_port_number": {
                                    "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address/properties/port_server_port_number",
                                    "type": "integer"
                                }
                            }
                        }
                    }
                }
            },
            "protocol": {
                "$id": "/properties/dms_comm/properties/protocol",
                "type": "string"
            },
            "poll_process_name": {
                "$id": "/properties/dms_comm/properties/poll_process_name",
                "type": "string"
            },
            "packet_timeout": {
                "$id": "/properties/dms_comm/properties/packet_timeout",
                "type": "integer"
            },
            "packet_retry_limit": {
                "$id": "/properties/dms_comm/properties/packet_retry_limit",
                "type": "integer"
            },
            "command_retry_limit": {
                "$id": "/properties/dms_comm/properties/command_retry_limit",
                "type": "integer"
            },
            "poll_cycle": {
                "$id": "/properties/dms_comm/properties/poll_cycle",
                "type": "integer"
            },
            "op_status": {
                "$id": "/properties/dms_comm/properties/op_status",
                "type": "integer"
            }
        }
    }
}

```

```

    },
    "dms_config": {
      "$id": "/properties/dms_config",
      "type": "object",
      "properties": {
        "dms_type": {
          "$id": "/properties/dms_config/properties/dms_type",
          "type": "string"
        },
        "manufacturer": {
          "$id": "/properties/dms_config/properties/manufacturer",
          "type": "string"
        },
        "lines": {
          "$id": "/properties/dms_config/properties/lines",
          "type": "integer"
        },
        "columns": {
          "$id": "/properties/dms_config/properties/columns",
          "type": "integer"
        },
        "has_beacons": {
          "$id": "/properties/dms_config/properties/has_beacons",
          "type": "boolean"
        },
        "brightness_level_day": {
          "$id": "/properties/dms_config/properties/brightness_level_day",
          "type": "object",
          "properties": {
            "brightness_level": {
              "$id":
"/properties/dms_config/properties/brightness_level_day/properties/brightness_level",
              "type": "integer"
            }
          }
        },
        "brightness_level_night": {
          "$id": "/properties/dms_config/properties/brightness_level_night",
          "type": "object",
          "properties": {
            "brightness_level": {
              "$id":
"/properties/dms_config/properties/brightness_level_night/properties/brightness_level",
              "type": "integer"
            }
          }
        },
        "location": {
          "$id": "/properties/dms_config/properties/location",
          "type": "object",
          "properties": {
            "description": {
              "$id":
"/properties/dms_config/properties/location/properties/description",
              "type": "string"
            },
            "roadway": {
              "$id":
"/properties/dms_config/properties/location/properties/roadway",
              "type": "string"
            },
            "direction": {
              "$id":
"/properties/dms_config/properties/location/properties/direction",
              "type": "string"
            },
            "latitude": {
              "$id":

```

```

"/properties/dms_config/properties/location/properties/latitude",
    "type": "number"
  },
  "longitude": {
    "$id":
"/properties/dms_config/properties/location/properties/longitude",
    "type": "number"
  }
},
"sign_type": {
  "$id": "/properties/dms_config/properties/sign_type",
  "type": "string"
},
"font_id": {
  "$id": "/properties/dms_config/properties/font_id",
  "type": "object",
  "properties": {
    "$id": "/properties/dms_config/properties/font_id/properties/id",
    "type": "object",
    "properties": {
      "id": {
        "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/id",
        "type": "integer"
      },
      "provider_name": {
        "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/provider_name",
        "type": "string"
      },
      "resource_type": {
        "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/resource_type",
        "type": "string"
      },
      "center_id": {
        "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/center_id",
        "type": "string"
      }
    }
  }
},
"font_type": {
  "$id": "/properties/dms_config/properties/font_type",
  "type": "string"
},
"sign_use": {
  "$id": "/properties/dms_config/properties/sign_use",
  "type": "string"
},
"can_publish": {
  "$id": "/properties/dms_config/properties/can_publish",
  "type": "boolean"
},
"color_support": {
  "$id": "/properties/dms_config/properties/color_support",
  "type": "boolean"
},
"graphic_support": {
  "$id": "/properties/dms_config/properties/graphic_support",
  "type": "boolean"
},
"default_auto_merge": {
  "$id": "/properties/dms_config/properties/default_auto_merge",
  "type": "boolean"
}
}

```

```

    }
  }
}

```

2. sunguide_dms_status – This will store the DMS message status and update history

```

{
  "$id": "http://example.com/example.json",
  "type": "object",
  "definitions": {},
  "$schema": "http://json-schema.org/draft-07/schema#",
  "properties": {
    "_id": {
      "$id": "/properties/_id",
      "type": "object",
      "properties": {
        "dms_id": {
          "$id": "/properties/_id/properties/dms_id",
          "type": "integer"
        },
        "update_timestamp": {
          "$id": "/properties/_id/properties/update_timestamp",
          "type": "string",
          "description": "Date and time when message was received from sunguide"
        }
      }
    },
    "op_status": {
      "$id": "/properties/op_status",
      "type": "string"
    },
    "control_mode": {
      "$id": "/properties/control_mode",
      "type": "string"
    },
    "power_status": {
      "$id": "/properties/power_status",
      "type": "string"
    },
    "temperature": {
      "$id": "/properties/temperature",
      "type": "object",
      "properties": {
        "temperature": {
          "$id": "/properties/temperature/properties/temperature",
          "type": "integer"
        },
        "temperature_error": {
          "$id": "/properties/temperature/properties/temperature_error",
          "type": "boolean"
        }
      }
    },
    "brightness_mode": {
      "$id": "/properties/brightness_mode",
      "type": "integer"
    },
    "brightness_level": {
      "$id": "/properties/brightness_level",
      "type": "integer"
    },
    "pixel_failure": {
      "$id": "/properties/pixel_failure",
      "type": "boolean"
    }
  }
}

```

```

    },
    "lamp_error": {
      "$id": "/properties/lamp_error",
      "type": "boolean"
    },
    "fan_status": {
      "$id": "/properties/fan_status",
      "type": "object",
      "properties": {
        "fan_error": {
          "$id": "/properties/fan_status/properties/fan_error",
          "type": "boolean"
        },
        "fan_status_array_length": {
          "$id": "/properties/fan_status/properties/fan_status_array_length",
          "type": "integer"
        },
        "fan_status_array": {
          "$id": "/properties/fan_status/properties/fan_status_array",
          "type": "string"
        }
      }
    },
    "multi_msg": {
      "$id": "/properties/multi_msg",
      "type": "object",
      "properties": {
        "multi_text": {
          "$id": "/properties/multi_msg/properties/multi_text",
          "type": "string"
        },
        "owner": {
          "$id": "/properties/multi_msg/properties/owner",
          "type": "string"
        },
        "priority": {
          "$id": "/properties/multi_msg/properties/priority",
          "type": "integer"
        },
        "duration": {
          "$id": "/properties/multi_msg/properties/duration",
          "type": "integer"
        },
        "beacons_enabled": {
          "$id": "/properties/multi_msg/properties/beacons_enabled",
          "type": "boolean"
        },
        "pixel_service_enabled": {
          "$id": "/properties/multi_msg/properties/pixel_service_enabled",
          "type": "boolean"
        }
      }
    },
    "last_successful_poll": {
      "$id": "/properties/last_successful_poll",
      "type": "string",
      "description": "Date and time of last successful poll"
    },
    "last_successful_message": {
      "$id": "/properties/last_successful_message",
      "type": "string",
      "description": "Date and time of last successful message"
    },
    "last_communication_attempt": {
      "$id": "/properties/last_communication_attempt",
      "type": "string",
      "description": "Date and time of last communication attempt"
    }
  }

```

```
}
}
```

3. sunguide_dms_current – This will store the combined config and message status at it latest state. This collection will only store the current snapshot, not the history

```
{
  "$id": "http://example.com/example.json",
  "type": "object",
  "definitions": {},
  "$schema": "http://json-schema.org/draft-07/schema#",
  "properties": {
    "_id": {
      "$id": "/properties/_id",
      "type": "object",
      "description": "The unique key of the document",
      "properties": {
        "dms_id": {
          "$id": "/properties/_id/properties/dms_id",
          "type": "integer"
        },
        "update_timestamp": {
          "$id": "/properties/_id/properties/update_timestamp",
          "type": "string",
          "description": "The date and time when the message was received from"
        }
      }
    },
    "sunguide"
  },
  "is_current": {
    "$id": "/properties/is_current",
    "type": "boolean"
  },
  "dateparts": {
    "$id": "/properties/dateparts",
    "type": "object",
    "properties": {
      "date": {
        "$id": "/properties/dateparts/properties/date",
        "type": "string",
        "description": "The date portion of the update_timestamp"
      },
      "year": {
        "$id": "/properties/dateparts/properties/year",
        "type": "integer",
        "description": "The year portion of the update_timestamp"
      },
      "month": {
        "$id": "/properties/dateparts/properties/month",
        "type": "integer",
        "description": "The month (number) portion of the update_timestamp"
      },
      "day": {
        "$id": "/properties/dateparts/properties/day",
        "type": "integer",
        "description": "The day number of the month of the update_timestamp"
      },
      "weekday": {
        "$id": "/properties/dateparts/properties/weekday",
        "type": "integer",
        "description": "The day of the week (1 = Sunday) of the update_timestamp"
      },
      "hour": {

```

```

        "$id": "/properties/dateparts/properties/hour",
        "type": "integer",
        "description": "The hour portion of the update_timestamp"
    },
    "minute": {
        "$id": "/properties/dateparts/properties/minute",
        "type": "integer",
        "description": "The minute portion of the update_timestamp"
    }
},
"data": {
    "$id": "/properties/data",
    "type": "object",
    "properties": {
        "dms_comm": {
            "$id": "/properties/dms_comm",
            "type": "object",
            "properties": {
                "dms_address": {
                    "$id": "/properties/dms_comm/properties/dms_address",
                    "type": "object",
                    "properties": {
                        "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address",
                        "type": "object",
                        "properties": {
                            "port_server_udp_address": {
                                "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address",
                                "type": "object",
                                "properties": {
                                    "address": {
                                        "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address/properties/address",
                                        "type": "integer"
                                    },
                                    "port_server_ip": {
                                        "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address/properties/port_server_ip",
                                        "type": "string"
                                    },
                                    "port_server_port_number": {
                                        "$id":
"/properties/dms_comm/properties/dms_address/properties/pmpp_address/properties/port_server_udp_address/properties/port_server_port_number",
                                        "type": "integer"
                                    }
                                }
                            }
                        }
                    }
                },
                "protocol": {
                    "$id": "/properties/dms_comm/properties/protocol",
                    "type": "string"
                },
                "poll_process_name": {
                    "$id": "/properties/dms_comm/properties/poll_process_name",
                    "type": "string"
                },
                "packet_timeout": {
                    "$id": "/properties/dms_comm/properties/packet_timeout",
                    "type": "integer"
                }
            }
        }
    }
}

```

```

    "packet_retry_limit": {
      "$id": "/properties/dms_comm/properties/packet_retry_limit",
      "type": "integer"
    },
    "command_retry_limit": {
      "$id": "/properties/dms_comm/properties/command_retry_limit",
      "type": "integer"
    },
    "poll_cycle": {
      "$id": "/properties/dms_comm/properties/poll_cycle",
      "type": "integer"
    },
    "op_status": {
      "$id": "/properties/dms_comm/properties/op_status",
      "type": "integer"
    }
  },
  "dms_config": {
    "$id": "/properties/dms_config",
    "type": "object",
    "properties": {
      "dms_type": {
        "$id": "/properties/dms_config/properties/dms_type",
        "type": "string"
      },
      "manufacturer": {
        "$id": "/properties/dms_config/properties/manufacturer",
        "type": "string"
      },
      "lines": {
        "$id": "/properties/dms_config/properties/lines",
        "type": "integer"
      },
      "columns": {
        "$id": "/properties/dms_config/properties/columns",
        "type": "integer"
      },
      "has_beacons": {
        "$id": "/properties/dms_config/properties/has_beacons",
        "type": "boolean"
      },
      "brightness_level_day": {
        "$id": "/properties/dms_config/properties/brightness_level_day",
        "type": "object",
        "properties": {
          "brightness_level": {
            "$id":
"/properties/dms_config/properties/brightness_level_day/properties/brightness_level",
            "type": "integer"
          }
        }
      },
      "brightness_level_night": {
        "$id": "/properties/dms_config/properties/brightness_level_night",
        "type": "object",
        "properties": {
          "brightness_level": {
            "$id":
"/properties/dms_config/properties/brightness_level_night/properties/brightness_level",
            "type": "integer"
          }
        }
      },
      "location": {
        "$id": "/properties/dms_config/properties/location",
        "type": "object",
        "properties": {

```

```

        "description": {
            "$id":
"/properties/dms_config/properties/location/properties/description",
            "type": "string"
        },
        "roadway": {
            "$id":
"/properties/dms_config/properties/location/properties/roadway",
            "type": "string"
        },
        "direction": {
            "$id":
"/properties/dms_config/properties/location/properties/direction",
            "type": "string"
        },
        "latitude": {
            "$id":
"/properties/dms_config/properties/location/properties/latitude",
            "type": "number"
        },
        "longitude": {
            "$id":
"/properties/dms_config/properties/location/properties/longitude",
            "type": "number"
        }
    },
    "sign_type": {
        "$id": "/properties/dms_config/properties/sign_type",
        "type": "string"
    },
    "font_id": {
        "$id": "/properties/dms_config/properties/font_id",
        "type": "object",
        "properties": {
            "$id":
"/properties/dms_config/properties/font_id/properties/id",
            "type": "object",
            "properties": {
                "id": {
                    "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/id",
                    "type": "integer"
                },
                "provider_name": {
                    "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/provider_name",
                    "type": "string"
                },
                "resource_type": {
                    "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/resource_type",
                    "type": "string"
                },
                "center_id": {
                    "$id":
"/properties/dms_config/properties/font_id/properties/id/properties/center_id",
                    "type": "string"
                }
            }
        }
    },
    "font_type": {
        "$id": "/properties/dms_config/properties/font_type",
        "type": "string"
    },
    "sign_use": {
        "$id": "/properties/dms_config/properties/sign_use",

```

```

        "type": "string"
      },
      "can_publish": {
        "$id": "/properties/dms_config/properties/can_publish",
        "type": "boolean"
      },
      "color_support": {
        "$id": "/properties/dms_config/properties/color_support",
        "type": "boolean"
      },
      "graphic_support": {
        "$id": "/properties/dms_config/properties/graphic_support",
        "type": "boolean"
      },
      "default_auto_merge": {
        "$id": "/properties/dms_config/properties/default_auto_merge",
        "type": "boolean"
      }
    }
  },
  "status": {
    "$id": "/properties/status",
    "type": "object",
    "properties": {
      "op_status": {
        "$id": "/properties/status/properties/op_status",
        "type": "string"
      },
      "control_mode": {
        "$id": "/properties/status/properties/control_mode",
        "type": "string"
      },
      "power_status": {
        "$id": "/properties/status/properties/power_status",
        "type": "string"
      },
      "temperature": {
        "$id": "/properties/status/properties/temperature",
        "type": "object",
        "properties": {
          "temperature": {
            "$id":
"/properties/status/properties/temperature/properties/temperature",
            "type": "integer"
          },
          "temperature_error": {
            "$id":
"/properties/status/properties/temperature/properties/temperature_error",
            "type": "boolean"
          }
        }
      },
      "brightness_mode": {
        "$id": "/properties/status/properties/brightness_mode",
        "type": "integer"
      },
      "brightness_level": {
        "$id": "/properties/status/properties/brightness_level",
        "type": "integer"
      },
      "pixel_failure": {
        "$id": "/properties/status/properties/pixel_failure",
        "type": "boolean"
      },
      "lamp_error": {
        "$id": "/properties/status/properties/lamp_error",
        "type": "boolean"
      }
    }
  }
}

```

```

    },
    "fan_status": {
      "$id": "/properties/status/properties/fan_status",
      "type": "object",
      "properties": {
        "fan_error": {
          "$id":
"/properties/status/properties/fan_status/properties/fan_error",
          "type": "boolean"
        },
        "fan_status_array_length": {
          "$id":
"/properties/status/properties/fan_status/properties/fan_status_array_length",
          "type": "integer"
        },
        "fan_status_array": {
          "$id":
"/properties/status/properties/fan_status/properties/fan_status_array",
          "type": "string"
        }
      }
    },
    "multi_msg": {
      "$id": "/properties/status/properties/multi_msg",
      "type": "object",
      "properties": {
        "multi_text": {
          "$id":
"/properties/status/properties/multi_msg/properties/multi_text",
          "type": "string"
        },
        "owner": {
          "$id":
"/properties/status/properties/multi_msg/properties/owner",
          "type": "string"
        },
        "priority": {
          "$id":
"/properties/status/properties/multi_msg/properties/priority",
          "type": "integer"
        },
        "duration": {
          "$id":
"/properties/status/properties/multi_msg/properties/duration",
          "type": "integer"
        },
        "beacons_enabled": {
          "$id":
"/properties/status/properties/multi_msg/properties/beacons_enabled",
          "type": "boolean"
        },
        "pixel_service_enabled": {
          "$id":
"/properties/status/properties/multi_msg/properties/pixel_service_enabled",
          "type": "boolean"
        }
      }
    },
    "last_successful_poll": {
      "$id": "/properties/status/properties/last_successful_poll",
      "type": "string",
      "description": "Date and time of last successful poll"
    },
    "last_successful_message": {
      "$id": "/properties/status/properties/last_successful_message",
      "type": "string",
      "description": "Date and time of last successful message"
    }
  },

```

```

        "last_communication_attempt": {
          "$id": "/properties/status/properties/last_communication_attempt",
          "type": "string",
          "description": "Date and time of last communication attempt"
        }
      }
    }
  }
}

```

Indexes:

Default unique index on
_id.dms_id and _id.update_timestamp

Additional indexes on
Location.roadway and
Location.diection

3.4.1.1.4 Transit AVL (TA-DR)

Streaming transit data will be collected from the General Transit Feed Specification – Real Time (GTFS-RT) system. Additional information regarding how to connect to and consume data from the GTFS-RT to be provided prior to critical design review iteration 2. Protocols for accessing GTFS-RT data streams can be found at: <https://developers.google.com/transit/gtfs-realtime/>.

3.4.1.1.5 Transit Static Data (TSD-DR)

Transit data is made available through General Transit Feed Specification (GTFS) feeds provided by FDOT and are available for various transit agencies (LYNX, SunRail etc.). The GTFS feeds are different for each agency.

For the iteration – 1, the data sources are limited to SunRail and Lynx static data. The other GTFS data feeds are not being made available to R-ICMS from an aggregated or centralized source by FDOT. ~~At the time of design phase iteration 1, only SunRail transit agency provides the GTFS feed for static data and the 1st iteration scope is limited to SunRail static data load.~~ The data is not yet available through any API feeds and is available through manual download from the respective agency sites.

Other Transit agency source feeds will be addressed in the later iterations.

Table 4 – Transit Location Specification

Data Set Name	Transit Feeds
Source location	- SunRail GTFS feed http://corporate.sunrail.com/about-sunrail/gtfs-data-download/ http://corporate.sunrail.com/wp-content/uploads/2016/05/google_transit.zip - Download link - Lynx GTFS feed https://www.golynx.com/lynxmap/DataDownload/index_files/Page414.htm http://www.golynx.com/lynxmap/GolYNX_data/google_transit.zip - Download link
File Name	google_transit.zip

Source Format	Comma Separated Variable (CSV)
Source ICD Location	https://developers.google.com/transit/gtfs/reference/
Target Location	HDFS and NOSQL database HDFS location: /user/dev/gtfs_transit_avl/<agency_name>/<year>/<month>/<day>/<agency_name>_transit_<date>_<time>.zip MongoDB location: Database: gtfs_transit_avl Collections: gtfs_feeds
Target Format	CSV (HDFS) and JSON (NoSQL)
Data load mechanism into R-ICMS	This iteration will only handle static data related to the SunRail and Lynx GTFS feeds which are available for manual download via their respective websites and will be loaded manually. The later iterations will create the scripts to automate the process to get the data from the GTFS API calls and loading into R-ICMS. The following GTFS information will be processed. 1. Agency 2. Routes 3. Stops 4. Trips Download the file and load the data into HDFS and NoSQL manually or making use of scripts wherever possible.
Frequency	<TBD>
Process Notifications	<TBD>
Logging	<TBD>
Notes	For the 1st iteration the following static data will be loaded into R-ICMS environment from the currently available SunRail and Lynx GTFS feeds 1. Agency 2. Routes 3. Stops 4. Trips Currently, there are no API feeds for this data and the data will be loaded manually downloading from the source and store in HDFS and NoSQL database with the help of scripts.

This data source has no accepted ICD. This data source's attributes are documented in the Data Sources Spreadsheet.

The GTFS static feed data is infrequently updated and not available through any API feeds, but via manual download from the SunRail website. A manual process with the help of scripts to load the data into the HDFS and NoSQL database will be used in iteration 1.

The R-ICMS designated system administrator will perform the following functions.

- Identify when an update is available for this dataset by checking the SunRail and Lynx websites and comparing the available version to the previously loaded version in the data dictionary
- Download the GTFS zip file from the SunRail and Lynx websites and place it in a specified staging location (server and folder) within the DFE (the specified staging location data will be stored in the configuration database)
- An automated script will read the GTFS zip file from the configured staging location, transform the data as needed, and load it into both HDFS and the NoSQL data store
- The script will perform the following tasks
 - Read the data and transform into the NoSQL document model per design and store into NoSQL database
 - Store the files in HDFS with date and timestamps appended to the filenames in a pre-defined folder structure
 - Log the execution using the common logging function
 -

3.4.1.1.5.1 Data Model:

The GTFS transit source data is available in multiple CSV format files with each one representing a different type. (Ex. Agency, Routes, Stops, Trips etc.) There are interdependencies among the data in the GTFS CSV files. Consequently, the data will be stored in a way that retains the dependencies. The following document model is designed to store the static data in a NoSQL data store.

Note that this model only takes into account the data files designated for this iteration only. When additional data comes into play during later iterations, the model may need adjustments.

JSON model for GTFS transit files:

The agency information will be at the top level within the document and the associated sets of information (Stops, Routes and Trips) will be part of sub-documents in the model.

The “update_timestamp” attribute is being added to track the history of data updates coming at regular intervals.

```
{
  "$id": "http://example.com/example.json",
  "type": "object",
  "definitions": {},
  "$schema": "http://json-schema.org/draft-07/schema#",
  "properties": {
    "_id": {
      "$id": "/properties/_id",
      "type": "object",
      "properties": {
        "agency_name": {
          "$id": "/properties/_id/properties/agency_name",
          "type": "string"
        }
      }
    }
  }
}
```

```
    },
    "update_timestamp": {
      "$id": "/properties/_id/properties/update_timestamp",
      "type": "string"
    }
  }
},
"agency_id": {
  "$id": "/properties/agency_id",
  "type": "string"
},
"agency_url": {
  "$id": "/properties/agency_url",
  "type": "string"
},
"agency_timezone": {
  "$id": "/properties/agency_timezone",
  "type": "string"
},
"agency_phone": {
  "$id": "/properties/agency_phone",
  "type": "string"
},
"agency_lang": {
  "$id": "/properties/agency_lang",
  "type": "string"
},
"stops": {
  "$id": "/properties/stops",
  "type": "array",
  "items": {
    "$id": "/properties/stops/items",
    "type": "object",
    "properties": {
      "stop_id": {
        "$id": "/properties/stops/items/properties/stop_id",
        "type": "integer"
      },
      "stop_name": {
        "$id": "/properties/stops/items/properties/stop_name",
        "type": "string"
      },
      "stop_desc": {
        "$id": "/properties/stops/items/properties/stop_desc",
        "type": "string"
      }
    }
  }
}
```

```
    },
    "stop_lat": {
      "$id": "/properties/stops/items/properties/stop_lat",
      "type": "number"
    },
    },
    "stop_lon": {
      "$id": "/properties/stops/items/properties/stop_lon",
      "type": "number"
    },
    },
    "zone_id": {
      "$id": "/properties/stops/items/properties/zone_id",
      "type": "integer"
    },
    },
    "stop_url": {
      "$id": "/properties/stops/items/properties/stop_url",
      "type": "string"
    },
    },
    "location_type": {
      "$id": "/properties/stops/items/properties/location_type",
      "type": "string"
    },
    },
    "parent_station": {
      "$id": "/properties/stops/items/properties/parent_station",
      "type": "integer"
    },
    },
    "stop_timezone": {
      "$id": "/properties/stops/items/properties/stop_timezone",
      "type": "string"
    },
    },
    "wheelchair_boarding": {
      "$id": "/properties/stops/items/properties/wheelchair_boarding",
      "type": "integer"
    }
  }
}
},
"routes": {
  "$id": "/properties/routes",
  "type": "array",
  "items": {
    "$id": "/properties/routes/items",
    "type": "object",
    "properties": {
      "route_id": {
        "$id": "/properties/routes/items/properties/route_id",
```

```
        "type": "integer"
      },
      "agency_id": {
        "$id": "/properties/routes/items/properties/agency_id",
        "type": "string"
      },
      "route_short_name": {
        "$id": "/properties/routes/items/properties/route_short_name",
        "type": "string"
      },
      "route_long_name": {
        "$id": "/properties/routes/items/properties/route_long_name",
        "type": "string"
      },
      "route_desc": {
        "$id": "/properties/routes/items/properties/route_desc",
        "type": "string"
      },
      "route_type": {
        "$id": "/properties/routes/items/properties/route_type",
        "type": "integer"
      },
      "route_url": {
        "$id": "/properties/routes/items/properties/route_url",
        "type": "string"
      },
      "route_color": {
        "$id": "/properties/routes/items/properties/route_color",
        "type": "string"
      },
      "route_text_color": {
        "$id": "/properties/routes/items/properties/route_text_color",
        "type": "string"
      },
      "route_sort_order": {
        "$id": "/properties/routes/items/properties/route_sort_order",
        "type": "integer"
      }
    }
  },
  "trips": {
    "$id": "/properties/trips",
    "type": "array",
    "items": {
```

```
"$id": "/properties/trips/items",
"type": "object",
"properties": {
  "route_id": {
    "$id": "/properties/trips/items/properties/route_id",
    "type": "integer"
  },
  "service_id": {
    "$id": "/properties/trips/items/properties/service_id",
    "type": "integer"
  },
  "trip_id": {
    "$id": "/properties/trips/items/properties/trip_id",
    "type": "string"
  },
  "trip_headsign": {
    "$id": "/properties/trips/items/properties/trip_headsign",
    "type": "string"
  },
  "trip_short_name": {
    "$id": "/properties/trips/items/properties/trip_short_name",
    "type": "string"
  },
  "direction_id": {
    "$id": "/properties/trips/items/properties/direction_id",
    "type": "integer"
  },
  "block_id": {
    "$id": "/properties/trips/items/properties/block_id",
    "type": "string"
  },
  "shape_id": {
    "$id": "/properties/trips/items/properties/shape_id",
    "type": "integer"
  },
  "wheelchair_accessible": {
    "$id":
"/properties/trips/items/properties/wheelchair_accessible",
    "type": "integer"
  },
  "bikes_allowed": {
    "$id": "/properties/trips/items/properties/bikes_allowed",
    "type": "integer"
  }
}
```

```
    }  
  }  
}  
}  
}
```

Indexes:

```
agency_name  
update_timestamp
```

3.4.1.1.6 Special Event (SE-DR)

Special Event data will be collected from the Active Arterial Management (AAM) system. Access protocols for special event data will be documented in the appropriate 90% design documentation.

3.4.1.1.7 Signal Performance Measures (SPM-DR)

Signal Performance Measure data will be collected directly from traffic signal controllers. Data will be ingested from data files collected from each controller containing the relevant data. The data source will provide the raw data necessary to calculate Turning Movement Counts for some of the types of traffic controllers. The controller event data will be used as the basis for the Turning Movement Count calculations. Protocols for the SPM driver will be specified in the appropriate 90% design documentation.

3.4.1.1.8 Intersection Movement Counts (IMC-DR)

Intersection movement count data will be available via the Gridsmart SQL database. The IMC-DR will regularly query the Gridsmart database to receive the latest information regarding this data. The Gridsmart database will also support the querying of historical data when needed. Protocols for the Gridsmart database driver will be specified in the appropriate 90% design documentation.

3.4.1.1.9 Weather (WEA-DR)

Weather data (watches and warnings) will be collected from the National Weather Service data feed. Only weather data relevant to the R-ICMS coverage area will be collected and stored. Protocols for the weather driver will be specified in the appropriate 90% design documentation.

3.4.1.1.10 Video (VID-DR)

Video data will be consumed from the iVEDDS system. Further detail regarding connection to the data source will be provided prior to critical design review iteration 2. Protocols for the video driver will be specified in the appropriate 90% design documentation.

3.4.1.1.11 Basemap (GIS) (BM-DR)

The basemap part of the GIS maps is critical for the R-ICMS application and enables displaying various parameters that are attributed as map display (Ex. real-time traffic conditions, real-time events etc.). This is a GIS dataset with required pre-defined features for displaying roadways, intersections and other device assets and is available in a special format understood by the GIS tools. As this is a special format, the load process will adopt a manual update approach. However, once the data file is gathered, it will be stored within the R-ICMS data file repository within HDFS.

Table 5 – Basemap Specification

Data Set Name	Basemap
Source location	NA
File Name	NA
Source Format	Binary GIS Shape file in ZIP format
ICD Source Location	http://www.arcgis.com/home/item.html?id=c908722b0841404ba6a87d6221f201e4
Target Location	HDFS File store: /user/dev/gis/base_map/<year>/<month>/<day>/base_map_<date>_<time>.zip
Target Format	1. GIS server database 2. Binary ZIP file in the HDFS
Data load mechanism into R-ICMS	Manual process to publish the data into GIS Server
Frequency	As needed
Process Notifications	<TBD>
Logging	<TBD>
Notes	This will be a manual process until a workflow is established for data collection.

The R-ICMS designated system administrator will perform the following functions.

- Identify when a basemap data update is available.
- Collect the data file and upload to the staging server at a designated location.
- Update the basemap on the GIS server using appropriate tools.

3.4.1.1.12 RCI Number of Lanes at Intersection (GIS) (RCI-DR)

The FDOT GIS Roads with Number of Lanes feature class provides spatial information on the number of through lanes. A through traffic lane is a lane of roadway intended to facilitate moving vehicles along a corridor. For a divided roadway, there will be two values, one for the left roadside and one for the right roadside. For a composite roadside, there will be one value. This information is required for all functionally classified roadways on or off the State Highway System (SHS) and Active-Exclusive roadways.

Table 6 – RCI # Lanes at Intersection Specification

Data Set Name	RCI # lanes at intersection
Source location	https://ftp.fdot.gov/file/d/FTP/FDOT/co/planning/transtat/gis/shapefiles/
File Name	number_of_lanes.zip
Source Format	GIS Shape files representing Roads with Number of Lanes. The data is provided statewide in ZIP format and is updated weekly.
ICD Source	

Location	
Target Location	1. Publish the data to GIS server 2. Store the binary ZIP file in the HDFS HDFS File store location: /user/dev/gis/rci_number_of_lanes/<year>/<month>/<day>/number_of_lanes_<date>_<time>.zip
Target Format	1. GIS server database 2. Binary ZIP file in the HDFS
Data load mechanism into R-ICMS	1. FTP process to bring the data into R-ICMS 2. Manual process to publish the data into GIS Server
Frequency	Weekly (The dataset is updated every Saturday)
Process Notifications	<TBD>
Logging	A log record will be created in the NoSQL database each time a job is executed with the date and timestamp details.
Notes	The loading of the data into R-ICMS GIS is assumed to be a manual process. Once the file is loaded into R-ICMS, a notification would be sent to the appropriate audience which can be configured in the application.

This data source has no accepted ICD. The data attributes for this data source are documented in work sheet with a name corresponding to this data source of the spreadsheet used to track the status of the required data sources spreadsheet.

3.4.1.1.12.1 Ingestion process:

To ingest the lane configuration data, create a script with the data source name as the script name encapsulating the following processes:

1. Use the common utility call to fetch the configuration specific to this data source from NoSQL
2. With the retrieved configuration parameters, use the FTP process function to fetch from source location and store it at the target location.
3. Add a log entry in the designated log file using the common logging framework.
4. Send notifications using notification utility function.
5. R-ICMS administrator will publish the data into GIS server using the file transferred to R-ICMS.

3.4.1.1.13 School Zones (GIS) (SZ-DR)

The school zones dataset is a compilation of speed zones posted at school surrounding roads with additional details. Due to this dataset not being updated at regular intervals, a manual process will be utilized to ETL this data.

Table 7 – School Zones Specification

Data Set Name		School Zones
Source location	NA	
File Name	NA	
Source Format	Binary GIS Shape File in ZIP format	
Target Location	1. Publish the data to GIS server	

	2. Store the binary ZIP file in the HDFS HDFS file store location: /user/dev/gis/school_zones/<year>/ <month>/<day>/school_zones_<date>_<time>.zip
Target Format	1. GIS server database 2. Binary ZIP file in the HDFS
Data load mechanism into R-ICMS	Manual process to publish the data into GIS Server
Frequency	As needed
Process Notifications	<TBD>
Logging	<TBD>
Notes	Since there are manual efforts to collect the data and compile the dataset, it is recommended to utilize manual coordination with the load process. On the other hand, the process can be partially automated, provided the following conditions are met and the process is subject to the feasibility of scripting the load process. • Data file is available in standard format at all times. • Manually transport the data file and deposit into a pre-defined location on a designated server in the R-ICMS environment. • Define the configuration for a script process. • Create a script to process the file transfer using the configuration settings. • Schedule the script to run at a frequent interval and process the file when a new data file is found in the pre-defined location. Such items can be evaluated during the project construction and consider adding them in the final iteration.

This data source has no accepted ICD. This data source's attributes are documented in the Data Sources Spreadsheet.

3.4.1.1.13.1 Ingestion process:

This data will be loaded through a manual process.

The R-ICMS designated system administrator will perform the following functions.

- Identify when an update is available for this dataset.
- Collect the data file.
- Publish data the GIS server using appropriate tools.
- Upload the shape file into HFDS at the designated location

3.4.1.1.14 School Locations (GIS) (SL-DR)

The school locations dataset is a compilation of school location along with other details. Due to this dataset availability is "As Needed" basis, a manual process will be utilized to load this data.

Table 8 – School Zones Specification

Data Set Name	School Zones
Source location	https://www.fgdl.org/metadataexplorer/explorer.jsp
File Name	FDOT provided zip file.
Source Format	Binary GIS Shape File in ZIP format

Target Location	1. Publish the data to GIS server 2. Store the binary ZIP file in the HDFS HDFS file store location: /user/dev/gis/school_locations/<year>/<month>/<day>/school_locations_<date>_<time>.zip
Target Format	1. GIS server database 2. Binary ZIP file in the HDFS
Data load mechanism into R-ICMS	Manual process to publish the data into GIS Server
Frequency	As needed
Process Notifications	<TBD>
Logging	<TBD>
Notes	Since there are manual efforts to collect the data and compile the dataset, it is recommended to utilize manual coordination with the load process. On the other hand, this can be partially automated provided the following conditions are met and subject to the feasibility of scripting the load process. • Data file is available in standard format all the time. Steps to load the data: • Manually transport the data file and deposit into a pre-defined location on a designated server in the R-ICMS environment. • Define the configuration for a script process. • Create a script to process the file transfer using the configuration settings. • Schedule the script to run at a frequent interval and will process the file when a new data file is found in the pre-defined location. Such items can be evaluated during the project construction and will be added in the final iteration if feasible.

This data source has no accepted ICD. The data attributes for this data source are documented in work sheet with a name corresponding to this data source of the spreadsheet used to track the status of the required data sources spreadsheet.

3.4.1.1.14.1 Ingestion process:

This data will be loaded through a manual process.

The R-ICMS designated system administrator will perform the following functions.

- Identify when an update is available for this dataset
- Collect the data file
- Publish data on the GIS server using appropriate tools
- Upload the shape file into HDFS at the designated location

3.4.1.1.15 Emergency Responder Locations (GIS) (ERL-DR)

Emergency Responder Locations consist of fire stations, hospitals and law enforcement offices. The data is compiled as GIS shape files by the respective counties and is made available through county websites. In addition, there are individual files for each of the sub-sets. This dataset is not updated at regular intervals and only happens on manual initiative. Since this dataset is not updated on regular intervals, a manual process will be used to load the data.

Table 9 – Emergency Responder Specification

Data Set Name		Emergency Responder Locations
Source location	NA	
File Name	NA	
Source Format	Binary GIS Shape File in ZIP format	
Target Location	1. Publish the data to GIS server 2. Store the binary ZIP file in the HDFS HDFS file store location: /user/dev/gis/emergency_responder_locations/agency/<year>/ <month>/<day>/emergency_responder_<date>_<time>.zip	
Target Format	1. GIS server database 2. Binary ZIP file in the HDFS	
Data load mechanism into R-ICMS	Manual process to publish the data into GIS Server	
Frequency	As needed	
Process Notifications	<TBD>	
Logging	<TBD>	
Notes	Since there are manual efforts to collect the data and compile the dataset, it is recommended to utilize manual coordination with the load process. On the other hand, this process can be partially automated provided the following conditions are met and the process is subject to the feasibility of scripting the load process. • Data file is available in standard format at all times • Manually fetch the data file and deposit into a pre-defined location on a designated server in the R-ICMS environment • Define configuration for a script process • Create a script to process the file using the configuration settings • Schedule the script to run at a frequent interval and will process the file when a new data file is found in the pre-defined location Such items can be evaluated during the project construction and consider adding them in the final iteration.	

This data source has no accepted ICD. This data source's attributes are documented in the Data Sources Spreadsheet.

3.4.1.15.1 Ingestion process:

This data will be loaded through a manual process. The R-ICMS designated system administrator will perform the following functions.

- Identify when an update is available for this dataset
- Collect the data file
- Publish data on the GIS server using appropriate tools
- Upload the shape file into HDFS at the designated location

3.4.1.2 Pipelines

The R-ICMS application will need to handle various data sources that flow in near real-time and the flow should reach data consumers at the same frequency. This requirement necessitates using a component to accomplish this within the R-ICMS environment.

Apache Kafka, a distributed streaming platform which is part of the Cloudera Hadoop stack, will be used as the streaming component for the R-ICMS application. This component is supported by the Cloudera license agreement. Kafka can be scaled into a cluster when workloads demand more streaming resources.

The Apache Kafka platform will be used for building real-time data pipelines and streaming applications within the R-ICMS application. Kafka works with the concept of Producers and Consumers. Producers are applications that publish data to topics. The published data (often called messages or records) is maintained in the order it was received within Kafka. Consumers are applications that read data from topics. Multiple consumers may read from the same topic from any starting point. For example, one consumer may feed a real-time dashboard via web sockets and another consumer may transform and save the data to a NoSQL data store. Multiple consumers can also be used to process data on a single topic. If a single consumer falls behind in reading a topic, then scaling up can be achieved by increasing the number of consumers. For example, if the consumer is doing something resource intensive with the data then the consumer may fall behind. Increasing the number of consumers may help in this situation provided the consumers don't contend with each other for resources.

Within the R-ICMS environment, Kafka will be used in situations where it makes sense to do so:

- When the data source is a stream: For example, if the source is a web socket then Kafka could be used to buffer the data for the downstream consumers
- When the message size is small: While Kafka can be configured to handle large messages, it works best with messages that are around 10KB in size. Note that in the case of semi-structured data, it may be possible to split the source data into smaller chunks. For example, if the source is a REST API that returns a 10MB JSON array of objects, but each object is only 1KB, it may be reasonable to split the array into separate objects and publish each object to Kafka. On the other hand, if the source is a 10MB GIS shape file it would be best not to publish that to Kafka.
- When a downstream system requires a stream: For example, a real-time dashboard should not poll for new data but should instead be pushed new data typically via web sockets which in turn may consume data from Kafka
- When multiple consumers need to process the same source data: For example, one consumer may write to a NoSQL data store while another consumer may write to a relational database while another may post to a REST API. The data need only be published once. New consumers may be easily added to support new uses for the source data.

Pipelines will be responsible for further storage / distribution to the other client processes as designed by the system workflow.

- a. Depending on the nature of the data, it will be transformed and stored into a database (SQL or NOSQL)
- b. One of the following approaches will be implemented to copy RAW data to HDFS to accommodate optimal usage of HDFS block storage size and data retrieval performance
 - i. A copy of the RAW data will be transformed into a suitable storage file format for HDFS and stored on a staging HDFS server to accommodate the configured HDFS block size. A batch job will be configured into the Cloudera which will load the RAW block data into HDFS. The staging data will be removed once loaded into HDFS.
 - ii. A copy of the RAW data will be transformed into a suitable file format for HDFS and the data will be appended to the configured block file in HDFS which will help in utilizing maximum block size.
 - iii. A copy of the RAW data will be transformed into a suitable file format for HDFS and will be stored as archival files
- c. Wherever applicable, the data will also be distributed to other clients through the streaming component Kafka. (Ex. user interface and internal sub-components like Modeling Engine, Rules Engine)

With a critical requirement of real-time data flow within the R-ICMS application, Kafka will be the ideal component for implementation.

3.4.1.2.1 Current Traffic Conditions (CTC-PL)

The Modeling Engine, the Business Rules Engine, and other consumers will need access to current traffic condition data as it arrives. The CTC-PL will provide near-real-time access to traffic data for storage and use by the data services and business services.

~~For processing traffic condition data, Kafka will be used in R-ICMS iteration 1 in the following layers:~~

- ~~● Data ingestion / Pipeline layers
 - ~~○ ITSQA real time traffic data which contains 3 distinct types of data:
 - ~~■ ITSQA link data~~
 - ~~■ ITSQA traffic data~~
 - ~~■ ITSQA classification data~~~~~~
- ~~● Service layer
 - ~~○ Geo event server~~~~

~~The traffic conditions from ITSQA need to be updated on the User Interface (UI) screen in real-time. To accomplish this update, the following process will be established and implemented in the components of the system:~~

~~Data Ingestion:~~

- ~~1. Each ISTIQA data source will be read by a python script, minimally transformed, and published to its own Kafka topic. Each of these python scripts functions as a producer.~~
- ~~2. The only transformation that will be performed is converting the single XML document into individual JSON objects (one per ITSQA link), propagating the timestamp from the XML root element into each JSON object.~~
- ~~3. These producers will be scheduled to run at a frequency to match the data source's update frequency. The ITSQA link data producer will run daily. The traffic and classification data producers will run every 60 seconds.~~
- ~~4. After the ITSQA data has been published to Kafka topics, consumers that are aligned to each destination will perform any additional transformations on the data and write the data to the appropriate destinations:~~
 - ~~a. MongoDB~~

~~Service Layer:~~

~~The GeoEvent Server will be configured to consume directly from the Kafka traffic topic to support real time updates in the map/UI. The GeoEvent Server will process the real time dynamic traffic data to sync with the pre-configured map application, which in turn will feed the user interface.~~

3.4.1.2.2 Traffic Conditions 5-Minute Average (CT5-PL)

While the near-real time data received from ITSQA will be needed for storage and, possibly, the Business Rules Engine, the near-real-time data stream may be too noisy for many of the business services. A rolling 5-minute average of the real-time data is likely to be used in some business services.

3.4.1.2.3 Events (EVT-PL)

The events pipeline will provide access to traffic incident data from SunGuide and to special event data from the AAM Dashboard.

3.4.1.2.4 Parking (PK-PL)

The Parking Pipeline will provide a single pipeline with information from the parking systems providing data to the R-ICMS.

3.4.1.2.5 Transit AVL (TAVL-PL)

The Transit Automatic Vehicle Location (AVL) pipeline (TAVL-PL) will provide a single pipeline with incoming data from the transit organizations providing AVL data to the R-ICMS.

3.4.1.2.6 Signal Timing (ST-PL)

The Signal Timing pipeline (ST-PL) is a single pipeline that will provide data from signal controllers or Automated Traffic Management Systems (ATMS), central signal control systems, (or SunGuide) that provide signal timing data to the R-ICMS.

3.4.1.2.7 ITS Device Status (IDS-PL)

The ITS Device Status pipeline will provide information on the status of ITS devices such as Dynamic Message Signs (DMS) and Ramp Meters. This information will be used in the execution of business rules as a part of determining which response plans will be suggested to the Corridor Manager.

FDOT has deployed a variety of ITS Devices which produce status updates in near real-time and the data collected from these devices is funneled through the SunGuide system. Once received within R-ICMS, this data needs to be distributed to various sub-systems within R-ICMS as well as stored in the database for operational purposes.

The following are the device types from which the SunGuide will provide the status feeds.

1. Dynamic Message Signs (DMS)
2. CCTV
3. Ramp meters

Device data feeds will provide the configuration of the device and the current status of the respective devices. Kafka streaming will be used to distribute the dynamic data in R-ICMS in the following layers:

1. Data ingestion / Pipeline layers
2. Service layer
3. Geo event server

The status of devices needs to be forwarded to the UI screen in real-time. To accomplish this, the following process will be established and integrated with other components of the system:

1. SunGuide Databus driver / client process (described in section - 3.4.1.1.2.3) will fetch the data from SunGuide and publish to the Kafka streaming aligned with the respective data topic as part of the data ingestion process. This will be the producer for the Kafka streaming.
 - a. This process will be continuously receiving updates from SunGuide Databus.
2. The ingestion process will perform any necessary validations and add missing data before publishing to the Kafka topics. The process will also perform transformations converting the single XML document into individual JSON objects, propagating the timestamp from the XML root element into each JSON object.

3. After the data has been published to Kafka topics, consumers that are aligned to each destination will perform any additional transformations on the data and write the data to the appropriate destinations:
 - a. MongoDB

Following is the typical workflow for device data streaming:

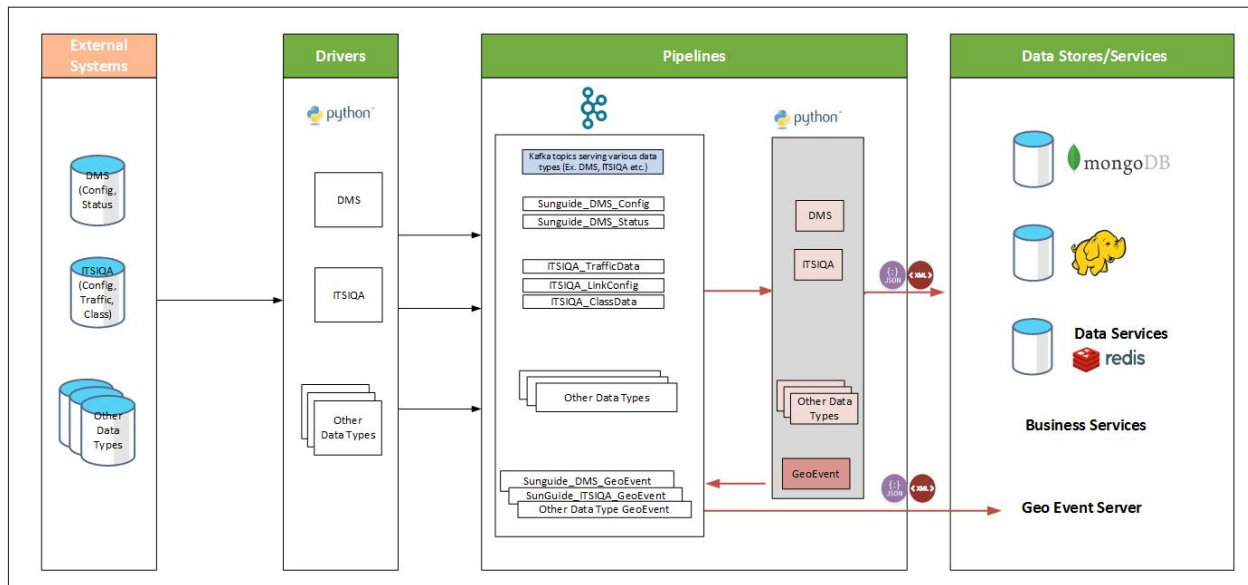


Figure 9 – R-ICMS Data Streaming Workflow

Service Layer:

The geo event server will be configured to consume directly from the Kafka traffic topic to support real-time updates in the map/UI. The Geo Event Server will process the real-time dynamic data to sync with the pre-configured map application which in turn feeds to the user interface.

3.4.1.2.8 Intersection Movement Counts (IMC-PL)

The Intersection Movement Counts pipeline (IMC-PL) is a single pipeline that provides the turning movement counts, vehicle detection, bike detection, and pedestrian detection data from the Intersection Movement Counts (IMC) system.

3.4.1.3 Data Store

There will be four different types of data stores used in the R-ICMS. Those types are described in the subsections below.

The following table describes the summary of the storage details for individual datasets covered in iteration 1.

Table 10 – Data Source Storage

Dataset Name	Data Format	Storage Locations	Storage Format	NoSql Storage
Basemap	Binary Shape File used for GIS	HDFS	Binary files stored with date and timestamp appended to the filename	N/A
Intersections Geometry	Binary Shape File used for GIS	HDFS	Binary files stored with date and timestamp appended to the filename	N/A
RCI—number of Lanes	Binary Shape File used for GIS	HDFS	Binary files stored with date and timestamp appended to the filename	N/A
School Zones	Binary Shape File used for GIS	HDFS	Binary files stored with date and timestamp appended to the filename	N/A
Emergency Responder Locations	Binary Shape File used for GIS	HDFS	Binary files stored with date and timestamp appended to the filename	N/A

GTFS - SunRail	Set of individual CSV files	HDFS	CSV Files stored in HDFS within the designated folder structure identified by data source / sub-set name and chronological order. File names appended with date and timestamp of the process	All static Transit data will be stored in MongoDB. A separate database will be used for this source. Database Name: gtfs_transit_location Collection Name: gtfs_feeds
ITS IQA Traffic conditions	Set of individual XML files	HDFS	XML Files stored in HDFS within the designated folder structure identified by data source / sub-set name and chronological order. File names appended with date and timestamp of the process	Real-time data will get transformed into Document model and stored in NoSQL database. Database Name: traffic_conditions Collections: 1. link_config 2. traffic_conditions 3. classification_data

SunGuide DMS	Set of individual XML files	HDFS	XML Files stored in HDFS within the designated folder structure identified by data source / sub-set name and chronological order. File names appended with date and timestamp of the process	Real-time data will get transformed into Document model and stored in NoSQL database. Database Name: sunguide Collections: 1. sunguide_dms_config 2. sunguide_dms_status 3. sunguide_dms_current
---------------------	-----------------------------	------	--	---

3.4.1.3.1 File Store (FS-ST)

Whole files will be stored in the File Store. Example of whole files that may be stored in the data store include signal timing plans, signed signal timing plans, and GIS files received from other systems. The File store will also be used for long term storage and distributed analytics access for the data streams received via the pipelines. The systems “data lake” will be stored in a distributed file store. It is anticipated that the File Store will be implemented as an HDFS data store. However, the HDFS data store may be augmented by a basic directory-based files store.

HDFS:

HDFS is the underlying default distributed and fault tolerant file system provided by the Hadoop eco-system. While this file system serves as basic storage for files, it also allows systems to utilize these files to access, process (map reduce jobs) and run analytics very efficiently within the Hadoop stack of tools. The Hadoop eco-system provides the capabilities for advanced analytics, machine learning and Artificial Intelligence. Proper planning and design of the file store enables these capabilities for future use. One important consideration during implementation of the R-ICMS is that storing very small files in large numbers is against the principle of HDFS storage due to the block size (128MB default) it uses for storage.

For the initial version of the RICMS, the Hadoop system will be used as a basic “data lake” where the system will bring in various formats of data and store it in one place. Once the data is being collected, users can begin evaluating the value of the data and design applications to utilize the data; applying the advanced capabilities like machine learning and artificial intelligence.

For R-ICMS, HDFS will be used for storage of the incoming data and will be organized as described below:

- The base folder will be “/user” by default

- Next, the environment will be designated to identify which stage of the data is being stored. This allows the same cluster for multiple environments like dev, uat and prod.
- The environment can be driven with configuration parameters within the application.
- The base Uniform Resource Locator (URL) for storage will be based on the parameters listed above
- Each data source will be designated as a separate folder with the data source name which will group the data files together. Within the data source, chronological order will be assigned with sub-folders (year, month and day)

For example, ITSIIQA traffic conditions files typical storage file names include:

```
/user/dev/traffic_conditions/2018/07/25/LinkConfig_20180725_161020.xml  
/user/prod/traffic_conditions/2018/07/25/TrafficData_20180725_161020.xml  
/user/dev/traffic_conditions/2018/07/25/ClassificationData_20180725_161020.xml
```

Each time new data is received a new file is written to HDFS and the file name will contain the date and timestamp of that new data consistent with the examples above.

For GIS data files, the base URL will be

```
/user/dev/GIS/number_of_lanes/<year-month-day>/
```

The storage locations will be listed under each data source description.

3.4.1.3.2 NoSQL (NOS-ST)

MongoDB will be used for R-ICMS data storage where applicable. MongoDB is a NoSQL database using the JSON document model and is most suitable for semi-structured and sensor-based information.

MongoDB supports the use of multiple databases allowing the design to have the flexibility of storing the data in different databases depending on the type and subject of the data.

Data coming from the various data sources will be stored in a single database and multiple collections (synonymous to tables in Relational Database Management System [RDBMS]).

A brief comparison of RDBMS to MongoDB is provided below for quick understanding.

Table 11 - SQL vs NoSQL

RDBMS (SQL Database)	MongoDB (NoSQL Database)
Relational database	Non-relational and document-oriented database
Pre-defined schema	Dynamic schema with JSON document model

RDBMS (SQL Database)	MongoDB (NoSQL Database)
Supports SQL query language	Supports JavaScript as query language Provides JavaScript client for querying
Tables	Collections with key-value pair documents
Row	Documents
Columns	Fields / Attributes
Primary and Combined indexes	Supports primary and secondary indexes including combined indexes
Primary Key	Primary Key (Default key _id provided by MongoDB itself)
Foreign Keys for relationships and query joins	No Foreign Keys and joins Allows embedding the child table data into the master document thus achieving the relationship effect by holding the data together
Group By	Aggregation
Emphasizes on ACID properties (Atomicity, Consistency, Isolation and Durability)	Emphasizes on CAP theorem (Consistency, Availability, and Partition tolerance)

Storage pattern in MongoDB:

Many of the external sources data will be hosted in MongoDB.

1. Each data set/source will be located in a separate database
2. The sub-sets of data source can be further divided into collections (similar to tables), or have additional collections depending on the need for segregating and reformatting the data for application needs

For example:

ITSIQA Traffic conditions will be stored in a database –
traffic_conditions

Within the database, a separate collection will be configured to store each feed type.

- Link_config
- Traffic_data
- Class_data

In addition, additional collections will be created to store merged / reformatted data to support queries needed by the APIs and other clients. These collections will evolve during the course of later iterations and will be documented within the respective iteration.

3.4.1.3.3 SQL Database (SQL-ST)

As indicated in the contract, the expectation is that the SQL database will be the District's standard MS SQL Server. Structured data, such as the SunGuide data, will be stored in the SQL database. Additionally, transactional data from within the R-ICMS will be stored for in the SQL database.

3.4.1.3.4 GIS Database (GIS-ST)

Incoming data intended for display on the R-ICMS map (ex. ITSQA or DMS data) will be published as a Stream Service using GeoEvent Server. The GeoEvent Server Manager will be used for publishing the stream service. While publishing the stream service, the "Store Latest" option will be used which helps to add and update a set of feature records maintained in a managed enterprise geodatabase. The Store Latest option will make sure the dynamic data is available to the UI during any failure while receiving data from Pipelines. Only the latest data, broadcasted by a stream service, will be persisted in the GeoEvent Datastore. All new connections to the GeoEvent stream service will first query the appropriate feature services to display the features on the map prior to receiving streaming dynamic data.

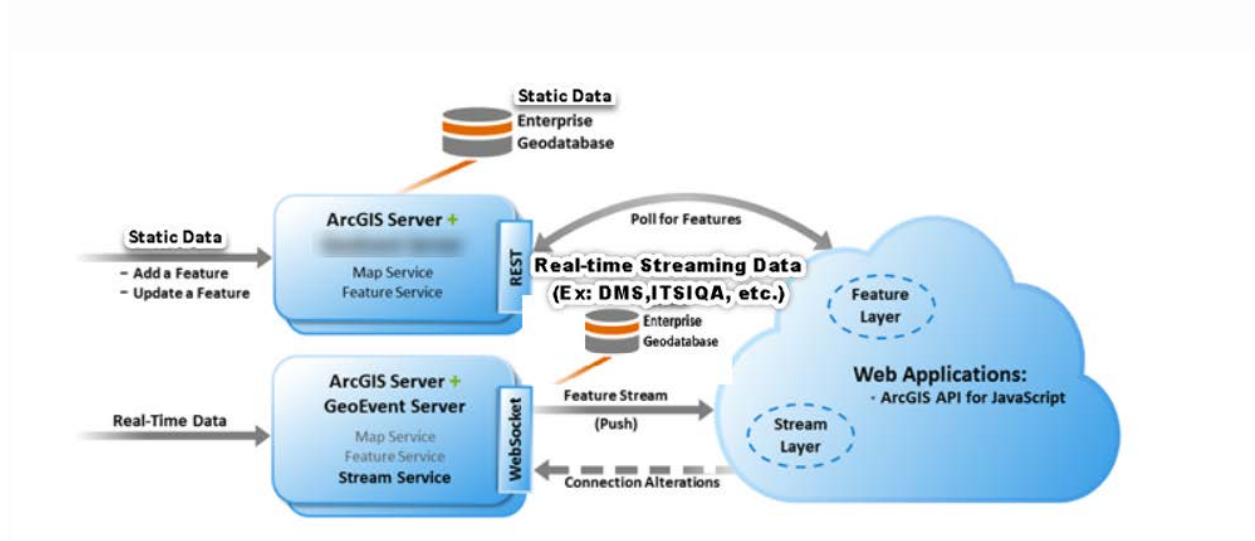


Figure 10 – GIS Data Store

The “Store Latest” option will enable the user to either choose from an existing feature or publish a new feature service, to use as a cache for maintaining the most recent event record for each uniquely received LINKID. To make use of this capability, the ArcGIS Server will use a managed enterprise geodatabase which will be created using ArcGIS Desktop or a spatiotemporal, big data store as a module of the ArcGIS Enterprise edition.

A projection is the means by which the data will be displayed using a coordinate system and data on a flat surface. Mathematical calculations will be used to convert the coordinate system used on the curved surface of earth to one for a flat surface. The shape files from the FDOT ftp site (<https://ftp.fdot.gov/file/d/FTP/FDOT/co/planning/transtat/gis/shapefiles/>) uses the ‘NAD 1983 UTM Zone_17n’ projection system. Therefore, all shape files used in R-ICMS will be converted to use the same “NAD 1983 UTM Zone_17n” projection system before publishing to the ArcGIS server.

The data flow for shape files and Geodata stores is illustrated in Figure 11 - Geodata Shape File Data Flow:

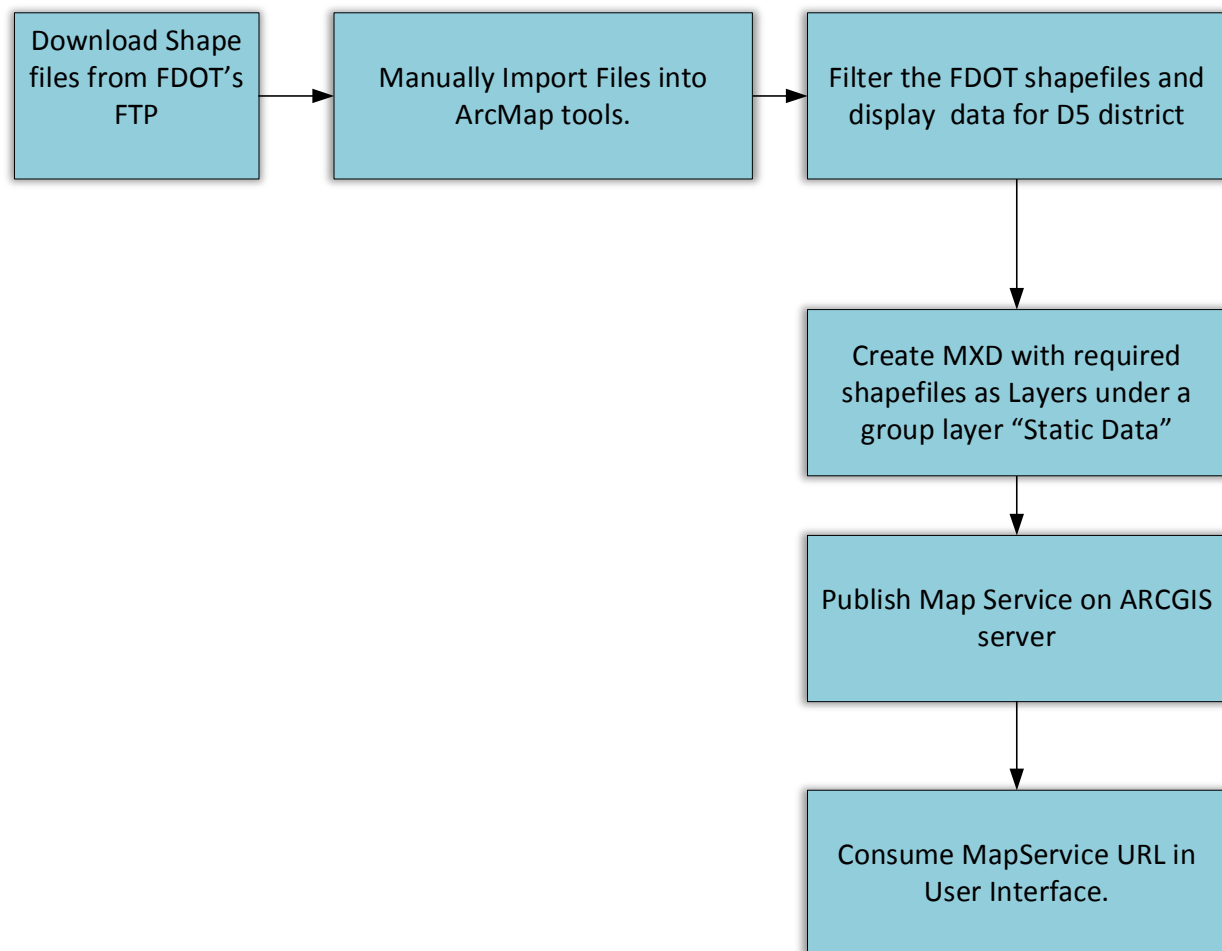


Figure 11 - Geodata Shape File Data Flow

1. The shape files will be manually downloaded from FDOT's FTP site.
2. The downloaded files will be unzipped to a desired location.

3. A .mxd (map service) (example: RICMS_StaticData.MXD) will be created using ArcMap for each shape file as a layer under the corresponding groups as shown in Figure 12 - Map Layer Shape File:

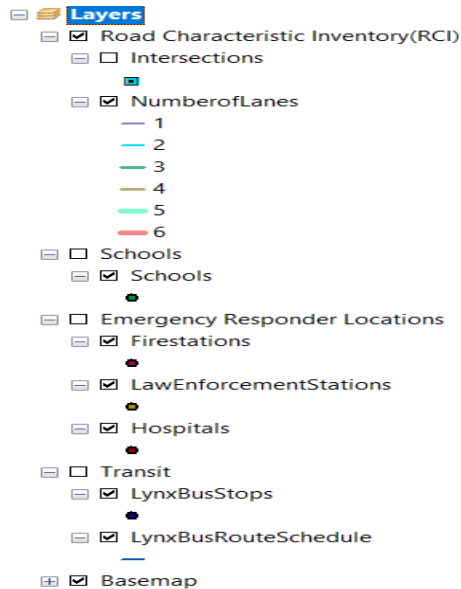


Figure 12 - Map Layer Shape File

4. The Spatial Analysis tools within the ArcToolbox will be used to analyze the layers and filter display data corresponding to FDOT D5.
5. Projections will be updated to 'NAD 1983 UTM Zone_17n' for projecting all data sources. A new enterprise geodatabase named 'RICMS_StaticData' will be created using ArcMap/ArcCatalog. A connection will be established to RICMS_StaticData.gdb using Catalog from the ArcGIS for Desktop application. The newly created enterprise geodatabase will be registered as a data store in the ArcGIS Server. The map from ArcMap will be published as a map service using 'Share as Service'.
6. A URL will be created for the published map service that will be further accessed using the ArcGIS Server Manager. The created REST URL will be consumed using corresponding methods from the ESRI JS API 3.25 user interface.

3.4.1.4 Data Services

R-ICMS data services provide both streaming and query access to both internal and external R-ICMS users. Data services supporting streaming data are responsible for maintaining a cache of the current status of the data for which they are responsible. Consumers of data services data will have the capability of setting up subscriptions to receive streaming updates as well as receiving the current state of the data. Additionally, through the use of defined filters, data consumers will have the capability to query historical data. The following identified data services for use in the R-ICMS are listed below.

There are three types of data services within the R-ICMS architecture. Generic Data Services provide access to the raw, transformed, or combined data collected from the data sources. GIS Data Services provide access to data that can be easily displayed on a map by the user interface. R-ICMS Data Services provide access to data generated by the R-ICMS itself (including data generated by the Modeling Engine, where appropriate).

3.4.1.4.1 Generic Data Services

For each type of data received from external systems, a data service will be provided to allow other services and systems access to the data. Filters will be defined for each data source to limit the way in which each type of data can be accessed. Types of generic data services include:

Table 12 - Generic Data Services

Data Service	Description
Traffic Conditions Data	Provides traffic condition data for the corridor from the ITS IQA feed
Event Data	Provides traffic event data within the corridor from SunGuide
CCTV Status	Provides the active status of Closed-circuit television (CCTV) devices
Ramp Meter Status	Provides the active status of ramp meter devices
DMS Status	Provides the status and messages on DMS devices
Connected Vehicle Unit Status	Provides the status of the connected vehicle units deployed in the network
Signal Performance Measures	Provides movement volume and arrival timing per movement related data from the signal devices
Transit AVL (Various Systems)	Provides the AVL status for various transit systems (LYNX, SunRail etc.)
Parking Data	Provides parking availability status data
Special Event Information	Provides planned Special Event information
Intersection Movement counts data	Provides details on the intersection movements (Ex. movement counts, approach/direction, number of lanes, saturation flow rates, etc.)
Origin Destination Data	Provides Origin Destination data
ATMS Controller Data	Provides signal phase timing and relevant data from ATMS Signal devices within the network
School Schedule Data	Provides school schedule information
Weather Data	Provides weather data collected from external sources
Traffic Signal Data	Provides traffic signal data from SunGuide
Roadway Characteristics Inventory (RCI) Data	Provides access to static, slowly changing Roadway Characteristics Inventory spatial data
Timing Plan Data	Provides access to current signal timing plans deployed onto controllers

IMC Data

Provides access to real-time intersection movement counts for traffic signal detection zones

3.4.1.4.1.1 Traffic Condition Data Service

The dynamic ITSQA traffic condition data will be published from Kafka streaming pipelines to the data services in-memory database, which in turn will be made available to other R-ICMS components. From the driver layer, the data will be sent to the Kafka streaming component where it will be published as topics (each data type will have its own topic) for further processing. The consumer processes will then start receiving the data from these Kafka topics and feed into the other components. This process flow has been detailed in Figure 8 – ITSQA Data Flow.

Steps regarding the data flow from the Kafka streaming pipelines are as follows:

1. A process (script/s) will be created which will subscribe to the Kafka topic (Ex. ITSQA-TrafficConditions). This process is commonly termed as "Consumer" and continuously reads data from the Kafka topics.
2. The consumer will read data from the topics and process the data to match the format needed by the client and feed it to the respective component (Ex. Modeling Engine, In-Memory database, MongoDB store).

This approach will be used to feed the dynamic data to the R-ICMS components as appropriate. The same methodology applies to the other types of dynamic data handled within the R-ICMS system.

The ITSQA traffic condition REST API details are shown in Table 13 – ITSQA API Details. This API method will be used for requesting historic data from the NoSQL data store.

Table 13 – ITSQA API Details

Title	<i>GetTrafficConditions</i>
URL	<i>/GetTrafficConditions/ begin_date/ end_date/ include_class_data/ county</i>
Method	<i>The request type</i> GET
URL Params	<i>If URL params exist, specify them in accordance with the name mentioned in the URL section. Separate into optional and required.</i> Required:- begin_date = [date time string] example: begin_date = '06-10-2018 02:30 AM' end_date = [date time string] example: end_date = '07-10-2018 02:30 AM'

	Optional- include_class_data = Boolean example: include_class_data = true County = {} example: ['seminole', 'orange']
Data Params	N/A
Success Response	Example:- Code: 200- Content: { { "update_timestamp": "", /* Date Time */ "links": [/* Array of sub documents */ { "link_id": "", /* String */ "road": "", /* String */ "direction": "", /* String */ "cross_street": "", /* String */ "county": "", /* String */ "lane_count": , /* Integer */ "speed_limit": , /* Integer */ "length": , /* Float */ "link_type": "", /* String */ "start_location": { /* Sub-document */ "latitude": , /* Double */ "longitude": /* Double */ }, "end_location": { /* Sub-document */ "latitude": , /* Double */ "longitude": /* Double */ }, "mid_points": [/* Array of sub documents */ { "latitude": , /* Double */ "longitude": /* Double */ } }, "upstream_link": "", /* String */ "downstream_link": "", /* String */ "traffic_conditions": { /* Sub-document */ "speed": , /* Integer */ "speed_data_quality": , /* Integer */ "volume": , /* Integer */ "volume_data_quality": , /* Integer */ "occupancy": , /* Integer */ "occupancy_data_quality": , /* Integer */ "travel_time": , /* Integer */ "travel_time_data_quality": /* Integer */ },

	<pre> "live_class": { /* Sub-document */ "class1": , /* Integer */ "class2": , /* Integer */ "class3": , /* Integer */ "class4": , /* Integer */ "class5": , /* Integer */ "class6": , /* Integer */ "class7": , /* Integer */ "class8": , /* Integer */ } } } } } </pre>
Error Response	{Records not found.}
Sample Call	<pre> Var response = \$http.get(config.serviceUrl + '/GetTrafficConditions/' + begindate + '/' + enddate + '/' + include_class_data + '/' + county).error(function (data, status, headers, config) { }); </pre>
Notes	Assumptions: - Request always specifies a date and time range for the query — mandatory - Other filters will be used to filter the data within the specified date and time range - Security related items will be added in the later iterations.

3.4.1.4.1.2 Transit (GTFS) Data Service

GTFS data published by the R-ICMS will be manually downloaded from the respective agencies to R-ICMS sub-systems. GTFS data at rest API details are as follows. This API method will be used for requesting historic data from the NoSQL data store.

Title	<i>GetGTFSdata</i>
URL	<i>/GetGTFSdata /AgencyName/begin_date/end_date/</i>
Method	<i>The request type</i> GET
URL Params	<i>If URL params exist, specify them in accordance with name mentioned in URL section. Separate into optional and required.</i> Required: AgencyName = String Example: Agency = SunRail begin_date =[date time string] example: begin_date = '06-10-2018 02:30 AM'

	end_date =[date time string] example: end_date = '07-10-2018 02:30 AM'
Data Params	NA
Success Response	Example: Code: 200 Content: <pre>{ (Returned GTFS JSON objects) }</pre>
Error Response	{Records not found.}
Sample Call	<pre>Var response = \$http.get(config.serviceUrl + '/ GetGTFSdata /' + begindate + '/' + enddate + '/' + Agency+ ').error(function (data, status, headers, config) { });</pre>
Notes	Assumptions: - Request always specifies Agency Name for the query. - Request specifies optional date range for historic data - When no date range is specified, then the current state of data will be made available - Security related items will be added in the later iterations.

3.4.1.4.1.3 SunGuide - DMS Data Service

DMS data at rest API details are as follows. This API method will be used for requesting historic data from the NoSQL data store.

Title	<i>GetDMSdata</i>
URL	<i>/ GetDMSdata/ begin_date/ end_date/current/roadway</i>
Method	<i>The request type</i> GET
URL Params	<i>If URL params exist, specify them in accordance with name mentioned in URL section. Separate into optional and required.</i> Optional begin_date =[date time string] example: begin_date = '06-10-2018 02:30 AM'

	end_date =[date time string] example: end_date = '07-10-2018 02:30 AM' current = Boolean example: current = true This is also default when date range is not present roadway = [] example: ['I-4','408']
Data Params	NA
Success Response	Example: Code: 200 Content: { (Returned DMS JSON objects) } }
Error Response	{Records not found.}
Sample Call	<pre>Var response = \$http.get(config.serviceUrl + '/ GetDMSdata /' + begindate + '/' + enddate + '/' + current + '/' + roadway).error(function (data, status, headers, config) { });</pre>
Notes	Assumptions: - Request specifies optional date and time range for the query - Request specifies optional current parameter. This will be defaulted to True, when a date range is not specified. - Current = True will provide the current status of the DMS signs and messages - Other filters will be used to filter the data within the specified criteria being applied. - Security related items will be added in the later iterations.

3.4.1.4.2 GIS Data Services

Two types of GIS data services will be made available for the purpose of providing map data. These services will be responsible for sending the correct map tiles to the user interface depending on the user's current zoom level and viewing location. The services are described in Table 14 - GIS Data Services.

Table 14 - GIS Data Services

Data Service	Description
GeoEvent Service	Provides status information for map features (e.g. DMS messages). This service provides the dynamic

	information for the static features provided by the ArcGIS Data Service.
ArcGIS Service	Provides map tiles and feature (e.g. DMS location) information. This service provides the effectively static (rarely updating) data for rendering maps.

3.4.1.4.2.1 GeoEvent Server (GEO-DS)

Real-time data will be streamed to a map page using the GeoEvent Server and web sockets. The GeoEvent Server Flow is illustrated in Figure 13 – GeoEvent Server Flow.

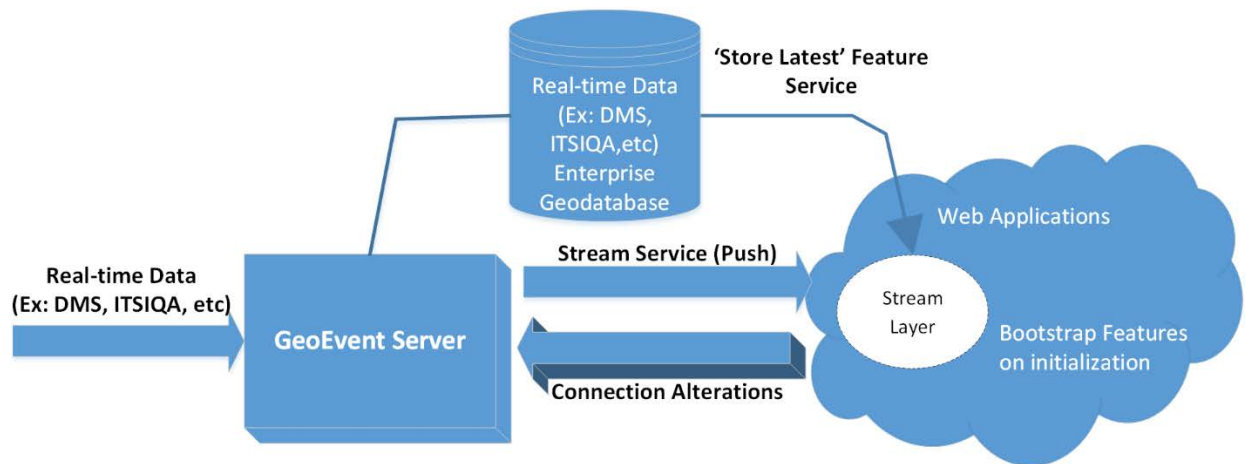


Figure 13 – GeoEvent Server Flow

Once the feed definition is created, an output connector will be created to push data from the Kafka feed into the stream service. The default update interval (in seconds) of 0.01 seconds specifies that events routed to the stream service will be batched and broadcasted one hundred times each second, which has to be updated on the map within 3 seconds for the ITSQA data.

A stream service will be published and an output connector will be created.

After creating the input and output connectors, a GeoEvent service will be published. The ITSQA data will flow from Kafka to the service layer and will be consumed by the GeoEvent service. Figure 14 – ITSQA GeoEvent Data Flow is the dataflow diagram for ITSQA data.

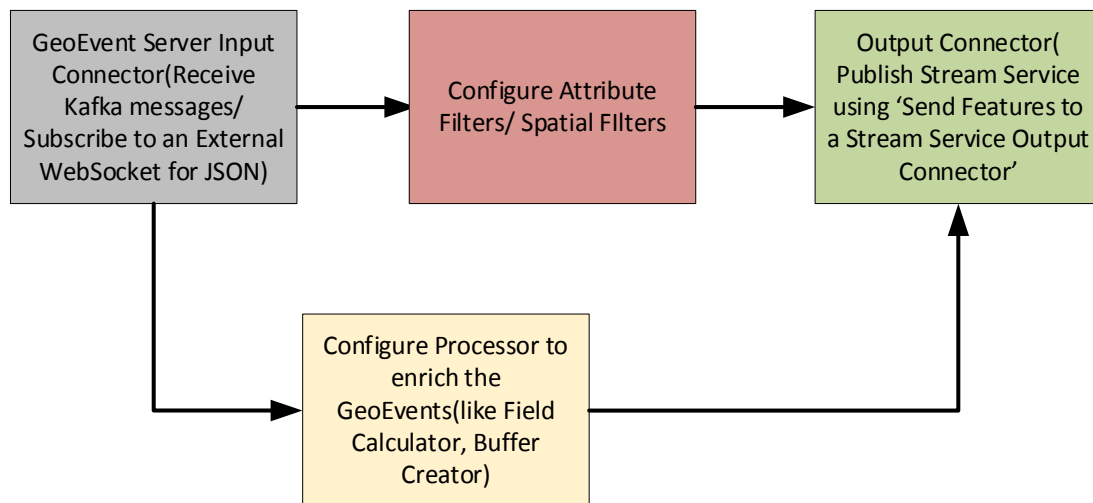


Figure 14 – ITSQA GeoEvent Data Flow

In ArcGIS server manager a Stream service will be deployed and the service endpoint (REST API) will be consumed from the user interface map page.

Table 15 - Stream Service Details shows the stream service API details:

Table 15 - Stream Service Details

Title	<i>StreamLayer</i>
URL	<i>/StreamLayer /url/options</i>
Method	<i>The request type</i> GET
URL Params	<i>If URL params exist, specify them in accordance with the name mentioned in the URL section. Separate into optional and required.</i> Required: url=[string] example: url= https://geoeventsample1.esri.com:6443/arcgis/rest/services/LABus/StreamServer Optional options=[array] example: options= { outFields: ["route_id"], infoTemplate: new PopupTemplate({ title: "Route {route_id}", description: "This bus is {expression/arcade-distance} miles from the convention center.", fieldInfos: [{ fieldName: "expression/arcade-distance", format: { digitSeparator: true,

	<pre>places: 2 }</pre>
Data Params	NA
Success Response	<i>What should the status code be on success and is there any returned data? This is useful when people need to know what their callbacks should expect!</i> Example: Code: 200 Content: { StreamLayer }
Error Response	There is a problem on loading the layer. Please refresh the page and try again.
Sample Call	<pre>var url = "https://geoeventsample1.esri.com:6443/arcgis/rest/services/LABus/StreamServer"; var layer = new StreamLayer(url, { outFields: ["route_id"], infoTemplate: new PopupTemplate({ title: "Route {route_id}", description: "This bus is {expression/arcade-distance} miles from the convention center.", fieldInfos: [{ fieldName: "expression/arcade-distance", format: { digitSeparator: true, places: 2 } }]}, expressionInfos: [{ name: "arcade-distance", title: "Distance from convention center", expression: document.getElementById("arcade-distance").text }] });</pre>
Notes	None.

3.4.1.4.2.2 ArcGIS Service

A REST API based FeatureLayer service, part of the ArcGIS data service layer, will be made available from the ArcGIS server. This API method will be used for fetching static data from the ArcGIS map service. The FeatureLayer service is described in Table 16 - ArcGIS Feature Service API Details.

Table 16 - ArcGIS Feature Service API Details

Title	<i>FeatureLayer</i>
URL	<i>/FeatureLayer /url/options</i>
Method	<i>The request type</i> GET

URL Params	<i>If URL params exist, specify them in accordance with the name mentioned in the URL section. Separate into optional and required.</i> Required: url=[string] example: url= https://services.arcgis.com/V6ZHFr6zdgNZuVG0/arcgis/rest/services/Landscape_Trees/FeatureServer/0 Optional options=[array] example: options= { mode: FeatureLayer.MODE_SNAPSHOT, outFields: ["NAME", "POP2000", "POP2007", "POP00_SQMI", "POP07_SQMI"] }
Data Params	NA
Success Response	<i>What should the status code be on success and is there any returned data? This is useful when people need to know what their callbacks should expect!</i> Example: Code: 200 Content: { FeatureLayer }
Error Response	There is a problem on loading the layer. Please refresh the page and try again.
Sample Call	<pre>var southCarolinaCounties = new FeatureLayer("https://sampleserver1.arcgisonline.com/ArcGIS/rest/services/Demographics/ESRI_Census_USA/ MapServer/3", {s mode: FeatureLayer.MODE_SNAPSHOT, outFields: ["NAME", "POP2000", "POP2007", "POP00_SQMI", "POP07_SQMI"] });</pre>
Notes	None.

Basemaps published in the ArcGIS server will provide the foundation to visualize and analyze data using the ArcGIS JavaScript API from the user interface. Basemaps will be configured as a vector tile service, which will be an ArcGIS Server web service originating from a vector tile package in ArcGIS Pro. This vector tile service (also known as vector tile layers) will be used to share and consume vector tiles to the map page in a web user interface. Table 17 - ArcGIS Vector Layer Server API describes the API.

Table 17 - ArcGIS Vector Layer Server API

Title	<i>VectorTileLayer</i>
URL	<i>/ VectorTileLayer/url/options</i>
Method	<i>The request type</i> GET
URL Params	<i>If URL params exist, specify them in accordance with the name mentioned in the URL section. Separate into optional and required.</i> Required: url=[string] example: url= https://basemaps.arcgis.com/arcgis/rest/services/World_Basemap_v2/VectorTileServer Optional options=[array] example: options= { maxScale:map.getScale(); }
Data Params	NA

Success Response	<i>What should the status code be on success and is there any returned data? This is useful when people need to know what their callbacks should expect!</i> Example: Code: 200 Content: { VectorTileLayer }
Error Response	There is a problem on loading the layer. Please refresh the page and try again.
Sample Call	<pre>var vlayer = new VectorTileLayer("https://www.arcgis.com/sharing/rest/content/items/4cf7e1fb9f254dca9c8fbadb15cf0f8/reso urces/styles/root.json"); map.addLayer(vlayer);</pre>
Notes	None.

3.4.1.4.3 R-ICMS Data Services

The R-ICMS will make data generated from within the R-ICMS available for consumption. The following data services will be provided for other systems and components to access R-ICMS generated data.

Table 18 - R-ICMS Data Services

Data Service	Description
Predictive Engine Data	Provides data produced by the Predictive Engine; e.g., modeling engine inputs and results
Business Rules Engine Data	Provides data produced by the Business Rules Engine; e.g., business rule execution, business rule initiated response plan evaluations
Evaluation Engine Data	Provides data updates regarding evaluation results; e.g., Measures of Effectiveness (MOE) for evaluated response plans based on current data and mesoscopic modeling projections of network impacts such as total delay, link delays, volumes of various types of vehicles
Response Plan Data	Provides response plan data and updates
Event Data	Provides event data and updates
SOT Data	Provides SOT data and updates
Metadata	Provides metadata regarding the data sources.

3.4.2 DSS/Business Services

The R-ICMS will include various business services in order to achieve the desired overall functionality of the system. Business services will be transactional in nature and will serve as an intermediary between the user interface and the big data environment. Business services may connect to data services to receive streaming data or may interface directly with pipelines if needed to ensure adequate response to changing conditions. The following identified business services for use in the R-ICMS are listed below.

3.4.2.1 Session Authentication (SA-BS)

The Session Authentication Business Service will handle login attempts to the R-ICMS. It will tie directly to FDOT's Active Directory service to authenticate users to the FDOT domain. As such, all R-ICMS users will need to have active FDOT domain accounts. Additionally, the SA-BS will be responsible for determining the permissions of the user, creating and storing a signed token in an in-memory data store, and returning the token to the user interface for use in further requests.

3.4.2.2 User Personalization (UP-BS)

The User Personalization Business Service will be responsible for storing and retrieving the user's preferences for their individual user interfaces. Preferences will include, but not be limited to the user's default view, dialogs, and dialog locations.

3.4.2.3 Admin (ADM-BS)

The Admin Business Service will provide the back-end functionality necessary for user account management. It will provide an interface for user administration including permissions, roles, agencies, agency groups, and agency devices.

3.4.2.4 Alert (ALT-BS)

The Alert Business Service will provide an interface for different internal systems to provide alerts to appropriate users. Functionalities include alerting users when response plans are ready for review, data outage alerts, etc. The Alert Business Service will send notifications/alerts via the user interface as well as through email notifications.

3.4.2.5 Orchestration (ORC-BS)

The Orchestration Business Service will serve as the central application for service management. This includes starting and stopping services, monitoring the status of system components, and dynamically setting log levels. Application orchestration may involve multiple nodes, which may be standalone hosts or virtual machines, and components may be deployed directly or as

containers running on the nodes. In the critical design phase, it will be determined if application management including high-availability deployment may benefit from using open source enterprise containerization and management, such as Docker and Kubernetes, possibly re-using the Executive Handler functionality from SunGuide, or whether to create custom orchestration modules for this system.

3.4.2.6 Reporting (RPT-BS)

The Reporting Business Service will provide generic capabilities to generate reports from templates. It will primarily be a COTS application which will connect to the appropriate data stores in order to generate custom reports. A base set of templates will be determined in critical design review.

3.4.2.7 Business Rules Engine (BRE-BS)

The Business Rules Engine Business Service will be responsible for managing and monitoring the business rules that trigger the selection of response plan sets for modeling and evaluation. The Business Rules Engine Business Service will serve as the business service layer for rule management and storage. Additionally, it will be a data consumer for the SunGuide event data to determine if R-ICMS response plans should be evaluated.

3.4.2.8 Predictive Engine (PRE-BS)

The Predictive Engine Business Service will serve as the wrapper for the external modeling engine. It will primarily be responsible for the API used to communicate information to/from the external modeling engine.

3.4.2.9 Evaluation Engine (EVE-BS)

The Evaluation Engine Business Service will be responsible for receiving results from the PRE and the external modeling engine. The EVE-BS will receive data regarding response plans and their relevant impacts on the system and will apply a custom algorithm to score the response plans appropriately. Additionally, the EVE-BS will be capable of receiving result data regarding signal optimizations and their relevant impact on the system.

3.4.2.10 Event (EM-BS)

The Event Business Service will be responsible for managing R-ICMS created events as well as monitoring SunGuide created events. This service will be responsible for determining if a correlation exists between different events and determining if the Business Rules Engine should evaluate an event. The Event Business Service will be responsible for sending events to SunGuide with the appropriate R-ICMS response plan elements to be activated. Additionally, the Event

Business Service will be the business interface for response plan selection, agency approval, and activation.

3.4.2.11 Response Plan (RP-BS)

The Response Plan Business Service will be responsible for response plan configuration management. The RP-BS will be the back-end service dedicated to response plan creation, modification, and deactivation.

3.4.2.12 Signal Optimization Timing (SOT-BS)

The Signal Optimization Timing Business Service will serve as the back-end process for handling SOT requests. It will host the API to call the HCS7 COTS tool and will connect to the PRE-BS and external modeling engine to model signal timing changes. It will be responsible for allowing authorized users to sign signal timing plans. Additionally, it will be responsible for creating and making available the files containing the signal timing plans needed for download into external traffic signal systems.

3.4.2.12.1 Overview and Goals

Running a set of traffic signals in coordination during specified time periods, as a corridor, currently requires extensive manual data collection, analysis, and configuration by traffic engineering professionals. Because of the high degree of resources required, re-evaluation of corridors is typically limited to every few years at best, and it is impossible to keep pace with changing real-world traffic environments and demand.

The SOT-BS component of the R-ICMS will allow flexible and continuous re-evaluation of traffic signal corridors with minimal effort by the combination of leveraging a real-time traffic data ingestion platform, automating a COTS signal timing analysis package, and providing a streamlined user interface.

Components and functionality for real-time data ingestion and user interfaces are not included in iteration 1. The focus of iteration 1 is the evaluation and automation of a selected COTS signal timing analysis package, HCS7. This section describes the design of components and functionality required to achieve the following goals for development iteration 1:

1. The SOT-BS will invoke the HCS7 application as a backend for R-ICMS signal corridor timing optimization
2. The SOT-BS will generate a set of optimal timing plans for a test intersection

HCS7 is a standalone application developed for Microsoft Windows, which may be used via a Graphical User Interface or a user-driven Command Line Interface (CLI). To use HCS7 as a signal analysis backend for the R-ICMS web application, the SOT-BS will implement a REST Hyper Text Transfer Protocol Secure (HTTPS) service with an API that defines requests to handled. Interaction with the SOT-BS API by internal services and R-ICMS user interfaces will trigger the generation of HCS7 CLI processes, allowing end users to run HCS7 through R-ICMS.

While HCS7 is being evaluated as part of iteration 1, end user requirements for SOT have not yet been finalized. As user needs are confirmed, selection and evaluation of other COTS packages may be required to meet required needs. This evolution of SOT requirements requires a flexible design for the SOT-BS.

The design process for SOT includes:

- Ongoing review and refinement of the SOT user requirements
- Ongoing review and evaluation of the HCS7 software
- Definition of user interactions and system functionality
- Definition of input data stores and raw data sources
- Definition of operational and output data stores
- Specification of APIs, logic, and control flow for the business service layer
- Specification of APIs, stores, and schemas for data services layer
- Selection of implementation and deployment frameworks and tools

3.4.2.12.2 Operations

Sequence diagrams have been created to illustrate the overall design of the SOT system operations, including user interaction, API calls and control flow of the SOT-BS, and interfaces to data stores via R-ICMS data services layer. The following sequence diagrams show detailed design of the R-ICMS SOT system:

Figure 52 – SOT Detailed Create Corridor Sequence Diagram

Figure 53 – SOT Detailed SOT-DS Intersection Geometry Diagram

Figure 54 – SOT Detailed Modify Corridor Sequence Diagram

Figure 55 – SOT Detailed Corridor Optimization 1

Figure 56 – SOT Detailed Corridor Optimization 2

Figure 57 – SOT Detailed Deploy Corridor

Figure 58 – SOT Detailed SOT-BS Logging

Figure 73 – Timing Plan Data Service

3.4.2.12.3 Deployment

The following diagrams show the proposed deployment of physical and virtual components for the R-ICMS, including the SOT system:

Figure 74 – Service Host Deployment Diagram

Figure 75 – Containerized Service Orchestration Diagram

3.4.2.12.4 API Specifications

R-ICMS business and data service APIs will be specified using OpenAPI Specification (OAS), a standard, programming language-independent interface description for APIs. Open source tools are available to generate both server and client code from OAS in a variety of programming languages and frameworks. The NSwag code generator and libraries, which support generating ASP.NET Core and TypeScript bindings from the common specifications, will be used for this project.

The following sections details the API specifications to be implemented in the SOT business and data services. Although they are not specified here, full SOT functionality relies on other internal services, such as the timing plan data service and the alerting and logging business services.

3.4.2.12.4.1 SOT-BS

Development of iteration 1 will include partial implementation of the SOT-BS API. API routes, example parameters, and functionality are listed below.

Possible invocations include:

Route: POST /api/sot/bs/v1.0/corridor

Route: PUT /api/sot/bs/v1.0/corridor/{id}

An example of a set of Body Parameters JSON is:

```
{Corridor: {id: 123, name: "US 441 weekly", direction: "northbound", maxPlans: 8, intersections: [10, 11, 12, 13, 14, 15, 16], days: [0, 1, 1, 1, 1, 1, 0], start: "00:00", end: "23:50", areaType: "other", baseSaturationFlow: 1900, phf: 0.92}}
```

Possible responses to the invocation might include one of the following:

Response JSON: {code: Success, message: "Requested corridor saved"}

Response JSON: {code: CorridorInvalid, message: "Invalid corridor provided"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method validates a corridor configuration before it is inserted (POST) or updated (PUT) into the SOT operational data store by invoking an associated data service route. This method is triggered by the user interface when a user submits a request to save a corridor configuration.

Route: POST /api/sot/bs/v1.0/corridor/optimization/run

Example Body Parameters JSON:

```
{Corridor: 123, Optimization: {}}
```

Response JSON: {code: Success, message: "Corridor optimization is running"}

Response JSON: {code: OptimizationInvalid, message: "Invalid optimization provided"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method triggers the server-side SOT processes to run an optimization. The user interface sends this request when the user clicks the appropriate button to run an optimization. Multiple processes are initiated on the R-ICMS servers to handle each optimization request, and users are notified asynchronously when all processes are complete. Servers-side functionality includes:

1. Fetch IMC demand volumes for selected historical period and the corridor activation time
2. Calculate or generate capacity for intersection movements
3. Generate and store Time Grouped Demand Clusters (TGDC) of IMC time interval data which have similar volume/capacity profiles, for up to K timing groups, where K represents the maximum number of timing plans to be generated by SOT
4. Generate SOT input files for each time group by combining the IMC data with intersection geometry, signal timing parameters, and selected optimization parameters
5. Invoke the HCS7 back end in separate processes for each input file
6. Monitor for the results from each invocation of HCS7
7. Store HCS7 resultant output files
8. Extract timing plans from HCS7 results
9. Store corridor optimization results and timing plans
10. Send notifications to subscribed consumers

3.4.2.12.4.2 SOT-DS

Development of iteration 1 will include partial implementation of the SOT-DS API. API routes, example parameters, and functionality are listed below.

Route: POST /api/sot/ds/v1.0/corridor

Route: PUT /api/sot/ds/v1.0/corridor/{id}

Example Body Parameters JSON:

```
{Corridor: {id: 123, name: "US 441 weekly", direction: "northbound", maxPlans: 8, intersections: [10, 11, 12, 13, 14, 15, 16], days: [0, 1, 1, 1, 1, 1, 0], start: "00:00", end: "23:50", areaType: "other", baseSaturationFlow: 1900, phf: 0.92}}
```

Response JSON: {code: Success, message: "Requested corridor saved"}

Response JSON: {code: CorridorInvalid, message: "Invalid corridor provided"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method inserts (POST) or updates (PUT) a corridor in the SOT operational data store. This method is triggered from the SOT business service after validation of the corridor configuration.

Route: POST /api/sot/corridor/optimization/capacity

Route: PUT /api/sot/ds/v1.0/corridor/optimization/capacity/{id}

Example Body Parameters JSON:

```
{Capacity: [{intersection: 123, movement: "northbound thru", period: 1, capacity: 1260},
{intersection: 123, movement: "northbound left", period: 1, capacity: 69}, {intersection:
123, movement: "northbound right", period: 1, capacity: 1104}, ...]}
```

Response JSON: {code: Success, message: "Capacity data for corridor saved"}

Response JSON: {code: CapacityError, message: "Error storing capacity for corridor"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method stores capacity for intersection movements calculated or generated for a corridor optimization.

Route: POST /api/sot/ds/v1.0/corridor/optimization/tgdc

Route: PUT /api/sot/ds/v1.0/corridor/optimization/tgdc/{id}

Example Body Parameters JSON:

```
{TGDC: [{period: 1, tgdc: 1}, {period: 2, tgdc: 1}, ..., {period: 13, tgdc: 2}, {period: 14, tgdc:
2}, ...]}
```

Response JSON: {code: Success, message: "TGDC data for corridor saved"}

Response JSON: {code: TGDCError, message: "Error storing TGDC for corridor"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method stores TGDC of IMC data for historical time intervals used by a corridor optimization.

Route: POST /api/sot/ds/v1.0/corridor/optimization/input

Example Body Parameters JSON:

```
{File: {Path: "/path/to/input.xml", ContentType: "application/xml"}}
```

Response JSON: {code: Success, message: "Input file for corridor optimization saved"}

Response JSON: {code: OptInputError, message: "Error storing optimization input file"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method stores COTS signal analysis back end input files generated for a corridor optimization in a NoSQL document store for operational use and in HDFS for offline analytics use.

Route: POST /api/sot/ds/v1.0/corridor/optimization/output

Example Body Parameters JSON:

{File: {Path: "/path/to/output.xml", ContentType: "application/xml"}}

Response JSON: {code: Success, message: "Output file for corridor optimization saved"}

Response JSON: {code: OptOutputError, message: "Error storing optimization output file"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method stores COTS signal analysis back end output files generated for a corridor optimization in a NoSQL document store for operational use and in HDFS for offline analytics use.

Route: POST /api/sot/ds/v1.0/corridor/optimization/results

Example Body Parameters JSON:

{corridor: 123, optimization: 123, planSet: 123, createdBy: "SOT HCS7", createTime: "20180701-00:00:00", moe1: 0.82, moe2: 97, recommended: "Yes"}

Response JSON: {code: Success, message: "Results for corridor optimization saved"}

Response JSON: {code: OptResultsError, message: "Error storing optimization results"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method formatted results of a corridor optimization after evaluation by the modeling engine.

Route: POST /api/sot/ds/v1.0/corridor/optimization/plans

Example Body Parameters JSON:

{plan: 123, planSet: 123, createdBy: "SOT HCS7", createTime: "20180701-00:00:00", updatedBy: "Bob Barker", updateTime: "20180701-08:32:15", timingPlan: {timing plan fields...} }

Response JSON: {code: Success, message: "Timing plans for corridor optimization saved"}

Response JSON: {code: OptPlansError, message: "Error storing optimization timing plans"}

Response JSON: {code: Unauthorized, message: "You do not have permission to perform this operation"}

Functionality:

This method stores timing plans extracted from the results of a corridor optimization.

3.4.2.12.5 SOT Data stores

3.4.2.12.5.1 SQL-ST

A relational database implemented in MS SQL Server will be used to store highly structured data used and generated by the R-ICMS system. The data tables, fields, and associated relationships required for SOT operations are available as an Entity-Relationship Diagram (ERD) in the following external document:

Figure 60 – SOT Operational Database Diagram

3.4.2.12.5.2 NOS-ST

Most COTS SOT products use file-based input/output (I/O) in a variety of formats, such as XML, CSV, and UTDF. A MongoDB document data store is ideally suited to store these semi-structured documents of diverse types. Document collections required for SOT:

1. SOT-Templates – This collection contains templates for optimization configurations that may be copied and populated with static and dynamic information to generate input files for a SOT optimization runs. These documents will be indexed by a unique template identifier.
2. SOT-Configs – This collection contains input files generated for a SOT optimization run. These documents will be indexed by unique identifiers for the corridor and optimization run and the date and time that the optimization was run.
3. SOT-Results – This collection contains the resulting output files from a SOT optimization run. These documents will be indexed by unique identifiers for the corridor and optimization run and the date and time that the optimization was run.

3.4.2.12.5.3 FS-ST

Long-term storage of SOT data will be in the HDFS data store, where a hierarchical directory structure will be defined to store the SOT input and output files for analytical use.

3.4.2.12.6 User Permissions

The following user permissions will be required for access to SOT functionality.

1. Set special days – This may be per agency or systemwide; remaining permissions will use the device and agency profiles in order of precedence
2. Set intersection defaults
3. Set intersection details

4. View intersection details
5. Set corridor details – Note: if a user doesn't have "set intersection details" permission, they will not be able to enter or modify intersection data used by SOT corridor analysis
6. View corridor details
7. Deploy corridor
8. Set optimization details
9. View optimization details
10. Run optimization
11. Set timing plan details
12. View timing plan details
13. Sign timing plan

3.4.2.12.7 Testing & Validation

Because iteration 1 is a partial implementation of SOT, testing and demonstration requires:

- A prepared optimization configuration file for an intersection
- A prepared set of TGDC data for the intersection
- A simple user interface with a button which will trigger the API methods required to generate the input data, run optimization, and store the results and extracted timing plans

The following use case will be implemented to demonstrate the generation of a set of optimal signal timing plans using the HCS7 as the SOT back end:

Figure 59 – SOT Use Cases

3.4.3 User Interface

The user interface will be the primary interface to the DFE and the DSS for R-ICMS users. It will be comprised of the necessary components for display of the data that the Data Services layer provides. It will communicate with services in the business layer for appropriate transactional requests.

The following User Interface components have been identified.

Angular JS, HTML5, CSS, Typescript and bootstrap will be used as development languages for the User Interface (UI). The Model View Controller (MVC) design pattern will be used for the UI implementation. The folder structure of the application can be found in the coding standards document. For the map page and streaming services ArcGIS API for JavaScript version [3.25](#) will be used.

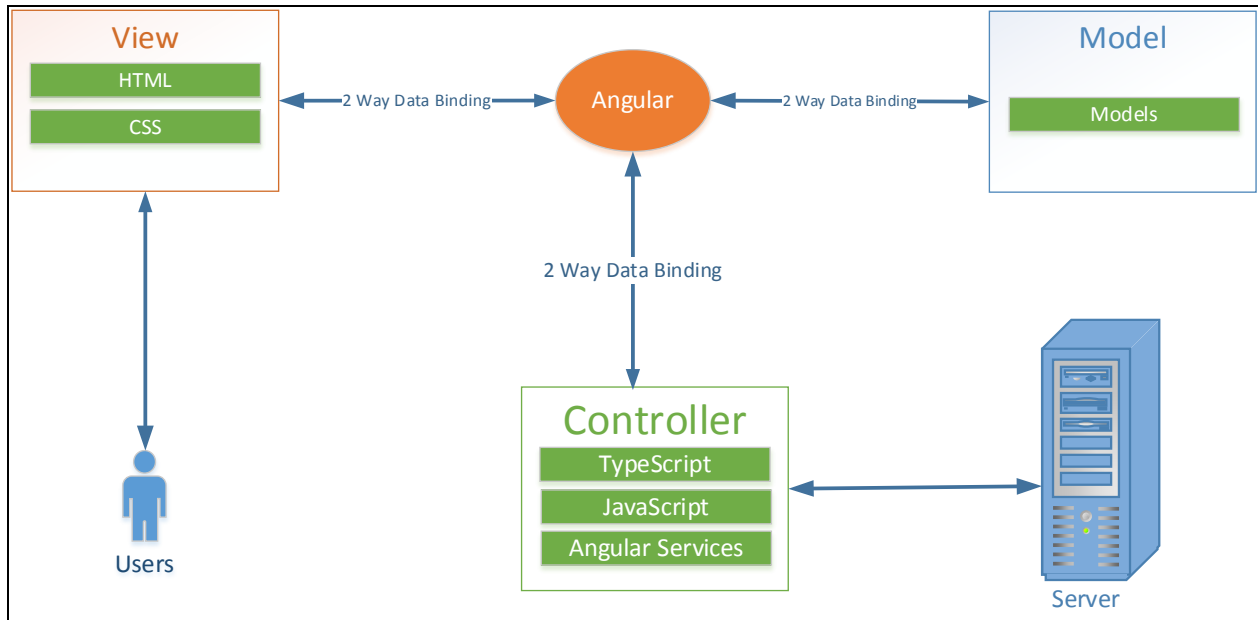


Figure 15 – Angular MVC

The overall navigation hierarchy of the application is illustrated in Figure 16 – User Interface Navigation Hierarchy.

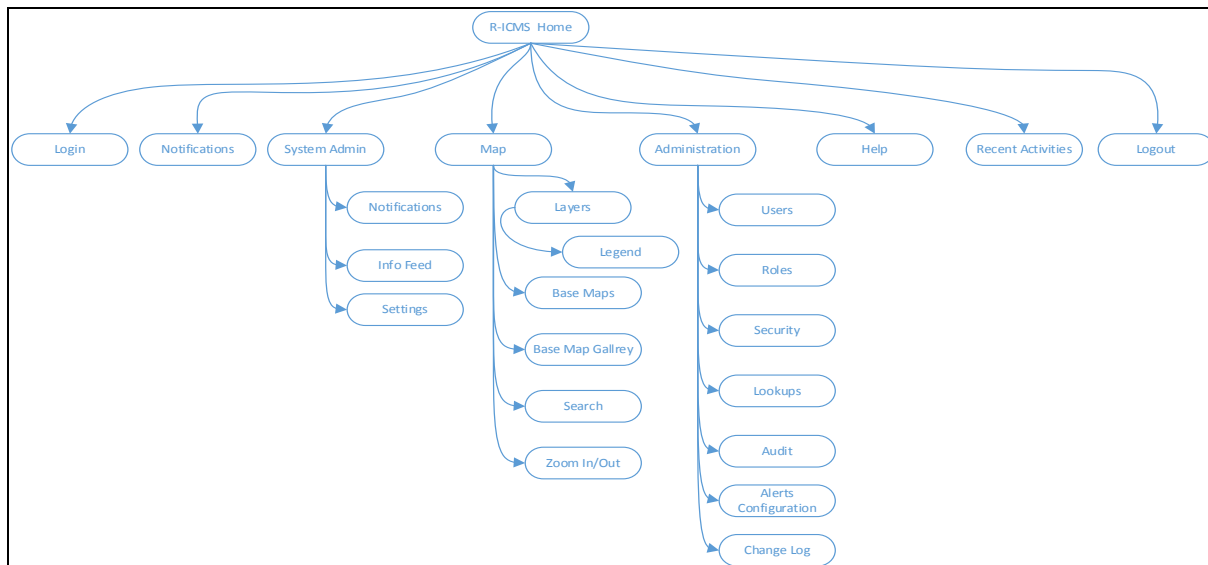
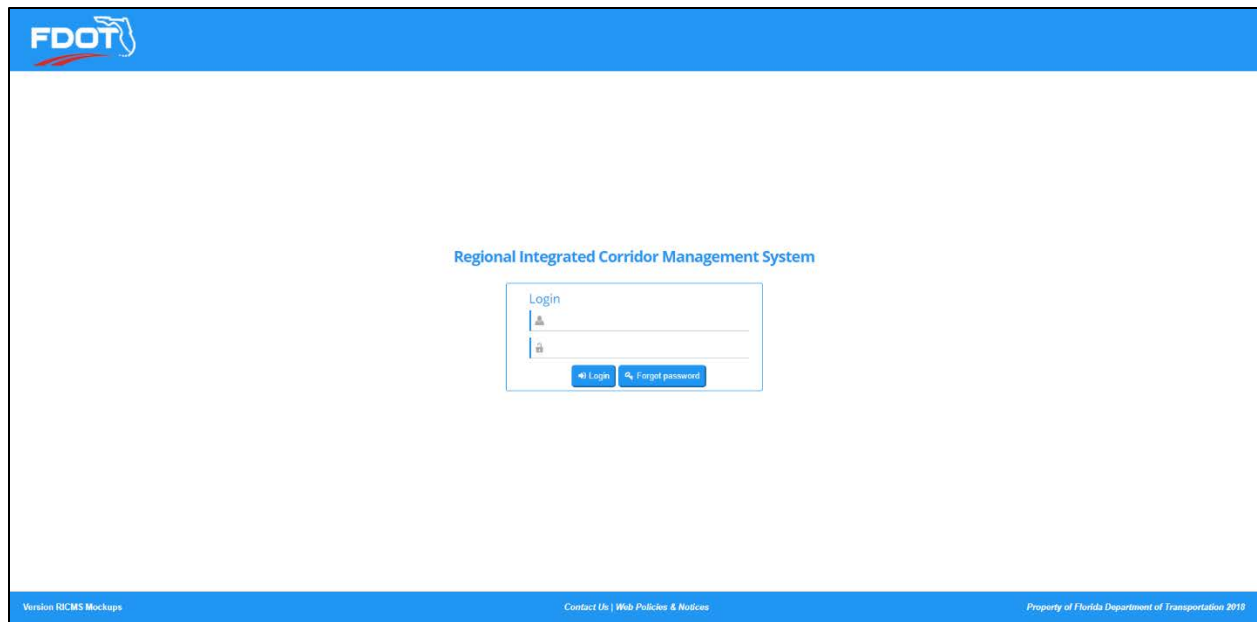
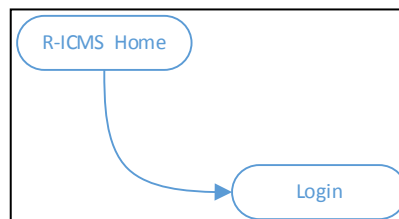


Figure 16 – User Interface Navigation Hierarchy

3.4.3.1 Login (Login-UI)

Upon navigating to the base R-ICMS website, the user will be redirected to the login screen. The user can log into the R-ICMS Application using the login screen shown below. After the user logs in to the R-ICMS, the user will be redirected to the designated landing page, which for most users will be the map page.

**Figure 17 - Login Screen****Figure 18 - Login Flow**

3.4.3.2 Map (MAP-UI)

The map component will be one of the primary entry points into the UI. It will connect to and process data from the GeoEvent service and the ArcGIS service. It will display data to the user based on permissions and current filters. Additional global filters will be selectable which will apply to all data shown on the map (as applicable to the filter). Users will be able to pan and zoom per common web map interfaces. Two types of data will be shown on the map. Data associated with pre-defined locations, such as speed of a specific link (and therefore its color) will be linked in the UI itself with that portion of the map based on an ID of that portion of the map (e.g., the pixels making up the link). Data not associated with predefined locations on the map (such as AVI data from a transit vehicle) will need to be drawn on top of the map by the UI. These two types of data will be handled by different data services but will be drawn by the UI onto the same map.

- The user will be able to Search on the map

- The user will be able to zoom in or zoom out the map and pan the map
- The user will be able to view the basemap, static data and dynamic data on the map by toggling the respective layers
- User will be able to view static data such as:
 - RCI data
 - Intersections
 - Number of Lanes
 - Schools
 - Emergency Responder Locations
 - Transit
- The user will be able to view dynamic data such as
 - Traffic conditions
- The user will be able to view a map legend for a selected layer

The following diagram depicts the dataflow for static and dynamic data from the data store to map page:

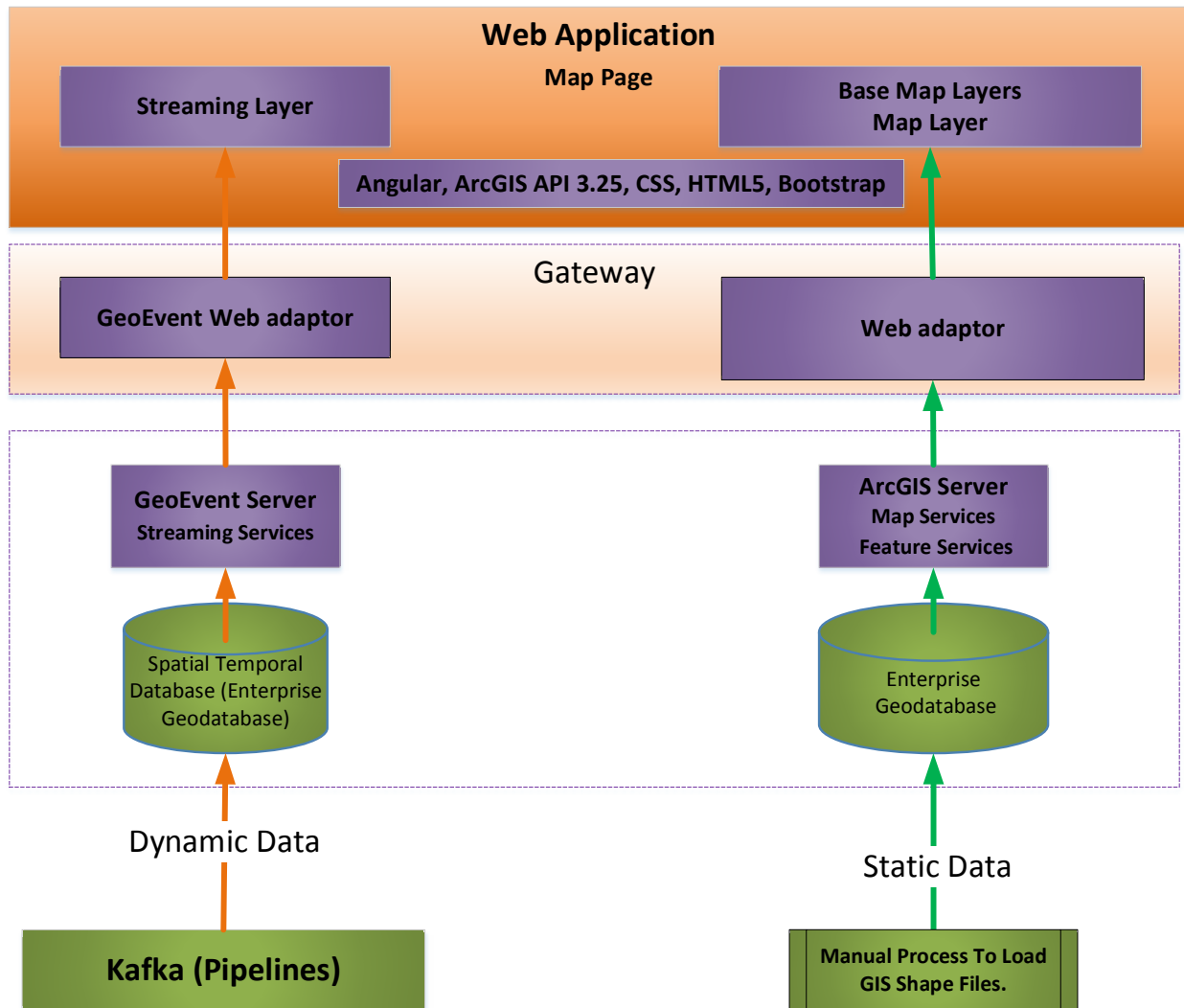


Figure 19 - Map Data Flow

The following screenshots show examples of the screens that users will see when viewing the map portion of the R-ICMS.

3.4.3.2.1 Main screen

Figure 20 - Main Screen depicts the initial screen the user sees after logging on.

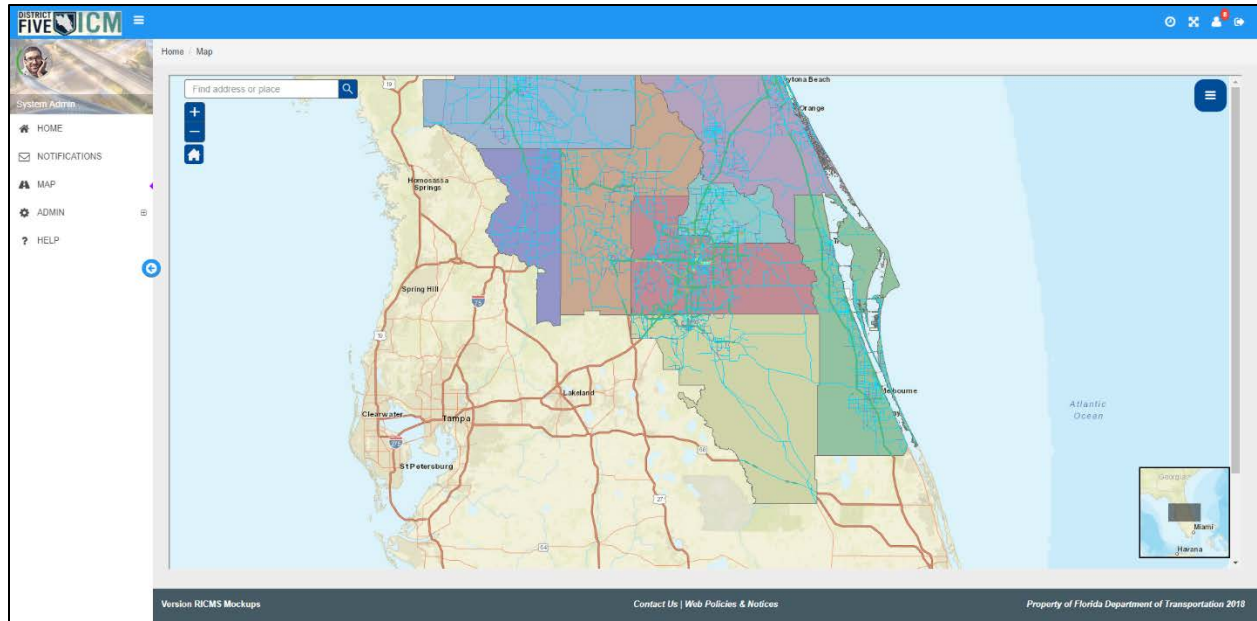


Figure 20 - Main Screen

3.4.3.2.2 Search:

Users will be able to search by address, street name, zip code and road name. The map will be zoomed in/out to an extent to show all the search results. Figure 21 - Search Screen depicts the screen used to search.

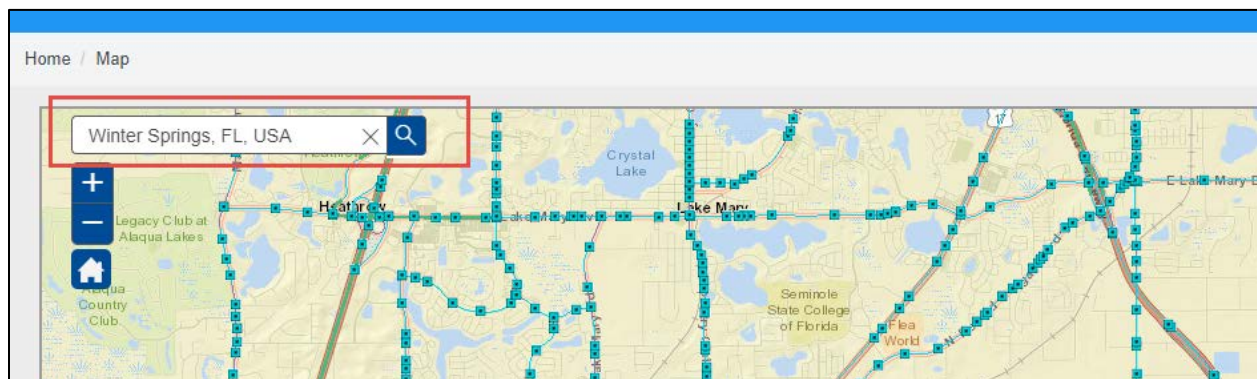


Figure 21 - Search Screen

3.4.3.2.3 Zoom In/Out, Pan

Users will have the ability to Zoom In/Out on the map page by selecting “+” and “-” icons. Also, users will have the ability to pan the map to a specific location. Figure 22 - Zoom features highlights the zoom features.

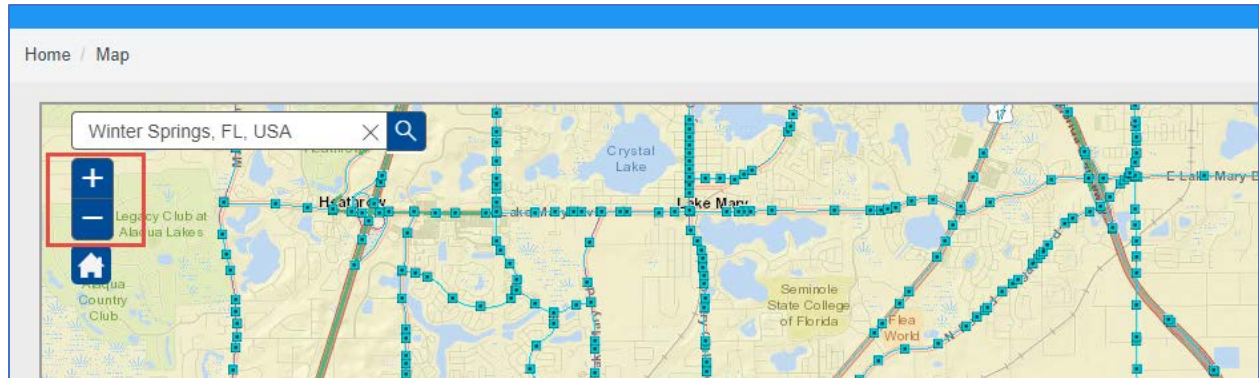


Figure 22 - Zoom features

3.4.3.2.4 Home

Users will have the ability to go back to the default extent when the Home button is selected. Figure 23 - Home button highlights the icon used to return to the home screen.

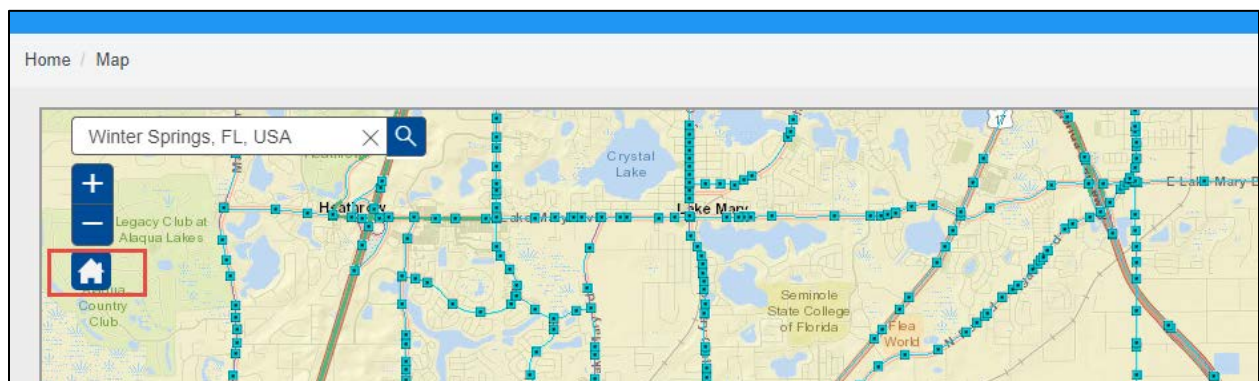


Figure 23 - Home button

3.4.3.2.5 Information Window

Details of a feature will be shown when the user clicks on a feature. Figure 24 - Feature Details illustrates the system showing details when a feature is selected.

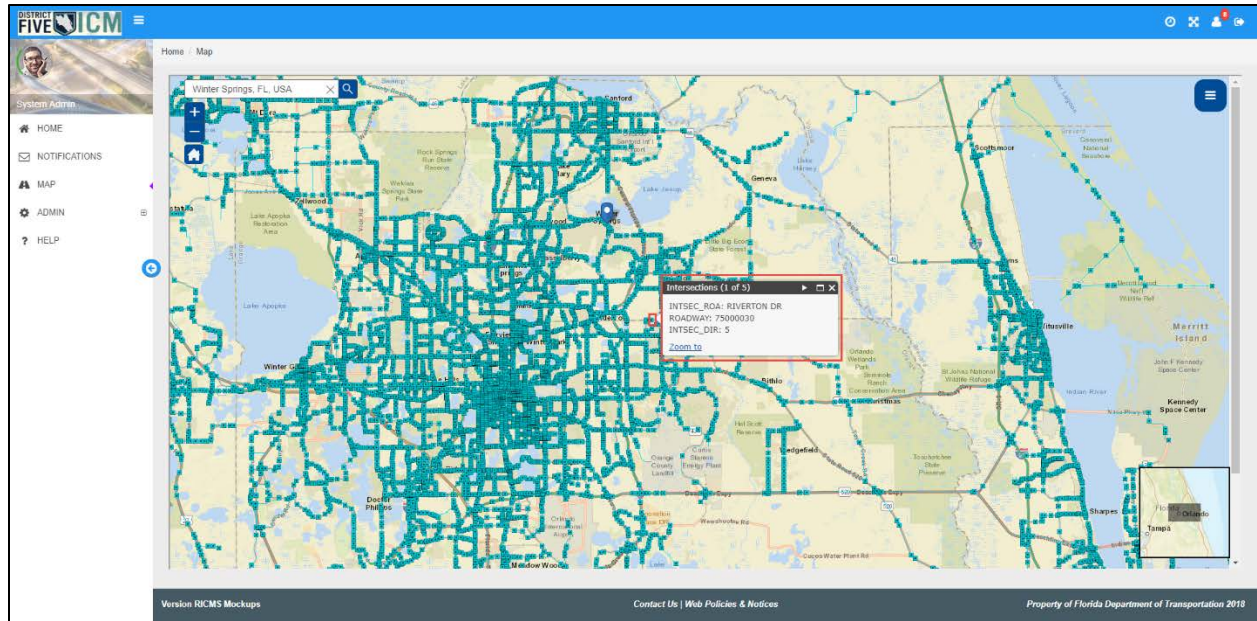


Figure 24 - Feature Details

3.4.3.2.6 Layers (Table of Contents)

3.4.3.2.6.1 Static Data

Users will be able to select static data layers. Based on the selected layers, the features will be displayed on the map. Below is a list of the layers that will be available for users to select.

- ~~Roadway Characteristics Inventory (RCI) data~~
 - ~~Intersections~~
 - ~~Number of Lanes~~
- Schools
- Emergency Responder Locations
- Transit

Figure 25 - Layer Selection Screen illustrates the selection of layers.

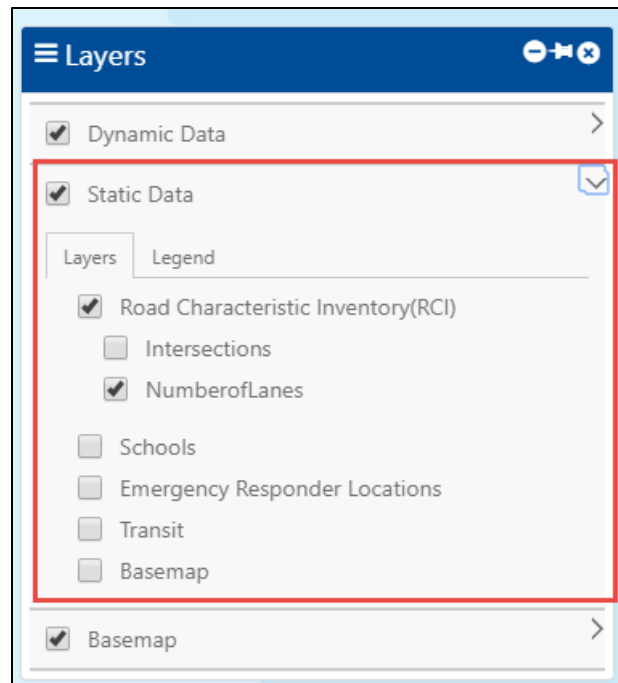


Figure 25 - Layer Selection Screen

3.4.3.2.6.2 Dynamic Data

User will be able to select the dynamic data layer. When the data layer is selected the map will display the appropriate data from GeoEvent Service. Figure 26 - Traffic Condition Layer Selection illustrates the selection of the Traffic Conditions layer.

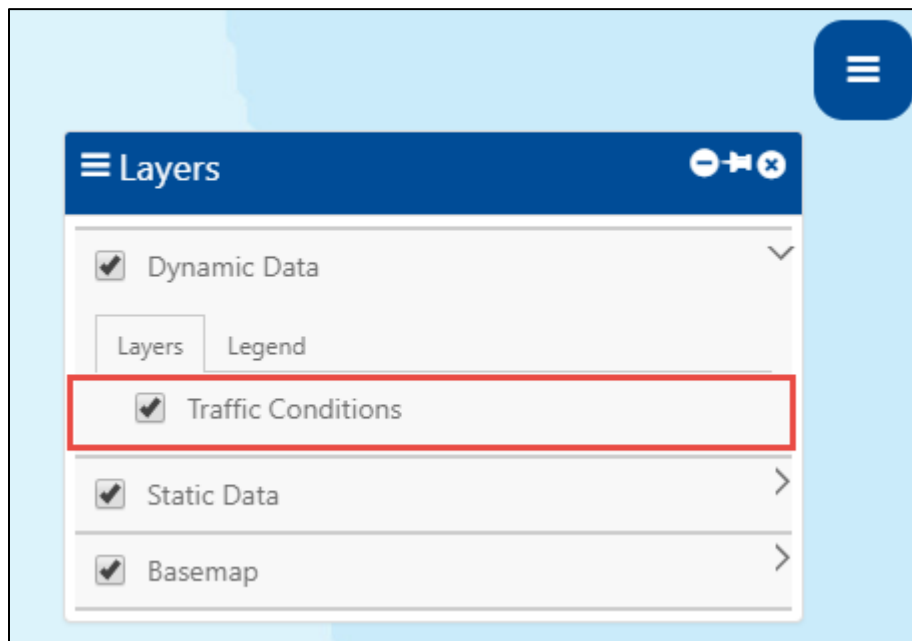


Figure 26 - Traffic Condition Layer Selection

3.4.3.2.6.3 Basemap

Users will be able to choose the basemaps from the basemap list as shown in Figure 27 - Basemap Selection.

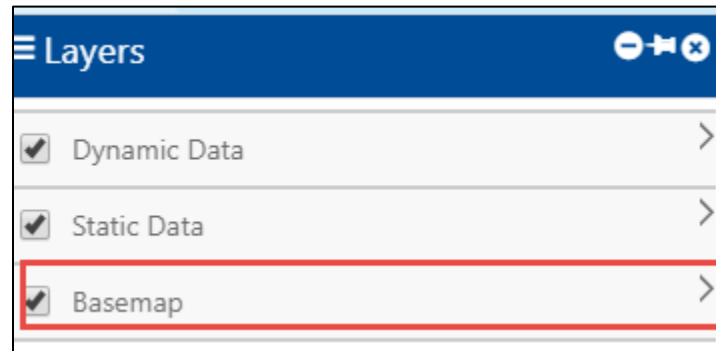


Figure 27 - Basemap Selection

3.4.3.2.6.4 Legend

Users will be able to view the legend for a selected layer, as shown in Figure 28 - Legend Selection.

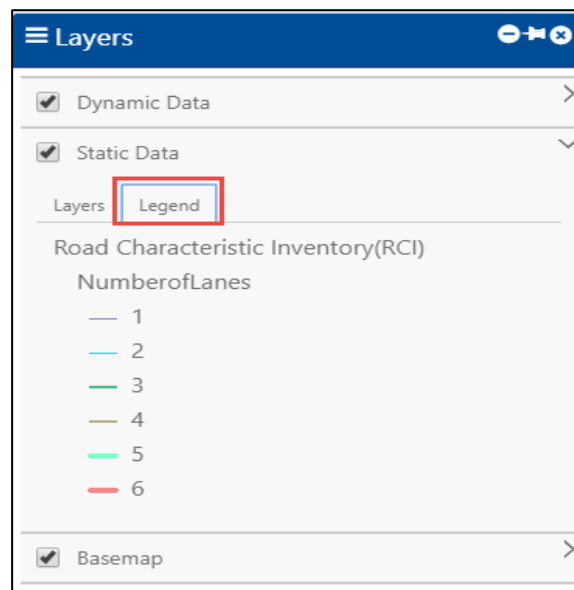


Figure 28 - Legend Selection

3.4.3.2.6.5 Navigation

Figure 29 - Navigation shows the navigation between screens.

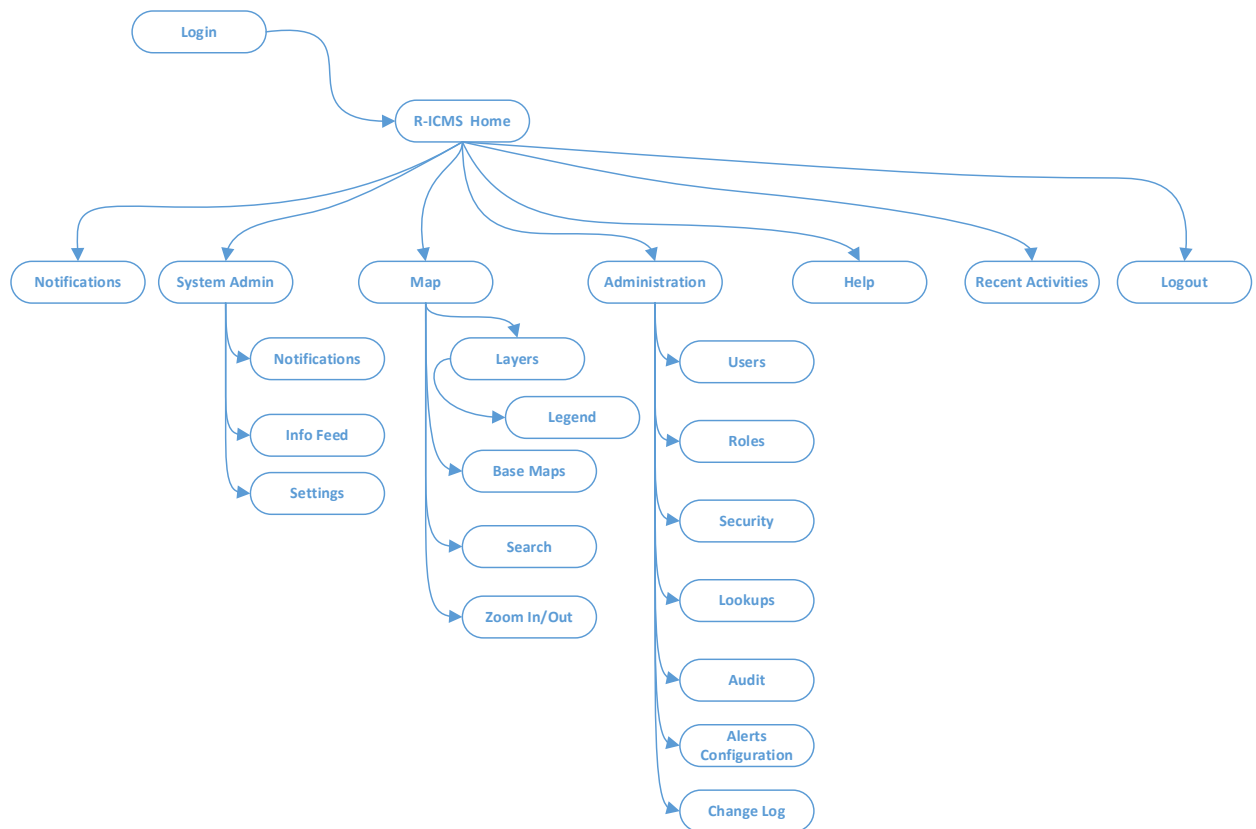


Figure 29 - Navigation

3.4.3.3 Admin (ADM-UI)

The Admin component of the User Interface will be the frontend to the Admin Business Service. It will provide the necessary functionalities to manage users, roles, agencies, and devices.

3.4.3.4 Notifications (NOT-UI)

The Notification component (also known as the information feed) of the UI will be responsible for display of user notifications. Notifications are not actionable and are purely informational in nature. A user will be able to open and close their notification window.

Figure 30 - Navigation

3.4.3.5 Alerts (ALT-UI)

The Alert component of the UI will handle system alerts coming from the Alerts Business Service in the server and display the information for user review. This information is typically actionable and will contain the necessary information needed for the user to make informed decisions.

Alerts will be displayed prominently to get the user's attention and indicate that some action needs to be performed.

3.4.3.6 Reporting (RP-UI)

The Reporting component of the UI will allow for users to run defined template reports on the system. It will interface with the RPT-BS to access data in the data stores and return the data to the UI. The initial version of the R-ICMS will include six of the following reports identified in the contract:

- Transportation System Performance Report
- DFE System Usage Report (including size of data transferred and data sets requested)
- Data Warehouse Report (including availability, errors, and available storage capacity by data store)
- Data Sources Status Report (including first available date-time, latest data date-time, and date-times of missing data)
- Prediction Quality Summary Report
- Corridor Recommendation Summary Report (including timing plan details, MOEs, Metrics, individual intersections, and agencies, by time-of-day pattern from AM, PM, off-peak or 24 hours)
- Event List Report

Users will be able to add new defined template reports to the system.

3.4.3.7 Event Management (EM-UI)

The Event Management component of the UI will allow users to manage events in the R-ICMS system. It will contain a summary list of events and allow for users to drill down to see event details. It will allow for the creation, modification, and closing of R-ICMS events.

3.4.3.8 Response Plan Management (RPM-UI)

The Response Plan Management component will be responsible for the creation, deletion, and modification of response plans and response plan elements. It will interface with the RP-BS to access data in the data stores and return the data to the user interface.

3.4.3.9 Signal Optimization Timing (SOT-UI)

The SOT-UI will be accessible from the main UI interface. It will be responsible for providing the screens and capabilities necessary to optimize traffic signals in the system. It will provide an interface for allowing authorized users to digitally sign traffic signal plans. It will provide the ability to download traffic signal plans for consumption into third party traffic signal software.

3.4.4 Common Core

Common core components, consisting of libraries, will be available for use in other R-ICMS components developed for the R-ICMS system. They include functionalities that services need in order to function properly in the defined R-ICMS environment. Shared libraries will ensure that services will utilize a common method of interacting.

3.4.4.1 Authorization (Auth-CC)

External data requests will include a signed token specifying the relevant user data as well as the individual user permissions. The Auth-CC component will be responsible for communicating with an in-memory cache to retrieve the signing key for the specified token. The Auth-CC will then validate the token with the retrieved signing key to retrieve the permissions assigned to the user that provided the request. If the user has permissions to the relevant permission for the service, the service will approve the request and provide data to the user.

3.4.4.2 Logging (Log-CC)

Services will be responsible for providing information for the system logs. The Log-CC component will allow each service to easily use a common interface to provide log data. Log levels will be defined in the system and will include Six levels of granularity:

- Fatal
- Error
- Warning
- Information
- Debug
- Verbose

Log levels will be able to be changed dynamically for specific components during system operations without restarting services. This will ensure that the system can be monitored at different levels at different times depending on user need.

Design Patterns - A singleton allows access to a single created instance - that instance (or rather, a reference to that instance) can be passed as a parameter to other methods and treated as a normal object. A static class allows only static methods. The Logging library will access a singleton logging service to log items. Figure 31 – Log Design Pattern illustrates the singleton pattern used in logging.

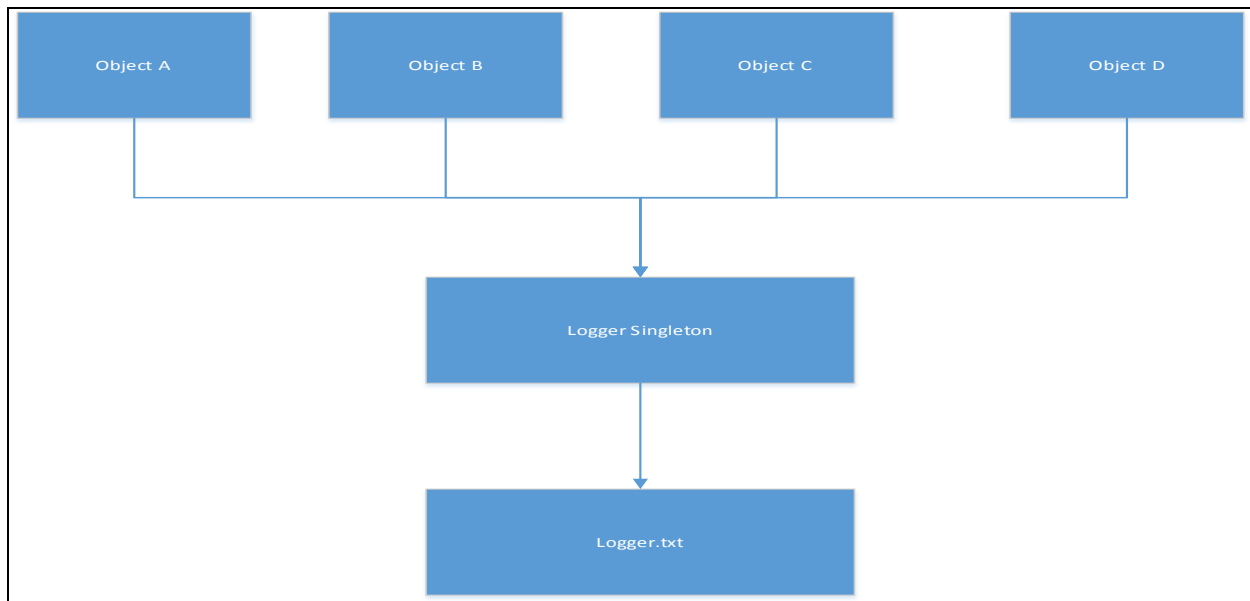


Figure 31 – Log Design Pattern

3.4.4.3 Monitoring (Mon-CC)

Services will be monitored to ensure the system is running properly. The Mon-CC component will provide the functionality to send current status to the monitoring service. This capability will allow the system to continuously keep track of the status of the services in use in order to have the earliest warning of failures, defects, or problems. When services are not running optimally, the system will utilize the Alerting Common Core component to notify appropriate system users that a problem may have occurred.

3.4.4.4 Alerting (AL-CC)

Some services will need to provide alerts relevant to users of the R-ICMS system. The AL-CC will provide common functionality that will allow a service to create and send alerts which will interface with an alerting service. The alerting service will then notify users appropriately and provide the alert information so that users can make informed decisions relevant to the content of the alert.

3.4.4.5 Gateway (GW-CC)

As shown in the high-level diagram, a Gateway (GW-CC) will sit between user systems and the services they can access. The Gateway will provide a single point of contact for user systems and provide the load leveling for the external user load on the data and business services. The Gateway will distribute messages from users to the appropriate service and messages from the services to the appropriate user.

3.5 System Interfaces

The R-ICMS will interface with multiple different systems to consume/aggregate data, provide response plan actions, and provide data to external systems. The following subsections describe the different interfaces initially proposed.

3.5.1 Data Source Drivers

As described in section 3.4.1.1, drivers will be used to interface with external data sources in order to ingest data into the DFE. Drivers will conform to the API of the system providing the data. Driver interfaces will include connecting to streaming data sources as well as requesting data on regular intervals as appropriate. Additionally, if the system providing the data supports it, the interface will allow for request of historical data.

3.5.2 SunGuide Driver

As described in section 3.4.1.1.3, a SunGuide driver will be built to support communication with the SunGuide system. This driver will support a single socket connection to the SunGuide Databus process and will communicate via XML as defined in the SunGuide interface Control Document. All data sent to and received from SunGuide will conform to the SunGuide XML schemas.

3.5.3 Third Party Access

The R-ICMS will support approved third-party users accessing the system to retrieve R-ICMS data. Data will be accessible to third party users via the Data Services defined in section 3.4.1.4. The data will be provided via a web services interface and will support data retrieval through a subscription-based interface, and a data request interface. Third party systems/users will be responsible for conforming to the R-ICMS APIs to receive data.

3.5.4 Signal Timing Files

The R-ICMS will support the creation and retrieval of signal timing plan files for eventual import into signal timing plan software and traffic signal controllers. Files will be retrievable via the R-ICMS SOT UI. Supported formats will be determined in a later critical design phase.

3.5.5 Bulk Load

The R-ICMS will support the ingestion of Response Plans (RP), including Response Plan Elements (RPE), from files via the R-ICMS RP interface. The file formats for the RP files will be determined in a later critical design phase.

3.5.6 Modeling Engine

The R-ICMS will send current or historical traffic condition data, RPs, and other data to the Modeling Engine (ME). It will retrieve the results of modeling runs from the ME. The interface between the ME and the R-ICMS will be defined by the R-ICMS contractor at the appropriate critical design phase.

3.5.7 Signal Timing Optimization Engine

The R-ICMS will provide the Signal Timing Optimization Engine (currently expected to be HCS7) historical traffic conditions, and traffic network information (both corridor and intersection), as well as modeling constraints to the Signal Timing Optimization Engine and will retrieve optimized signal timing plans and signal timing plan sets from the Signal Timing Optimization Engine.

3.6 System Resources

To support the R-ICMS in its proposed configuration, the following hardware needs have been identified. All servers will be hosted in the FDOT D5 hosting environment and will follow FDOT D5 standards for hosted systems. As specified in the contract, these standards include:

- Agency for State Technology (AST) Chapter 74-2 F.A.C., Information Technology Security, also known as the Florida Cybersecurity Standards (FCS)
- FCS, 74-2 F.A.C. Section 74-2.002 (4)

3.6.1 Hardware Configuration

Components within the business and data layers are built to run in a container. The container dynamically occupies a portion of resources on a server allocated to that layer. The dynamic allocation is administered using the Kubernetes orchestration tool as described in Section 3.8.2

Network infrastructure is provided by the Department.

Table 19 - Hardware Configuration describes the servers and the layers they are assigned to.

Table 19 - Hardware Configuration

Server details	# machines	Virtual / Physical	Configuration	Layer
SQL DB Server – 1 GIS DB Server - 1	2	V	128 GB / 8 Core / 4 TB RAID 10	Data Store
ArcGIS Server – 1 Geo Event Server -1	2	V	128 GB / 16 Core / 1 TB RAID 10	Data Store
GIS App Server – 1 Business Services Server – 1 Gateway Server - 1	3	V	64 GB / 8 Core / 1 TB RAID 10	Data/Business Services
Web Servers – 2 Application Servers - 2	4	V	128 GB / 16 Core / 4 TB RAID 10	Data/Business Services

Server details	# machines	Virtual / Physical	Configuration	Layer
Kafka Cluster	3	P	2 x 8 Core , 128GB RAM w/ 2 x 1TB RAID 1, 4 x 2TB RAID 1 individual	Pipelines
Cloudera Manager	3	P	2 x 8 Core , 128GB RAM w/ 2 x 1TB RAID 1, 2 x 1TB RAID 1, 2 x 1TB individual	Data Store
Cloudera Data Nodes	9	P	2 x 8 Core , 128GB RAM w/ 2 x 1TB RAID 1, 6 x 2TB Individual	Data Store
Edge nodes for client sessions / users	3	P	2 x 8 Core , 128GB RAM w/ 2 x 1TB RAID 1, 1 x 1TB Individual	User Interface
Edge nodes for ETL	3	P	2 x 8 Core , 256GB RAM w/ 2 x 1TB RAID 1, 6 x 2TB Individual	Data Store
Elastic search Master nodes	2	P	2 x 8 Core , 128GB RAM w/ 2 x 1TB RAID 1, 1 x 1TB individual	Data Store
Elastic Search Workers	3	P	2 x 8 Core , 128GB RAM w/ 2 x 1TB RAID 1, 2 x 8TB RAID 1 individual	Data Store
NoSQL	3	P	2 x 8 Core , 256GB RAM w/ 2 x 1TB RAID 1, 6 x 2TB individual	Data Store

3.6.2 Physical Deployment: Servers and Network

This diagram shows the deployment of physical servers hosting R-ICMS services within private virtual networks, including routing through an internet firewall for any external server access.

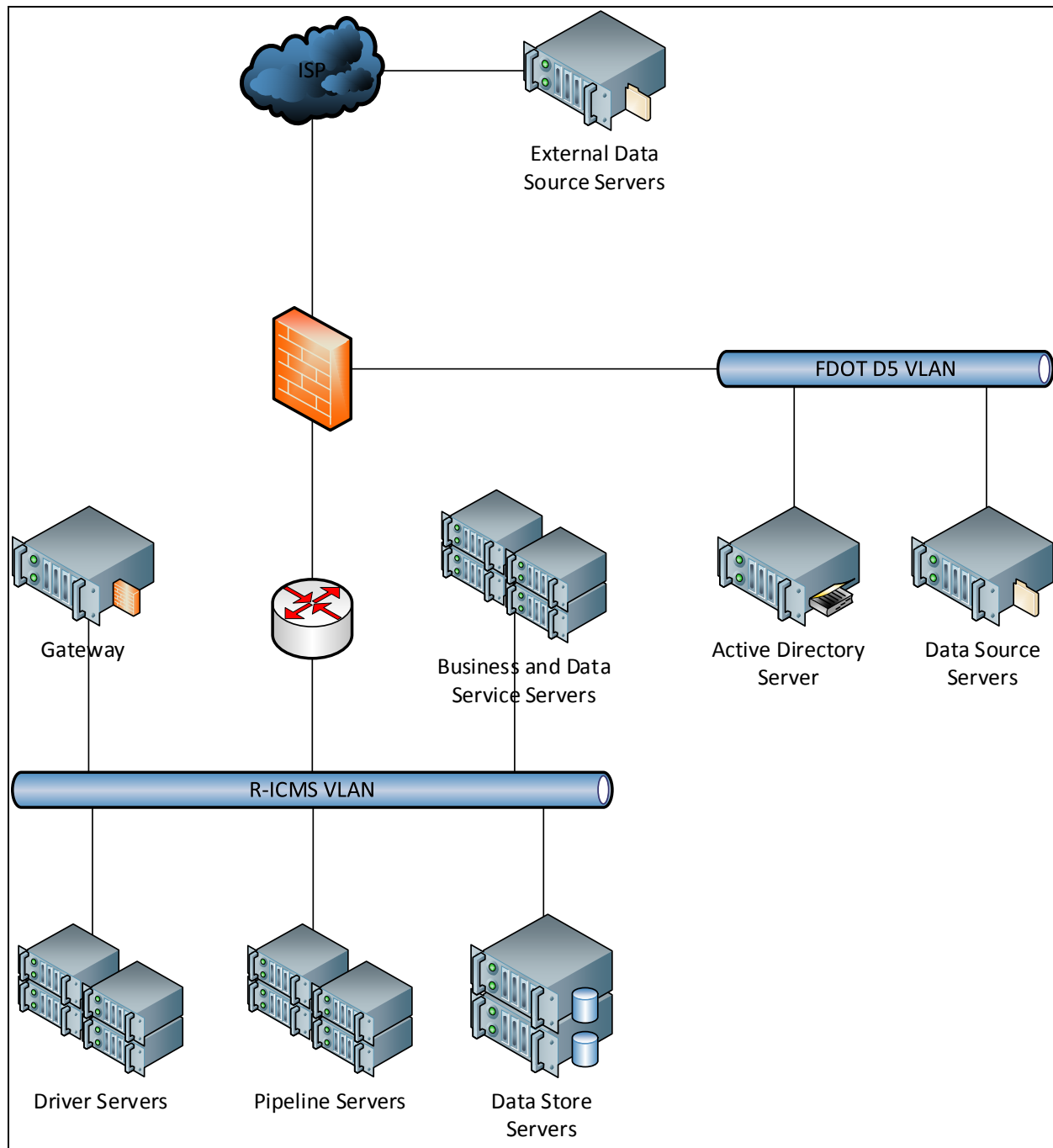


Figure 32 – Physical Deployment: Servers and Network Diagram

3.7 Concept of Execution

The section contains the diagrams and descriptions showing the dynamic relationships among the software modules including how the modules will interact during system operation. The sequence diagrams use standard Unified Modeling Language (UML), as constrained by using Visio. In the sequence diagrams, a yellow arrow is used to imply that the content of a message is stored, as well being transmitted in the message. The initial sequence diagrams provide a more

detailed understanding of the authentication and authorization steps in the interactions of system components. The lack of those detailed steps in the remaining diagrams is not intended to imply that they do not occur; rather, the lack of detail is only intended to make the diagrams cleaner and therefore clearer for the specific interactions being focused on. Likewise, the logging sequence diagrams show more details for the logging of data than other diagrams. Similarly, the intent is not that the logging will not occur during other interactions, but that the other sequence diagrams can be clearer by not spelling out all of the logging in those interactions. The sequence diagrams shown in these graphics primarily address the “happy path” interactions. “Unhappy paths” will typically take the form of error messages. Additional “unhappy paths” implementation will be addressed in the appropriate detailed design section.

Other diagrams are more informal in nature and intended primarily to clarify the information presented.

A list of components used in the diagrams is included in Section 3.4 along with descriptions of the components. In some cases, generic components are referenced when any component of that type can be involved in the interaction.

The following color scheme will be used to depict the layer within the high-level architecture that each represented component resides.

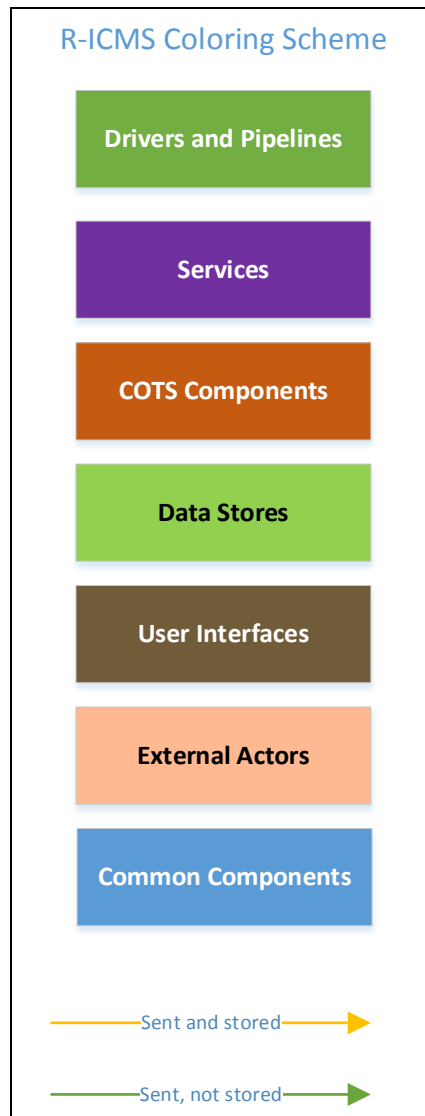


Figure 33 - R-ICMS Coloring Scheme Diagram

3.7.1 Security

Security is essential to the functionality of the R-ICMS. This section contains the relevant diagrams concerning the security processes needed to ensure data is accessed in a secure manner. The following diagrams are intended to show how the security processes will work throughout the system such that following diagrams will not have to specifically reference security features.

Exchanges of credentials/tokens and other data between users and the gateway are performed over HTTPS to prevent sniffing.

3.7.1.1 Login

Figure 34 - Login Sequence Diagram shows the system login process. User login requests are authenticated via Active Directory/ Lightweight Directory Access Protocol (LDAP), and active sessions are represented internally as signed tokens containing user permissions.

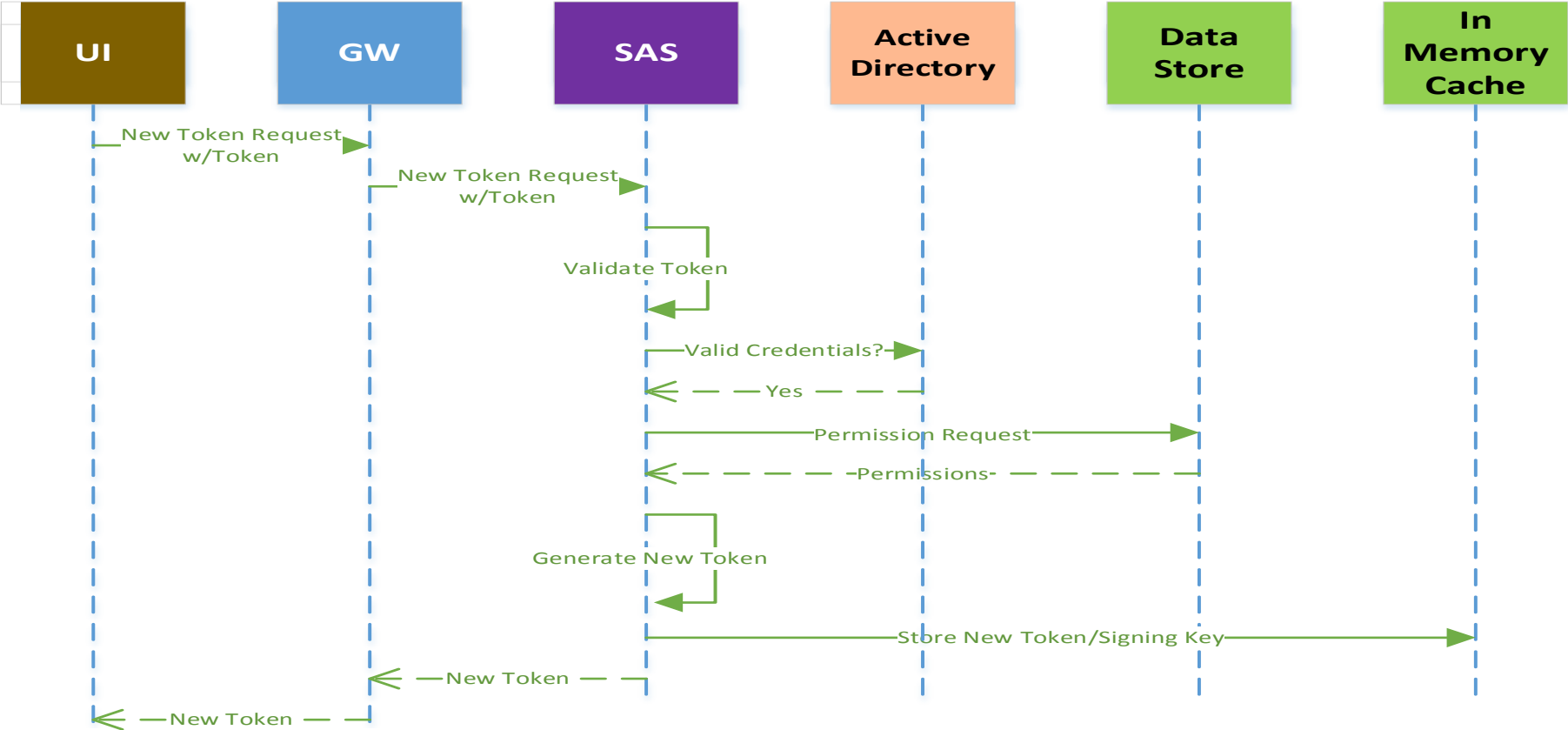


Figure 34 - Login Sequence Diagram

3.7.1.2 User Data Request

Figure 35 - User Data Request Sequence Diagram shows the interaction for any data request or setting up a streaming data subscription using a token. This includes verification of the token and user permissions for that request. System services use the authorization library, a common core component, to validate security tokens using the associated signing key from an in-memory cache. The authorization library is also used to validate that permissions required by a system request match the user permissions within the token.

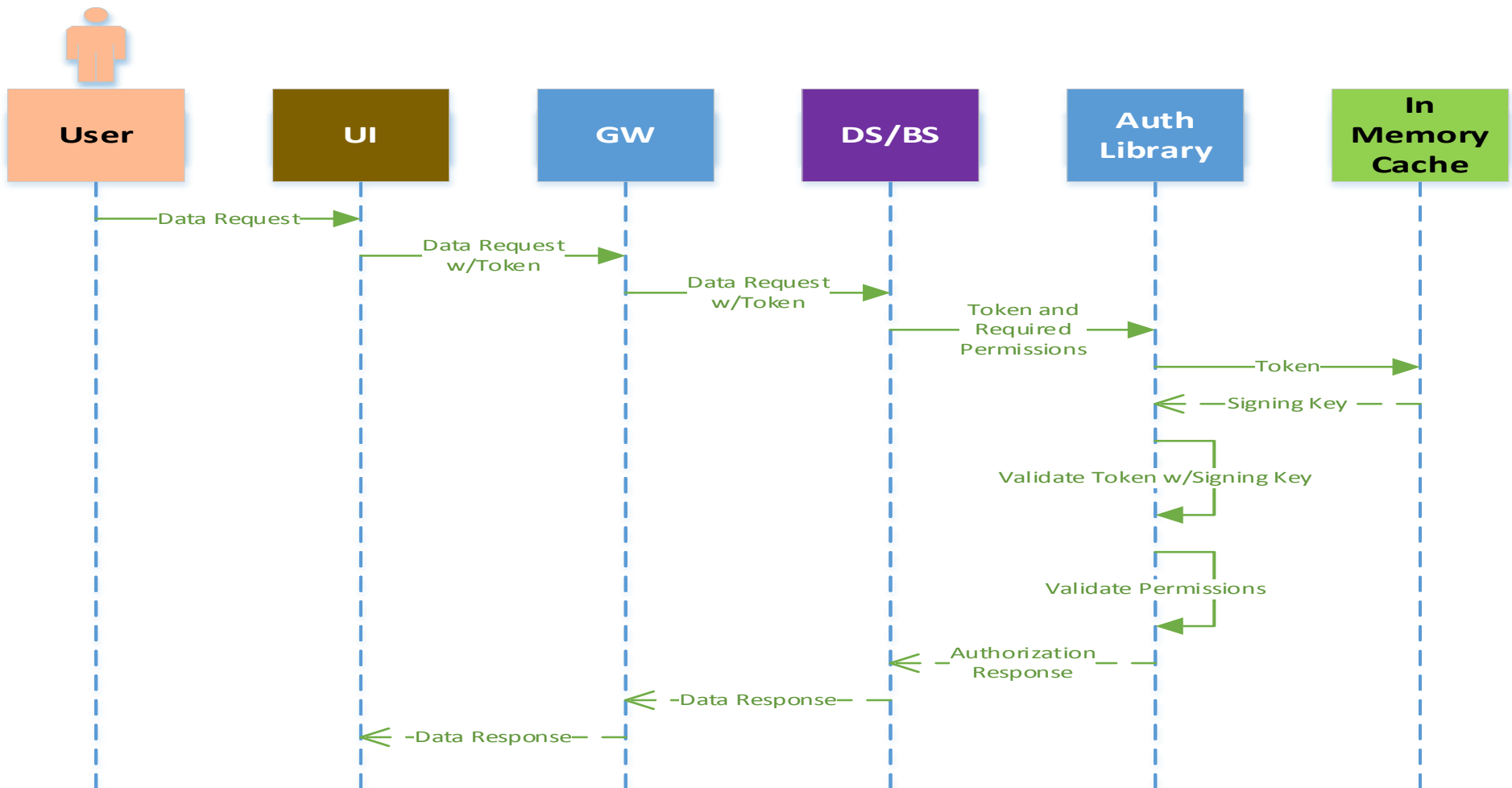


Figure 35 - User Data Request Sequence Diagram

3.7.1.3 User Authorization Update

Figure 36 - User Authorization Update Sequence Diagram shows how an active user session may be extended beyond the token expiration time by renewing a token. The user interface is responsible for automatically renewing a token before it expires without any user interaction.

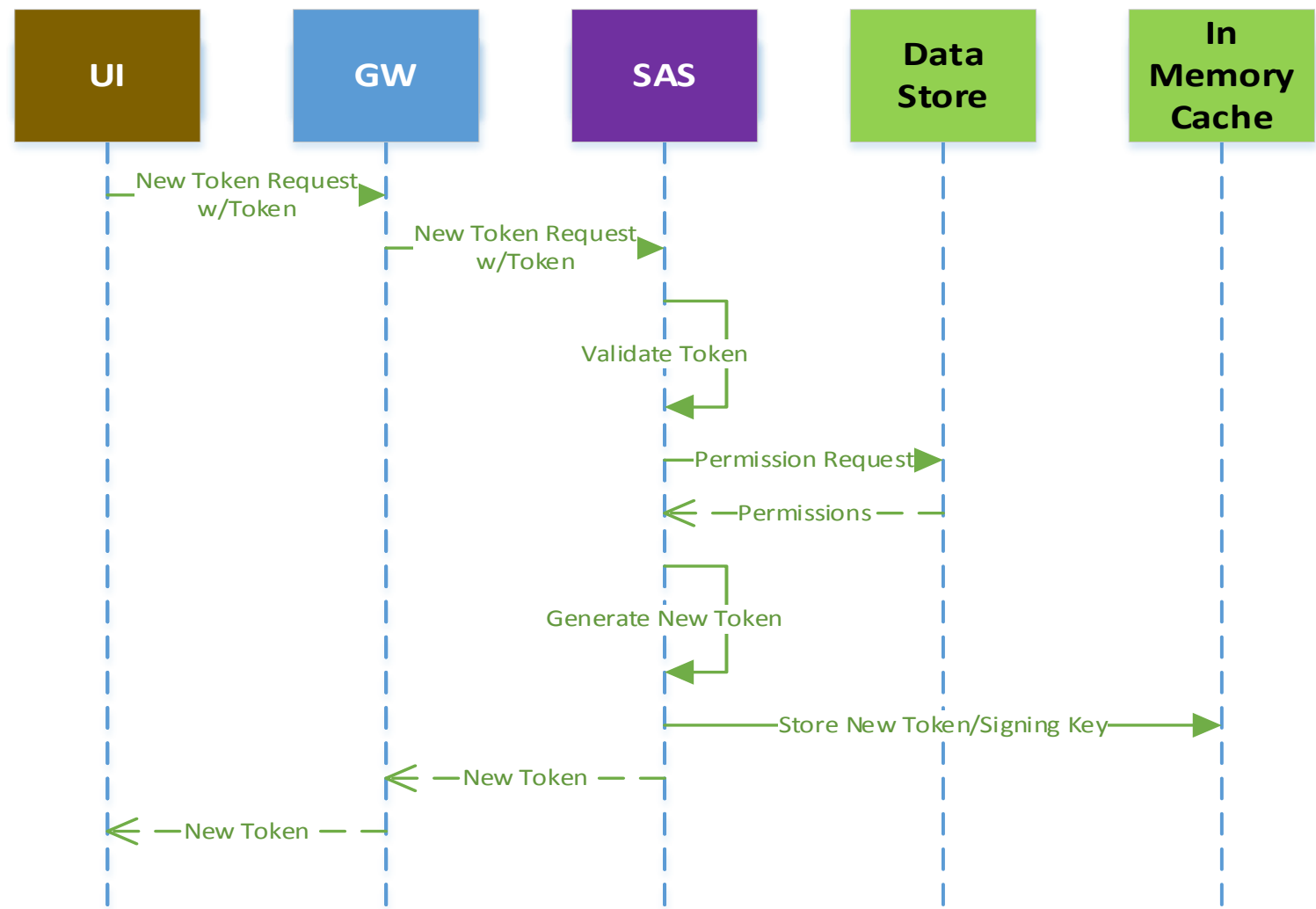


Figure 36 - User Authorization Update Sequence Diagram

3.7.1.4 User Personalization

Figure 37 - User Personalization Sequence Diagram shows how user may personalize view and filter settings, which are persisted across login sessions via the User Personalization business service.

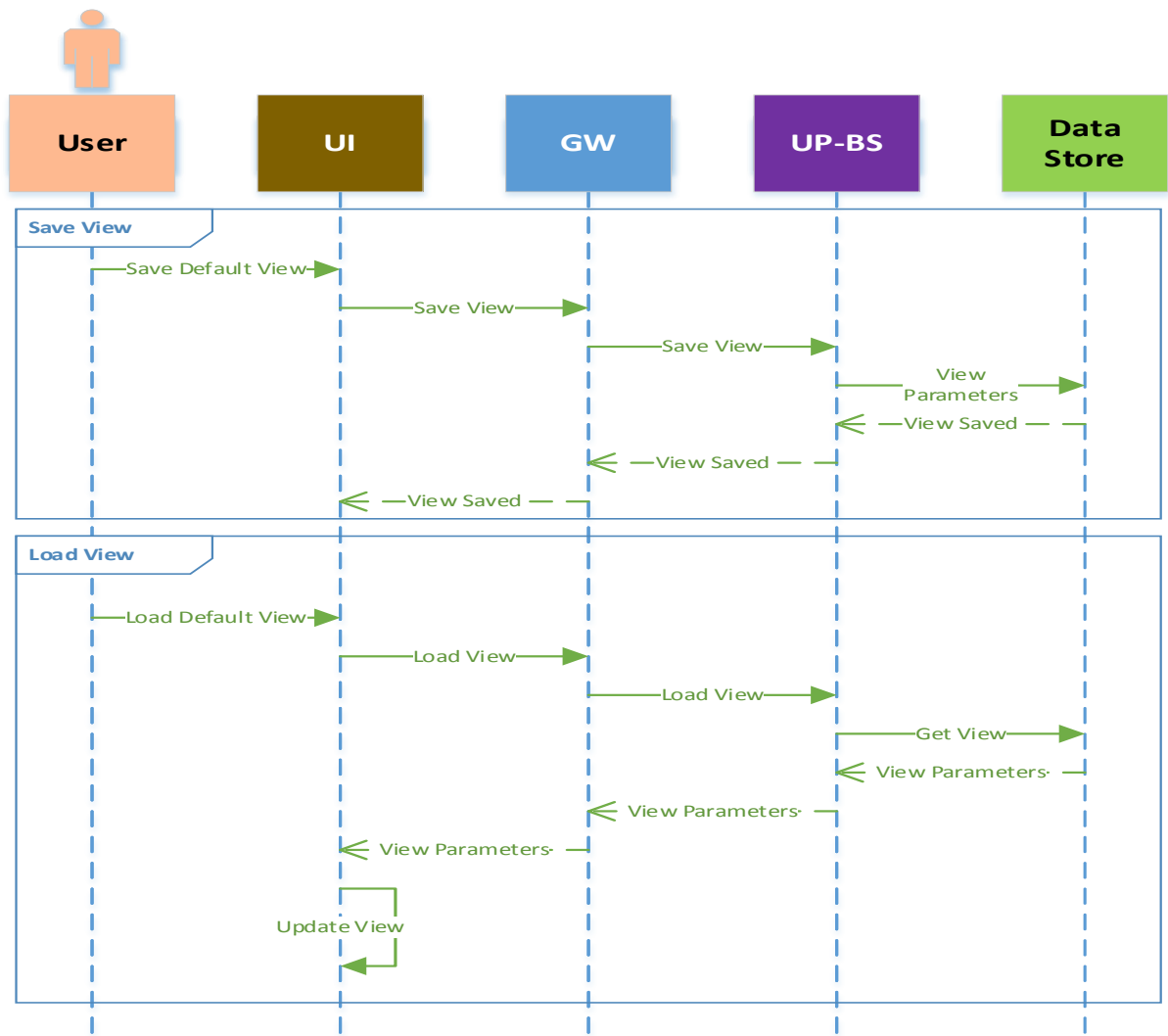


Figure 37 - User Personalization Sequence Diagram

3.7.1.5 GIS Use Cases

Figure 38 – GIS Use Cases shows the use cases supported by the GIS. The tables following the use case diagram, Table 20 - Use Case View Map Details, Table 21 - View GIS Layers Use Case Details, Table 22 - View Details Use Case Details, and Table 24 - Receive Dynamic Data Use Case Details, provide details for the illustrated use cases.

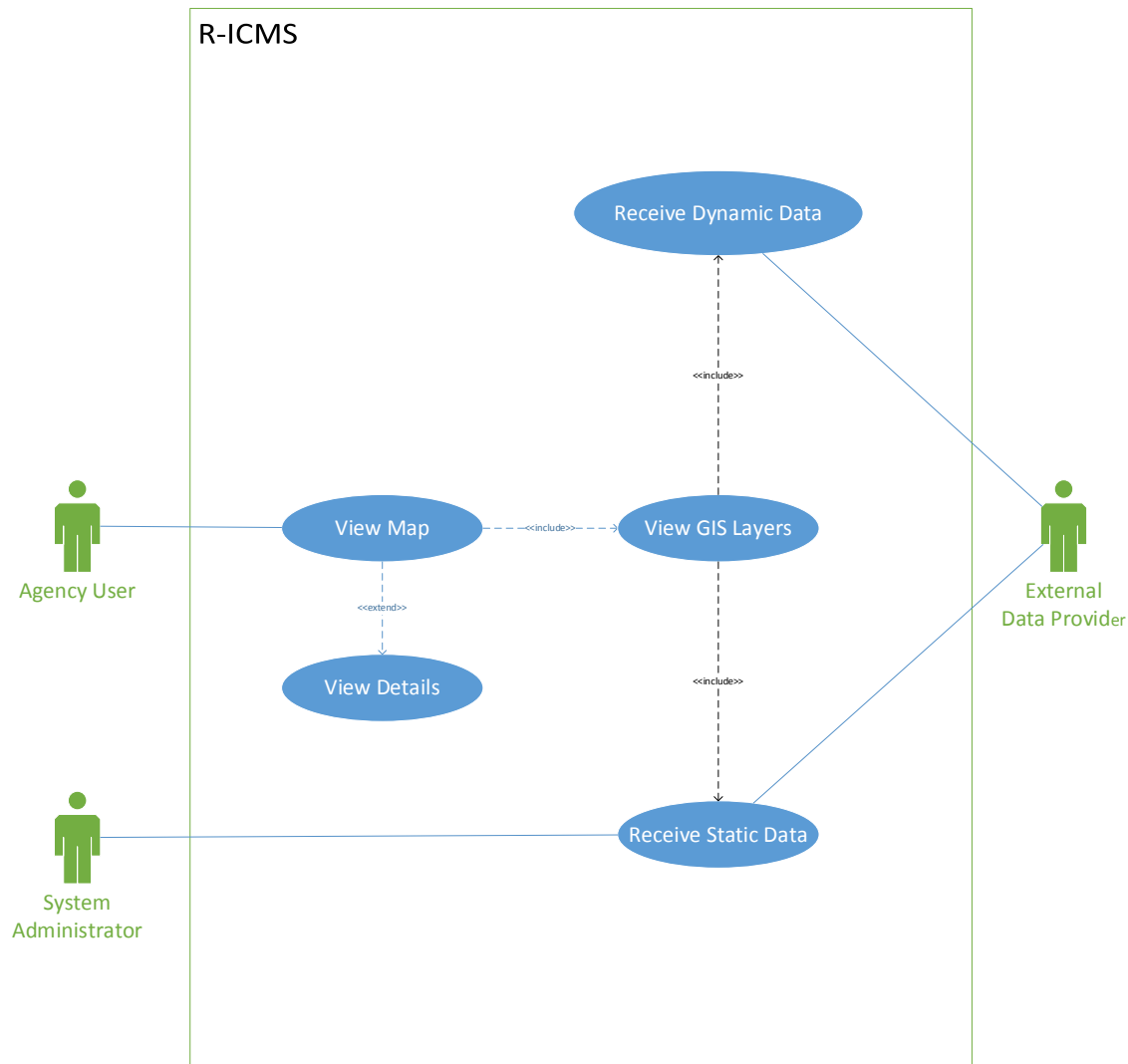


Figure 38 – GIS Use Cases

Table 20 - Use Case View Map Details

Use Case Information	
Use Case ID	UC.GIS.1
Use Case Name	View Map

Summary	User is able to view the map, map navigation and legend
Actors	
Primary	System User
Secondary	None
Pre-conditions	The user must be logged into the system and have privileges to access the map feature
Main Scenario	
Trigger	User Interaction
Typical Events	1. The use case begins when the user selects the function to view the map 2. The system displays the Map 3. Use case ends
Includes	View Map Details, View GIS Layers
Input	None
Output	Map View
Alternate Scenarios	
Refresh the map	1. During the basic flow, [system displays the map], the user may select to refresh the page 2. The system will refresh the contents displayed on the map and the use case continues in the basic flow at [system displays the map]
Zoom In and Out	1. During the basic flow, [system displays the map], the user may select to change the zoom (in/out) 2. The system will adjust the view to the user selected zoom level and the use case continues in the basic flow at [system displays the map]
Pan	1. During the basic flow, [system displays the map], the user may select to pan the map to view a different area by using the click and drag or map overview methods 2. The system will adjust the view to the user selected pan location and the use case continues in the basic flow at [system displays the map]
Home View	1. During the basic flow, [system displays the map], the user may select to return to the default view 2. The system will adjust the view to the system default zoom and location view and the use case continues in the basic flow at [system displays the map]
Search	1. During the basic flow, [system displays the map], the user may search using the parameters configured for the respective layers/features 2. The system will perform the search and display the results on the map and the use case continues in the basic flow at [system displays the map]
Extends / Uses	View GIS Layers
Post-conditions	Map is displayed
Frequency	User driven
Exceptions	User cancels the use case. The use case ends
Related Use Cases	None

Table 21 - View GIS Layers Use Case Details

Use Case Information	
Use Case ID	UC.GIS.2
Use Case Name	View GIS Layers
Summary	User selects/deselects items in the map layers to customize view

Actors	
Primary	System User
Secondary	None
Pre-conditions	The user must be logged into the system and have privileges to use the map feature
Main Scenario	
Trigger	User Interaction
Typical Events	1. The use case begins when the user selects the function to view map layers 2. The system displays the available layers 3. User selects/deselects layers 4. Map displays user selected layers 5. The use case ends
Includes	None
Input	Selection of desired layers
Output	Map is displayed with user selected layers
Alternate Scenarios	
Variants	View Legend 1. During the basic flow, [system displays the available layers], the user may select to view the map legend 2. The system will display the legend and the use case continues in the basic flow at [system displays the available layers] View Sublayers 1. During the basic flow, [system displays the available layers], the user may select to view the map sublayers 2. The system will display the sublayers that exist for each map layer and the use case continues in the basic flow at [system displays the available layers]
Extends / Uses	Events, Weather, Traffic, Device, Parking Data
Post-conditions	Map is displayed with user selected layers
Frequency	User driven
Exceptions	User cancels the use case. The use case ends
Related Use Cases	None

Table 22 - View Details Use Case Details

Use Case Information	
Use Case ID	UC.GIS.3
Use Case Name	View Details
Summary	User selects item on map to view/edit
Actors	
Primary	System User
Secondary	None
Pre-conditions	1. The user must be logged into the system and have privileges to view the map 2. The user has selected a layer to display items on a map
Main Scenario	
Trigger	User Interaction
Typical Events	1. The use case begins when the user selects an item on the map 2. The system displays item details 3. Use case ends
Includes	None
Input	None

Output	The map displays the detail of the user selected item
Alternate Scenarios	
Variants	Close details
Extends / Uses	None
Post-conditions	None
Frequency	User driven
Exceptions	No details available. User cancels the use case. The use case ends
Related Use Cases	None

Table 23 - Receive Statis Data Use Case Details

Use Case Information	
Use Case ID	UC.GIS.4
Use Case Name	Receive Static Data
Summary	System receives static data to populate GIS layers
Actors	
Primary	System, System Administrator
Secondary	External Data Provider
Pre-conditions	Static data file is available in pre-defined location on the external source Schedule exists for static file updates Authentication credentials exist for external data source
Main Scenario	
Trigger	New static data file is identified OR A scheduled event is triggered
Typical Events	1. The use case begins when the system executes a script to connect to the external data source 2. The system credentials are authenticated and returns the file details 3. The system will log the file activity and verify that updates exist since the previous file update 4. The system will ingest the updated data 5. The system will update the data in the GIS server 6. The system will refresh the map data on the UI display within 3 seconds 7. The use case ends
Includes	None
Input	FTP file
Output	Updated GIS Map, Log
Alternate Scenarios	
Variants	Manual Update 1. The use case begins when the system administrator copies the file to the RICMS system 2. The system will log the file activity and identify duplicates or changes in the data and appends the data to the system 3. The system administrator publishes the updates through GIS tools 4. The system will refresh the map on the UI display within 3 seconds 5. The use case ends
Extends / Uses	None
Post-conditions	Updated GIS Map, Log history
Frequency	TBD
Exceptions	User cancels the use case. The use case ends
	No update available
Related Use Cases	None

Table 24 - Receive Dynamic Data Use Case Details

Use Case Information	
Use Case ID	UC.GIS.5
Use Case Name	Receive Dynamic Data
Summary	System receives dynamic data to populate GIS layers
Actors	
Primary	System
Secondary	External Data Provider
Pre-conditions	Dynamic data file is available in pre-defined location on the external source Authentication credentials exist for external data source
Main Scenario	
Trigger	New dynamic data file is identified
Typical Events	1. The use case begins when the system executes a script to connect to the external data source 2. The system credentials are authenticated and returns the file details 3. The system will verify that updates exist since the previous file update and log history of activity 4. The system will ingest the updated data and refresh the map within 3 seconds for any GIS data 5. The use case ends
Includes	None
Input	FTP file
Output	Updated GIS Map, Log
Alternate Scenarios	
Variants	
Extends / Uses	None
Post-conditions	Updated GIS Map, Log history
Frequency	TBD
Exceptions	User cancels the use case. The use case ends
	No update available
Related Use Cases	None

3.7.2 Situational Awareness

Situational Awareness pertains to updates coming in from external sources that need to be shared with relevant data consumers including (but not limited to) the ME, business rules engine, and GeoEvent service, which filters and forwards info to agency users based on their map or other UI views. The following diagrams depict the flow of data when updates come in from ITSQA as well as how the map updates the views for pre-located (e.g. DMS) as well as dynamically-located (e.g. Events) data.

3.7.2.1 ITSIIQA Ingestion

Figure 39 - ITSIIQA Ingestion Sequence Diagram shows the asynchronous flow of updates through the ICMS system and to the user interface after an update subscription has been established. In this example, unbounded traffic data is ingested from an ITSIIQA pipeline to a traffic data service. Users subscribed to a traffic data service feed receive updates as data arrives.

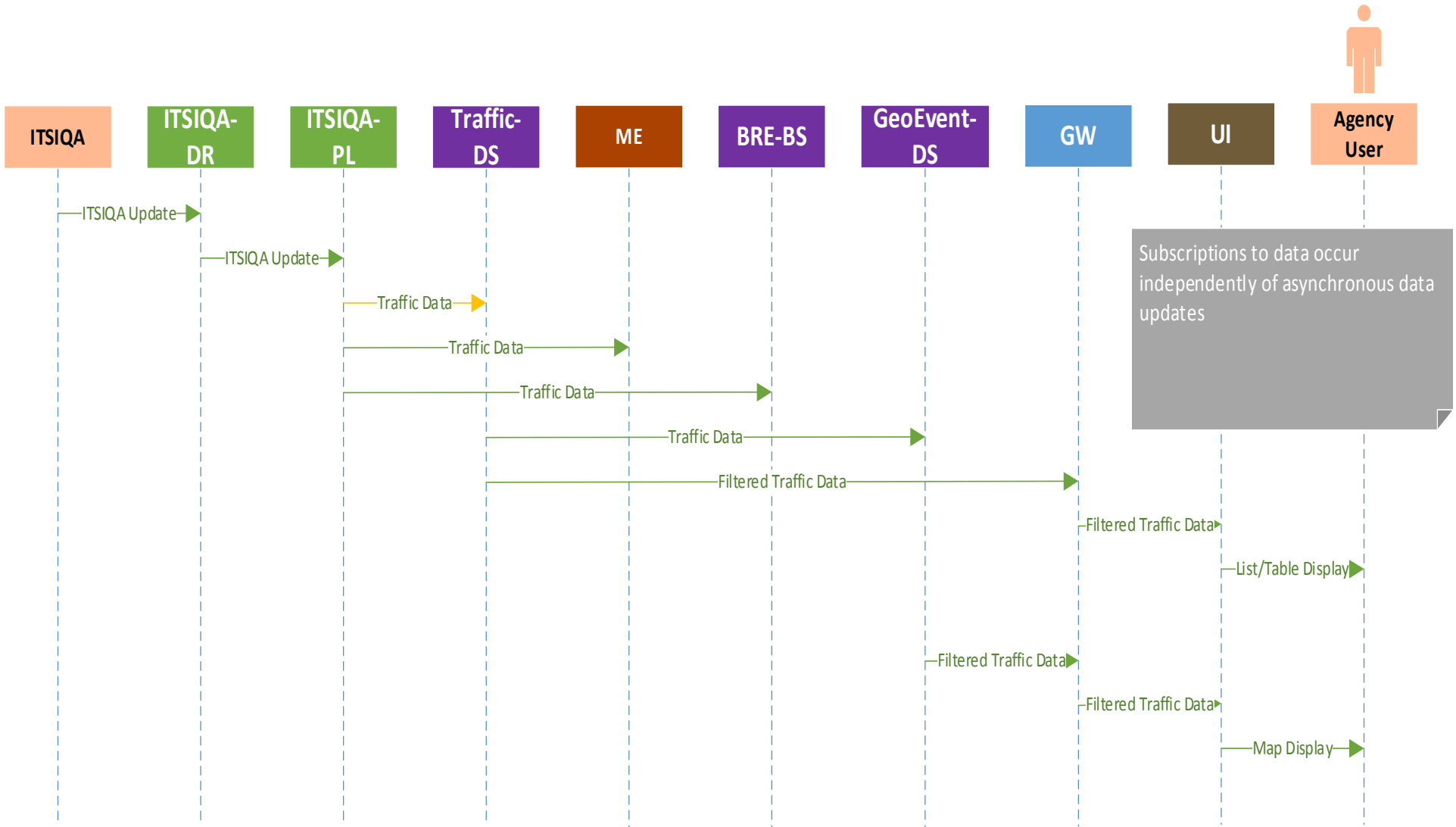


Figure 39 - ITSIIQA Ingestion Sequence Diagram

3.7.2.2 Map Update for Pre-Located Data

Figure 40 - Map Update for Pre-Located Data Sequence Diagram depicts the initiation of a subscription for pre-located feature updates. This includes features that have a static position that have dynamic status, such as DMS. A single subscription request is made to the GeoEvent Data Service and it will push any updates it receives from the pipeline that match the current filters, such as geographical extent, to the user interface, which will then update the feature in its map.

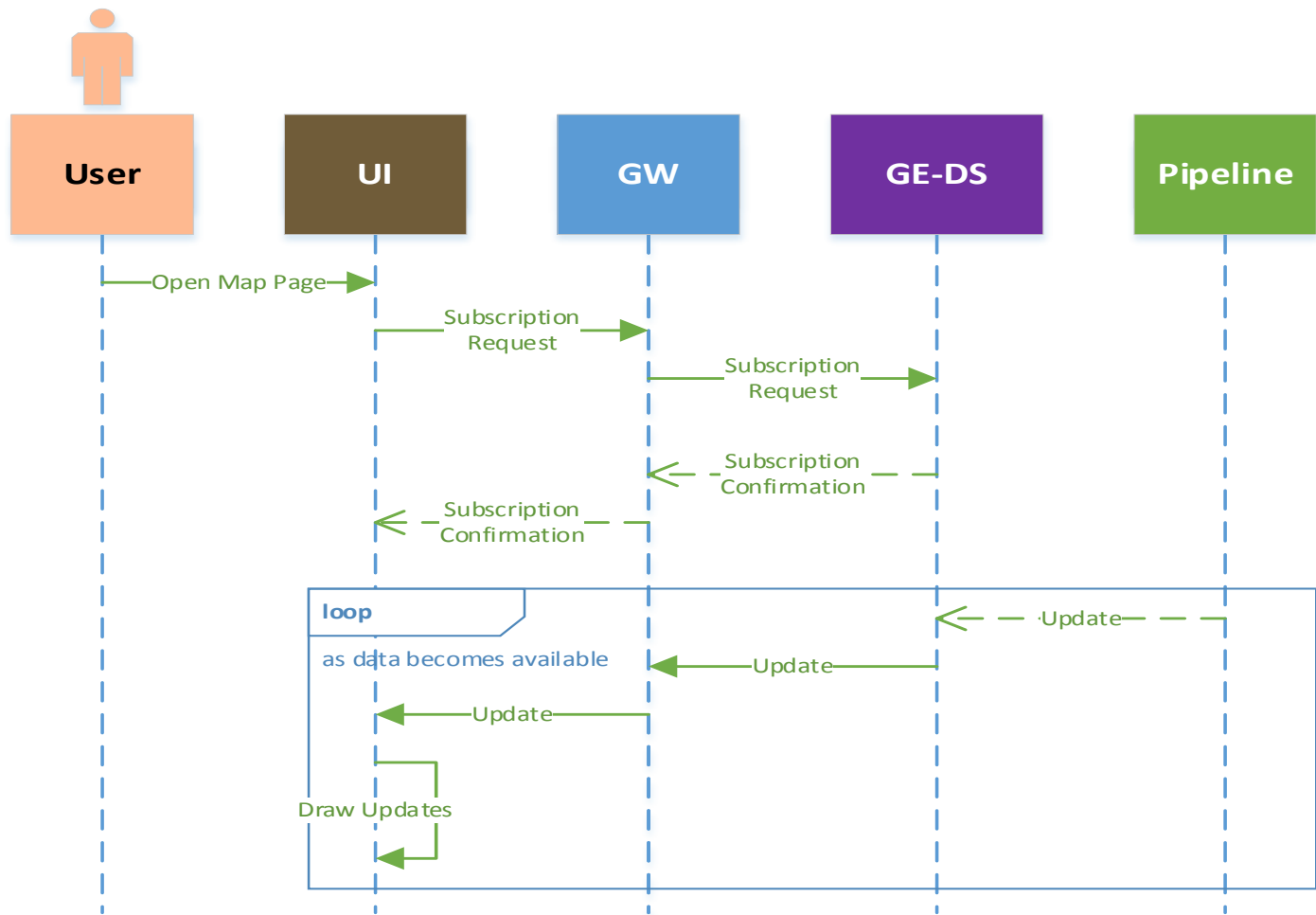


Figure 40 - Map Update for Pre-Located Data Sequence Diagram

3.7.2.3 Map Update for Dynamically Located Data

Figure 41 - Map Update for Dynamically Located Data Sequence Diagram is similar to Figure 40 - Map Update for Pre-Located Data Sequence Diagram but generalizes subscriptions for dynamically located data. Dynamically located data does not come from the GeoEvent Data Service and may come from multiple other Data Services.

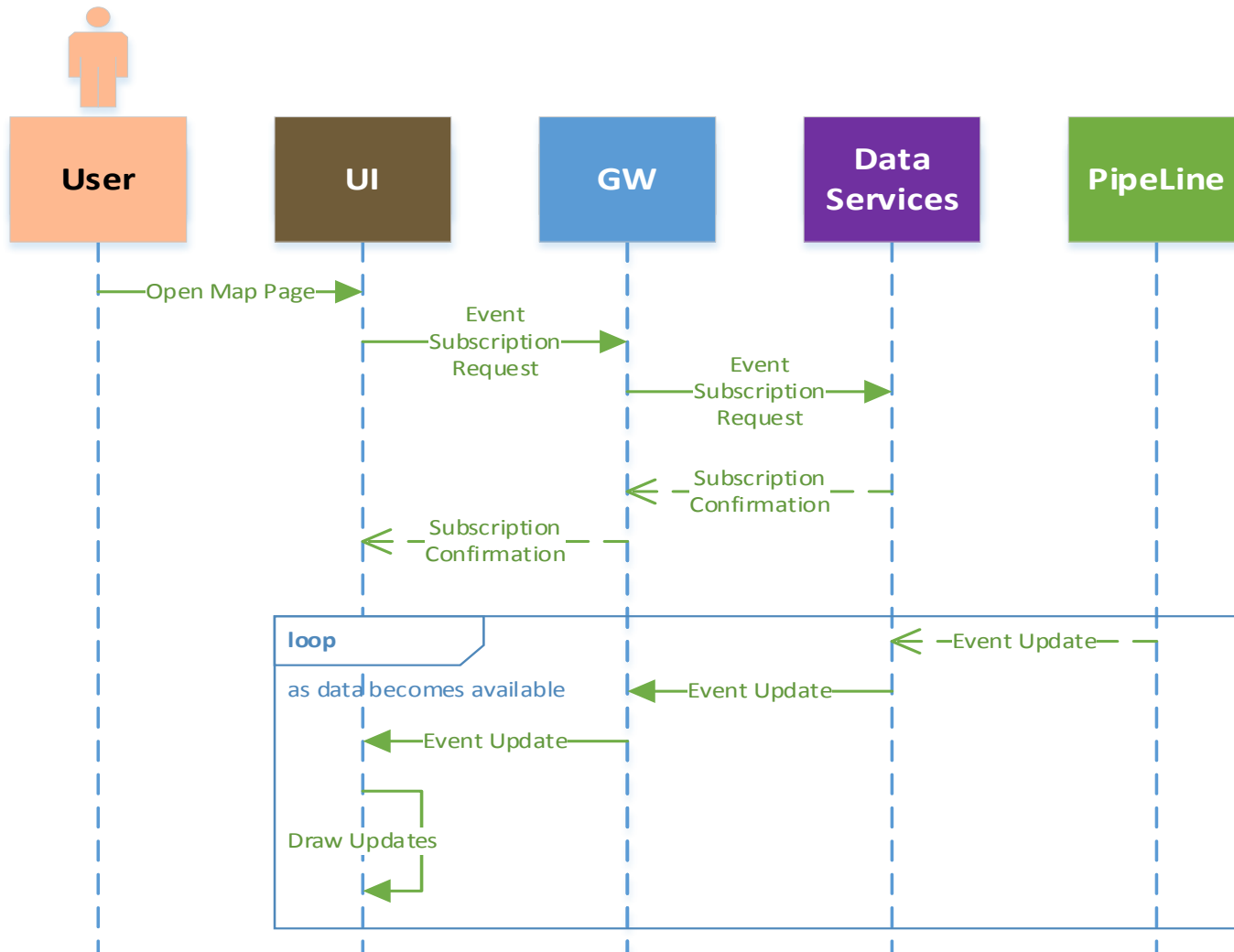


Figure 41 - Map Update for Dynamically Located Data Sequence Diagram

3.7.3 Event Diagrams

The initial implementation of the R-ICMS will generate response plans when events are generated from SunGuide or within R-ICMS. This section depicts the diagrams related to the processing of these events. It describes the main functionality contained within the DSS for processing events and initiating response plans. It describes the high-level functionality for response plan selection, response plan approval, response plan activation, and response plan reevaluation.

3.7.3.1 SunGuide / R-ICMS Event

The diagram in Figure 42 - SunGuide / R-ICMS Event Flow Diagram is a flow-chart-like diagram that shows the relationship between SunGuide and R-ICMS events. SunGuide's Incident Detection System will notify an operator when the R-ICMS system activated a response plan. If the event that triggered the response originated in SunGuide, then identifiers for the event in both systems are known and linked automatically. If the trigger event originated in R-ICMS, the SunGuide operator will be notified and have the option to link the event to a known SunGuide event.

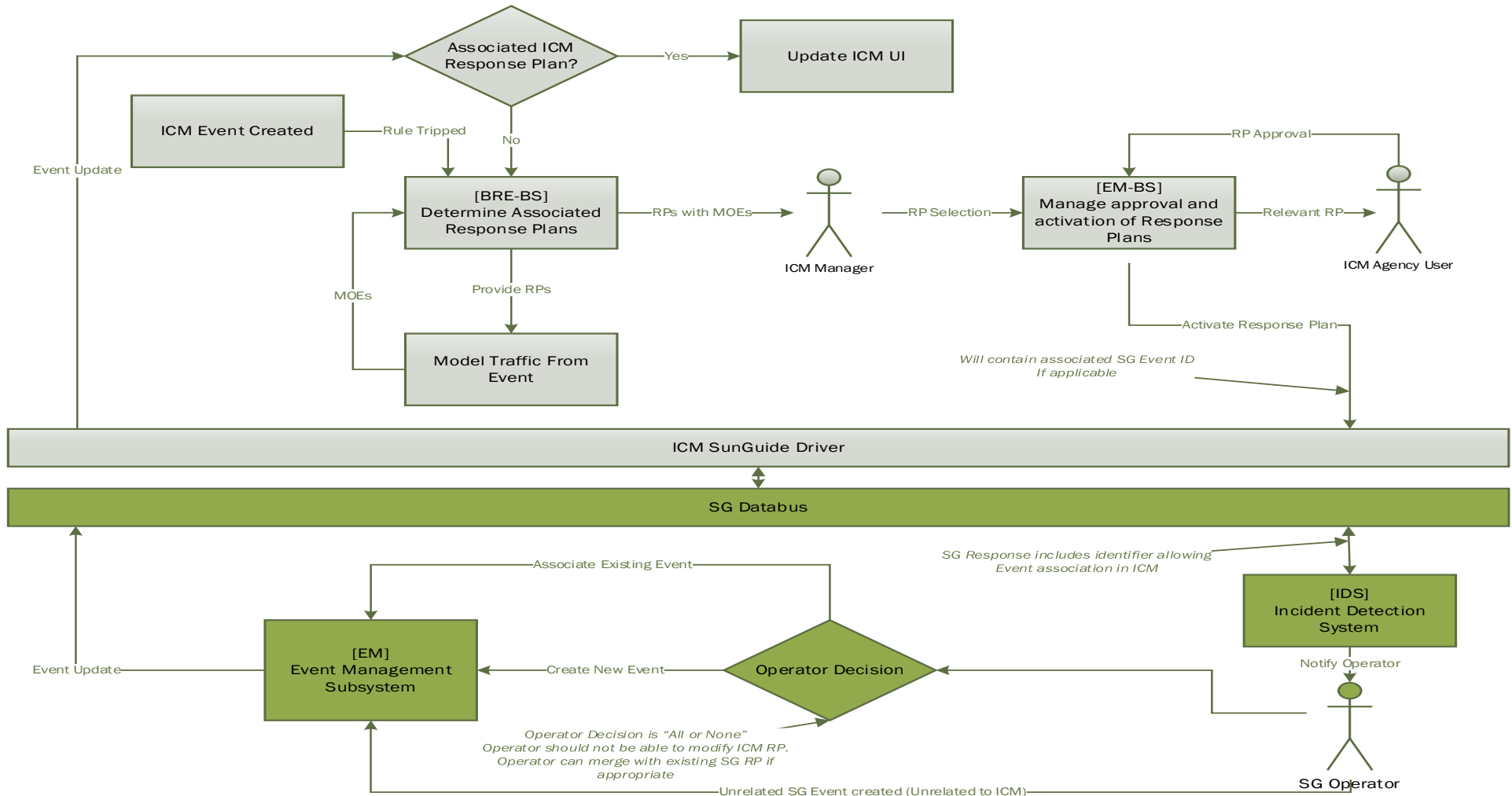


Figure 42 - SunGuide / R-ICMS Event Flow Diagram

```
sequenceDiagram
    participant Pipelines
    participant DFE
    participant EM-DS
    participant EM-BE
    participant BRE-BE
    participant GEO-DS
    participant GW
    participant UI
    participant User1 as Agency User
    participant User2 as Agency User

    Note over Pipelines,DFE: SunGuide Event
    Pipelines->>EM-DS: SunGuide Event
    activate EM-DS
    EM-DS->>EM-BE: ICM event
    deactivate EM-DS
    activate EM-BE
    EM-BE-->>EM-DS: correlate / create ICM event
    EM-BE->>BRE-BE: ICM event
    deactivate EM-BE
    activate BRE-BE
    BRE-BE->>GEO-DS: ICM event
    deactivate BRE-BE
    activate GEO-DS
    GEO-DS->>GW: ICM event
    deactivate GEO-DS
    activate GW
    GW->>UI: ICM event
    deactivate GW
    activate UI
    UI->>User1: view map
    UI->>User2: view map
    User1->>User1: view event detail from map
    User1->>UI: view event list
    User2->>UI: view event detail from list
    UI->>EM-BS: edit event
    deactivate UI
    activate EM-BS
    EM-BS->>EM-DS: ICM event
    deactivate EM-BS
    EM-DS-->>EM-BS: edit success response
    deactivate EM-DS
    activate EM-BS
    EM-BS->>BRE-BE: ICM event
    deactivate EM-BS
    activate BRE-BE
    BRE-BE->>GEO-DS: ICM event
    deactivate BRE-BE
    activate GEO-DS
    GEO-DS->>GW: ICM event
    deactivate GEO-DS
    activate GW
    GW->>UI: ICM event
    deactivate GW
    activate UI
    UI->>User1: 
    UI->>User2: 
    deactivate UI
```

Figure 43 - SunGuide / R-ICMS Event Flow Sequence Diagram

3.7.3.3 Business Rule Configuration

Figure 44 - Business Rule Configuration Sequence Diagram shows that business rules are managed by the Business Rules Engine business service (BRE-BS). R-ICMS Managers may add and modify business rules via a configuration screen in the UI or by uploading files. A pre-determined file format for business rules definition will be used to validate files loaded from external systems. Rules may be activated/deactivated by the R-ICMS Manager.

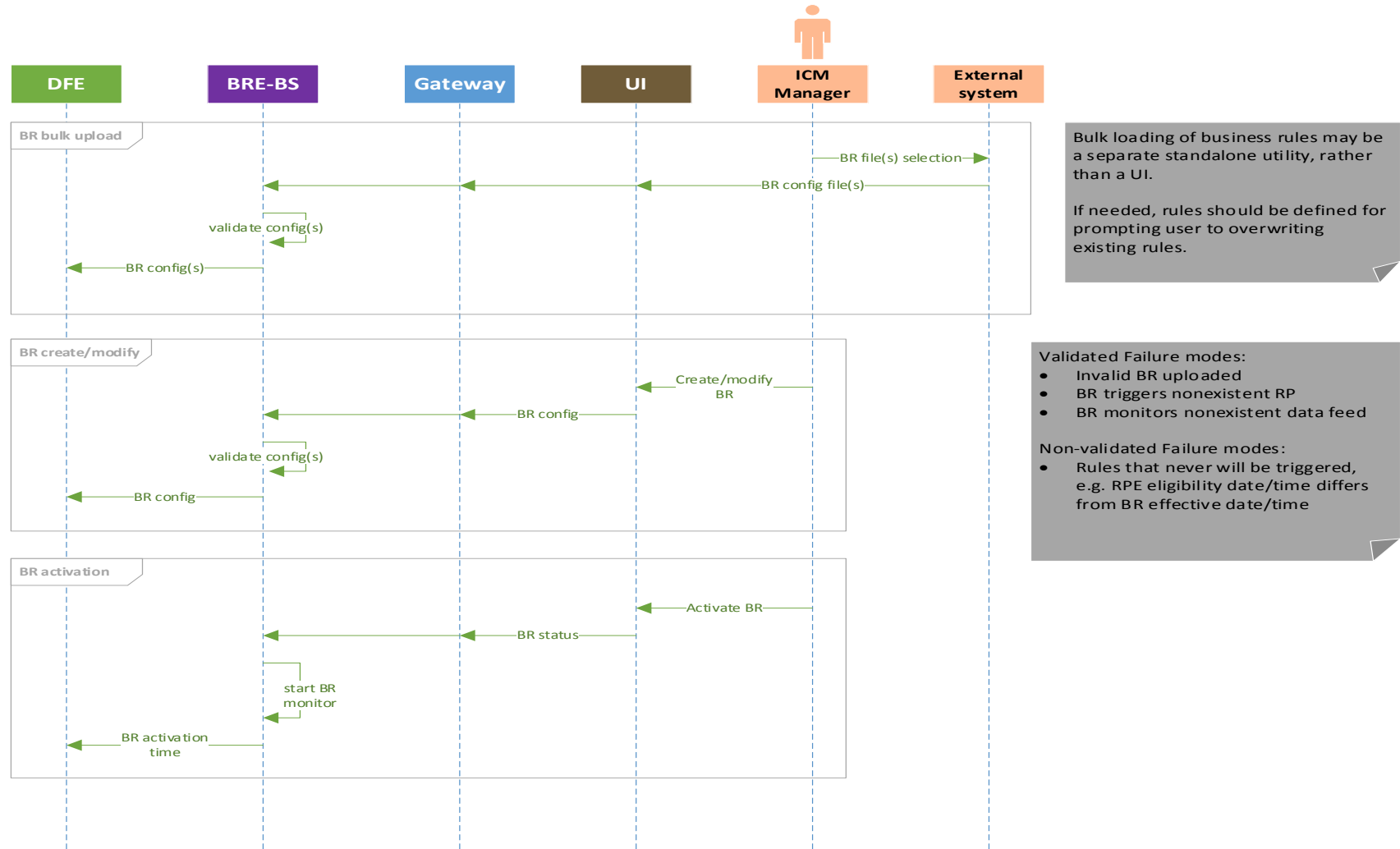


Figure 44 - Business Rule Configuration Sequence Diagram

3.7.3.4 Response Plan Selection

Figure 45 - Response Plan Selection Sequence Diagram shows the process for background triggering and ranking of alternative response plans caused by external data feeds and periodic runs of the business rules engine and surfacing those results to a Corridor Manager user.

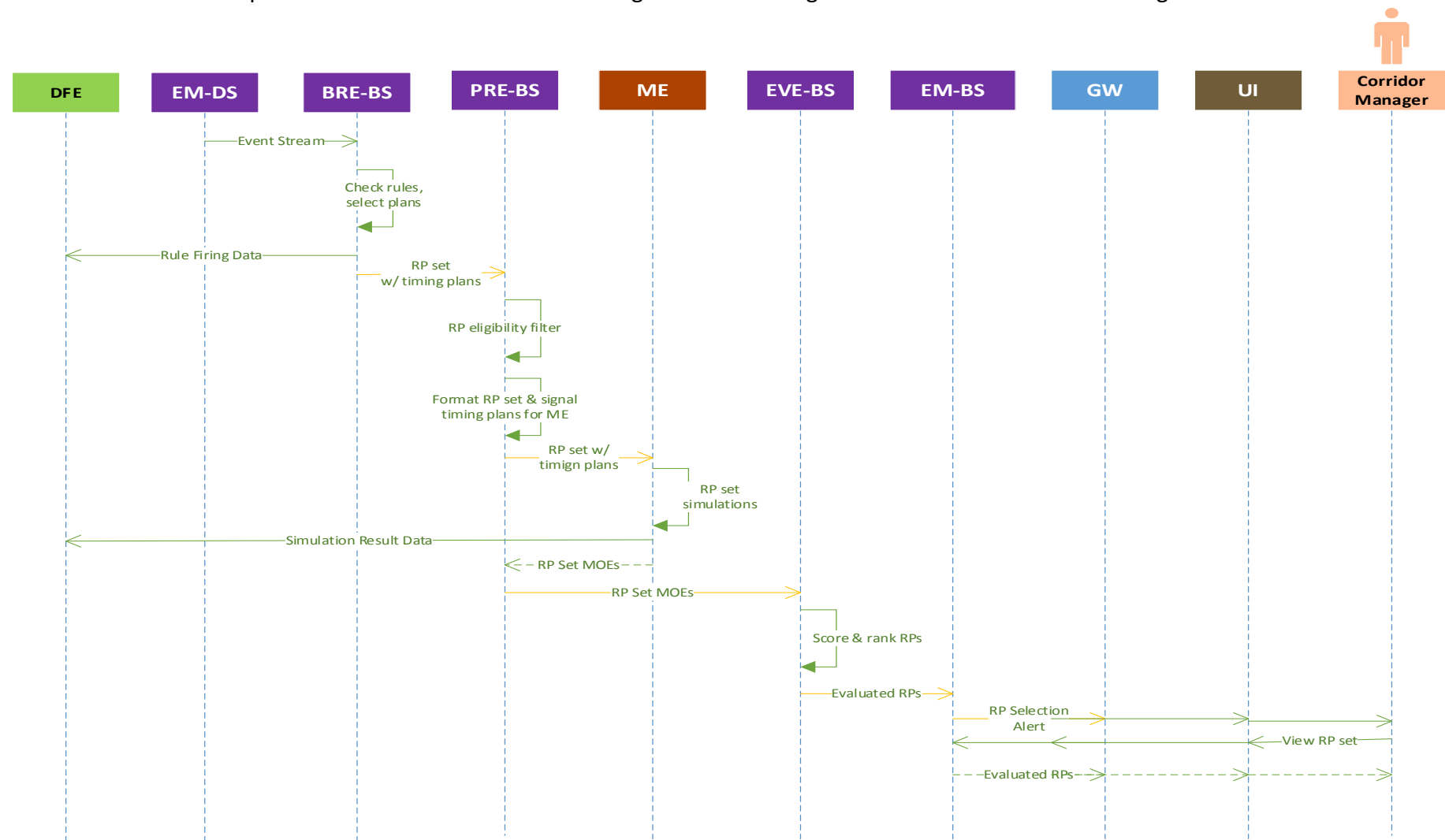


Figure 45 - Response Plan Selection Sequence Diagram

3.7.3.5 Response Plan Approval

Figure 46 - Response Plan Approval Sequence Diagram shows the process of a corridor manager user choosing a response plan from those provided by the RP Selection flow, R-ICMS soliciting for and receiving the approval of all involved agencies, and the corridor manager initiating activation of the approved response plan.

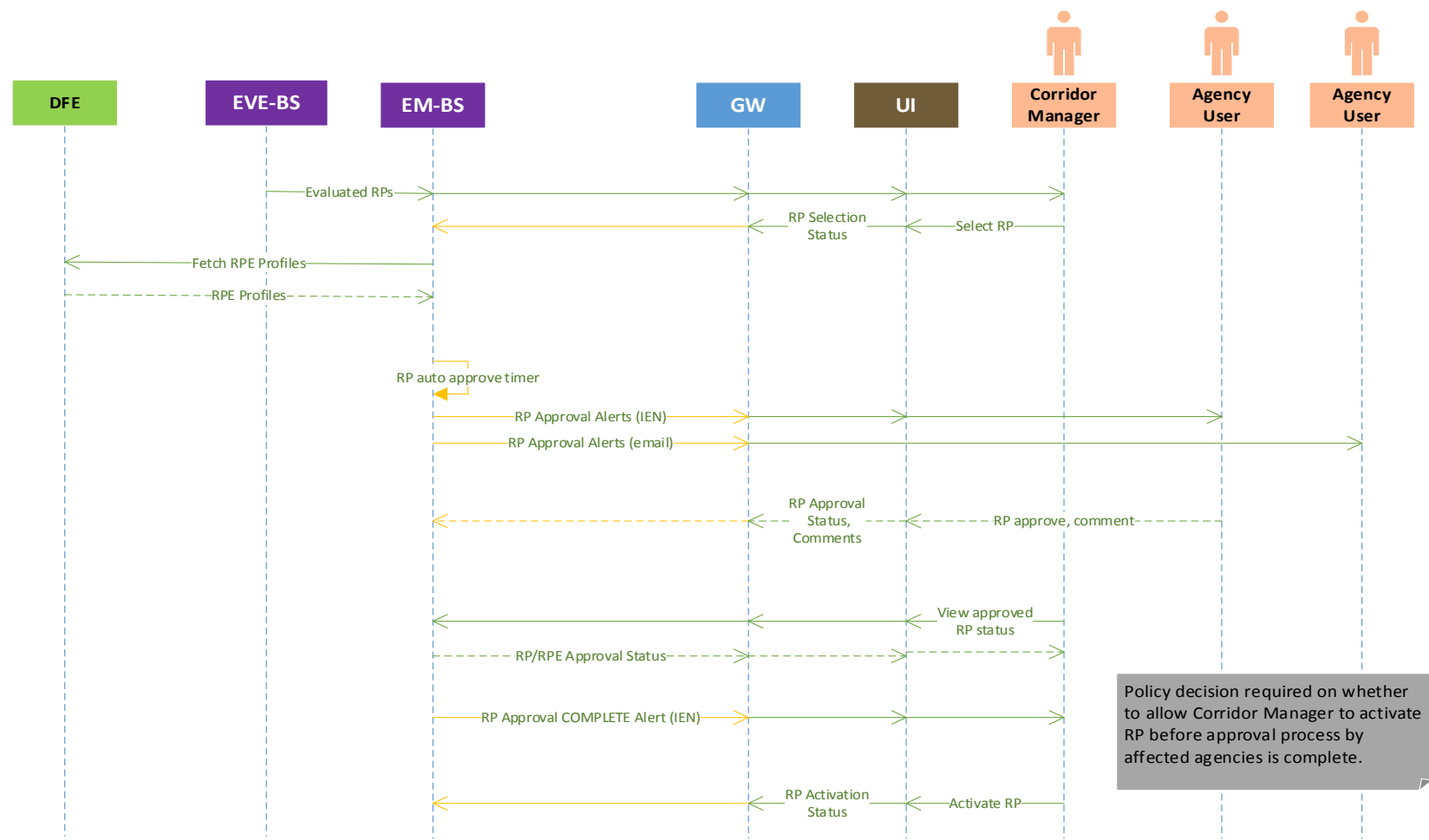


Figure 46 - Response Plan Approval Sequence Diagram

3.7.3.6 Response Plan Activation

Figure 47 - Response Plan Activation Sequence Diagram shows how a response plan is enacted after it has been activated. This involves both automatic and manual changes to the involved infrastructure.

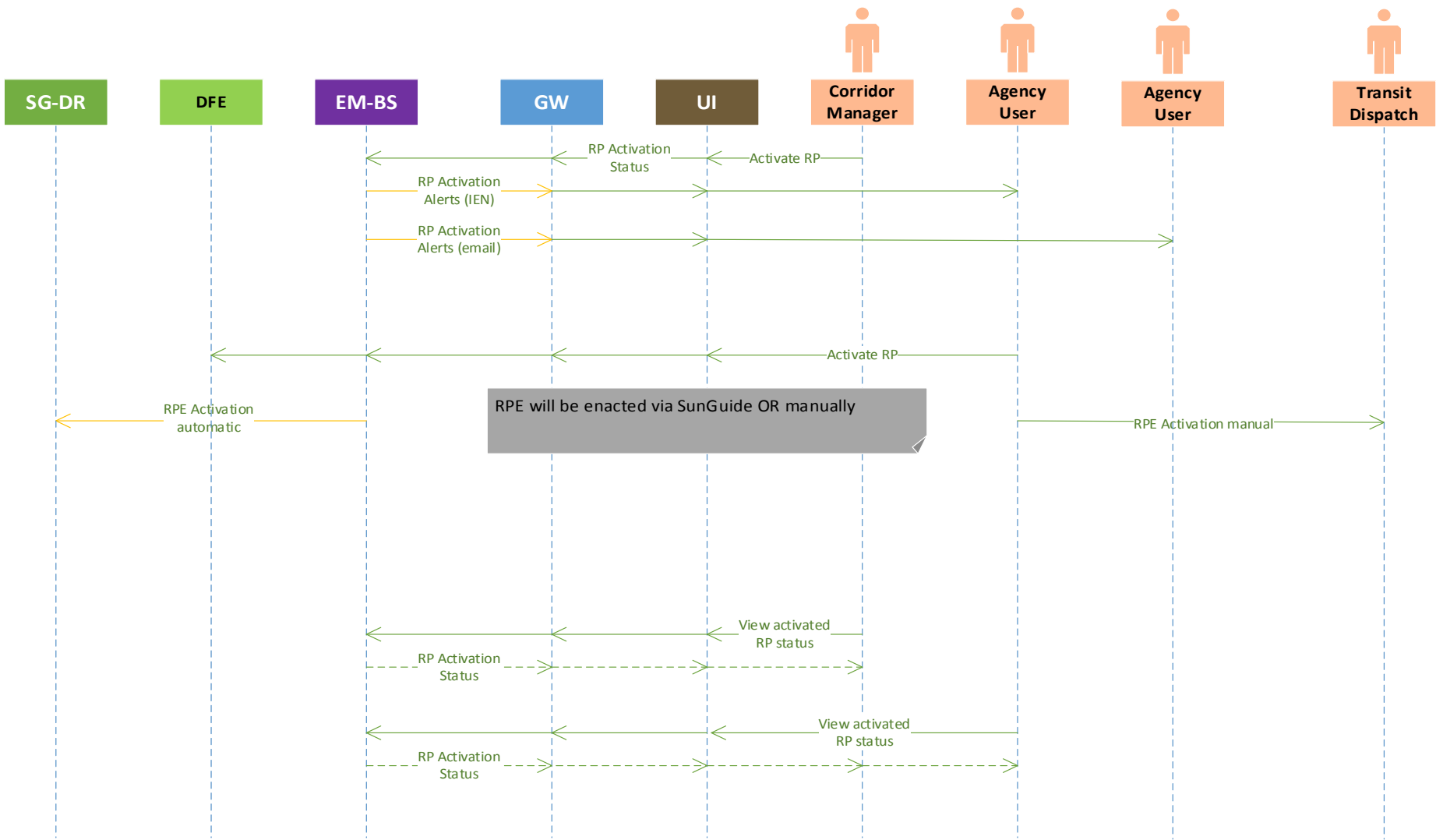


Figure 47 - Response Plan Activation Sequence Diagram

3.7.3.7 Response Plan Monitor/Reevaluation

Figure 48 - Response Plan Monitor/Reevaluation Sequence Diagram shows the continuous re-evaluation of activated response plans to show the Corridor Manager how effective the activated plan is and whether normal operations should resume.

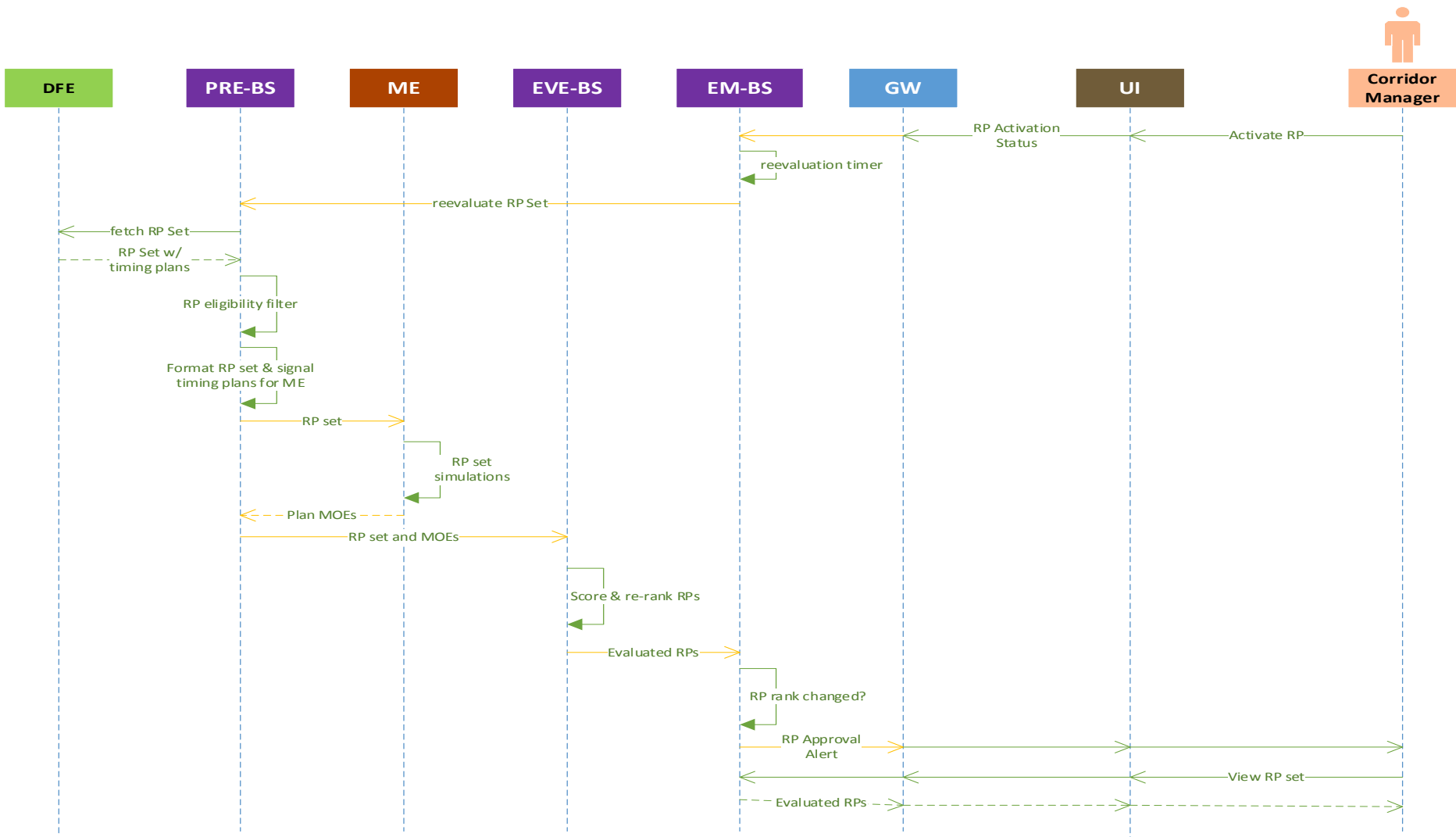


Figure 48 - Response Plan Monitor/Reevaluation Sequence Diagram

3.7.4 Signal Optimization Timing Diagrams

The R-ICMS will provide an offline Signal Optimization Timing Process Service which will allow users to optimize timing patterns on traffic signals within pre-defined corridors. The diagrams in the following subsections depict the ability for users to initiate user defined optimizations, perform periodic optimizations, and to modify the optimized signal timing plans.

3.7.4.1 Periodic Optimization

Figure 49 - SOT - Periodic Optimization Sequence Diagram shows the automatic, periodic background signal optimization process. The process is performed for all corridors on a periodic basis, and each corridor may be configured individually. Once configured, the SOT business service (SOT-BS) periodically clusters traffic demand for corridors into time ranges based on intersection movement counts, fetches the timing plans for the resultant cluster, and interfaces with an HCS7 Streets system to perform timing plan optimizations. The SOT-BS uses the Predictive Engine business service (PRE-BS) to interface with the modeling engine to simulate the implementation of optimized timing plans along with the normal time-of-day timing plans and passes them to the Evaluation Engine business service (EVE-BS) for scoring and ranking to predict the effectiveness of optimized plans.

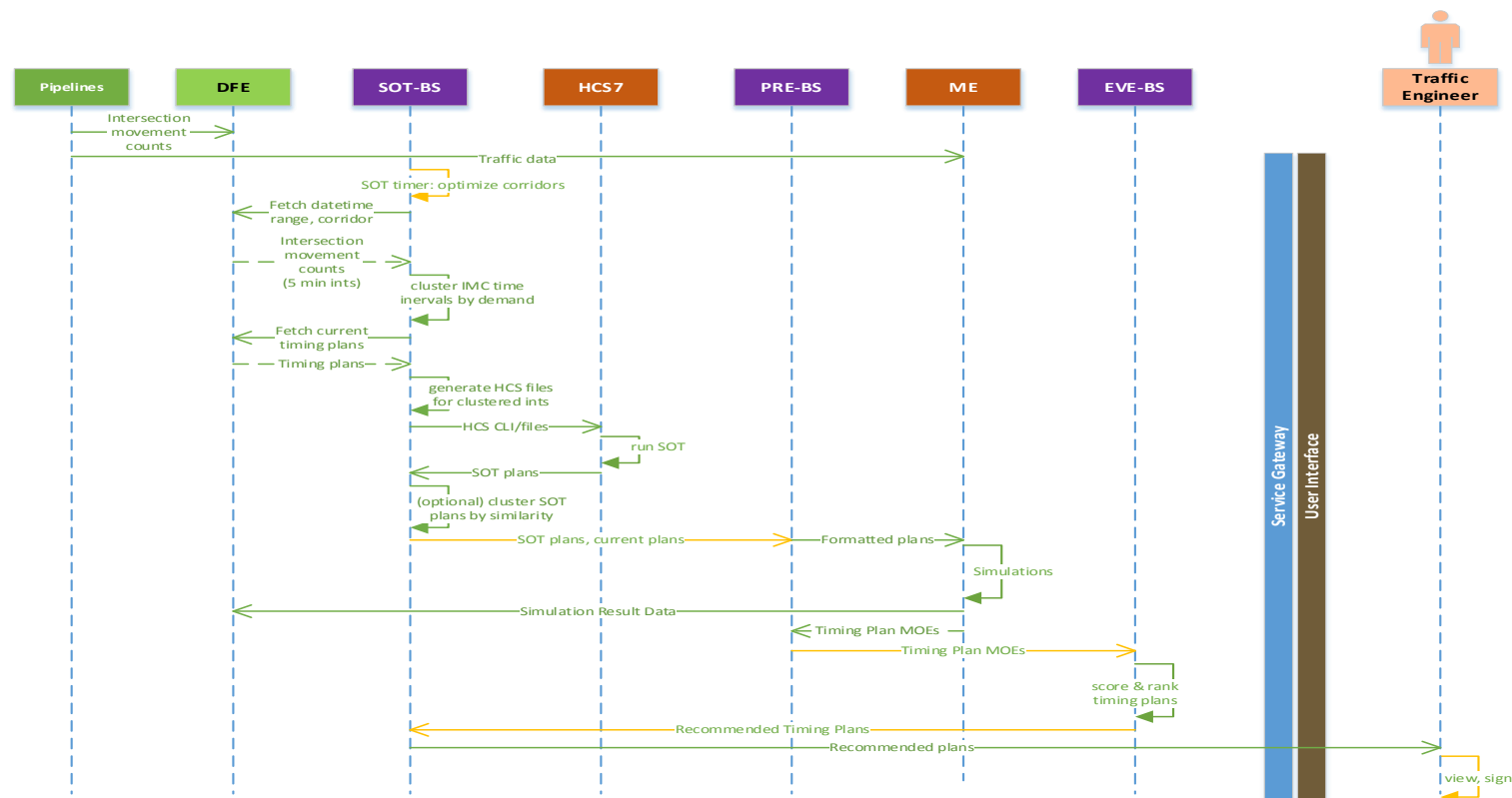


Figure 49 - SOT - Periodic Optimization Sequence Diagram

3.7.4.2 On Demand Optimization

Figure 50 - SOT On Demand Optimization Sequence Diagram shows a user-initiated signal optimization process. Via the UI, a user may select a corridor, a date range of historical intersection count data, and other possible items used by the SOT tool (HCS7 Streets).

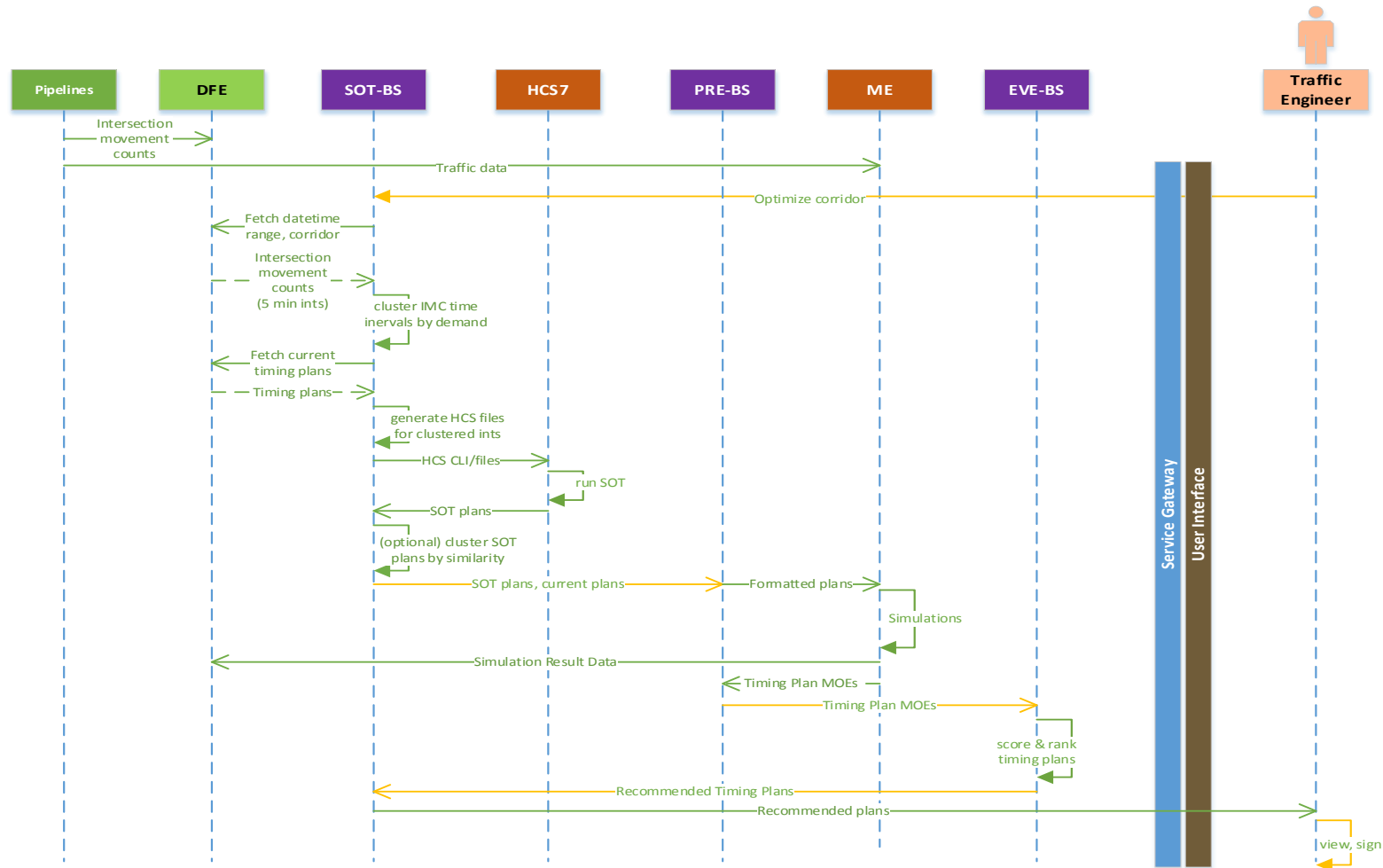


Figure 50 - SOT On Demand Optimization Sequence Diagram

3.7.4.3 Modified Timing Plan

Figure 51 - SOT Modified Timing Plan Sequence Diagram shows the process of manually modifying the offsets of a suggested signal timing plan and re-evaluating the modification.

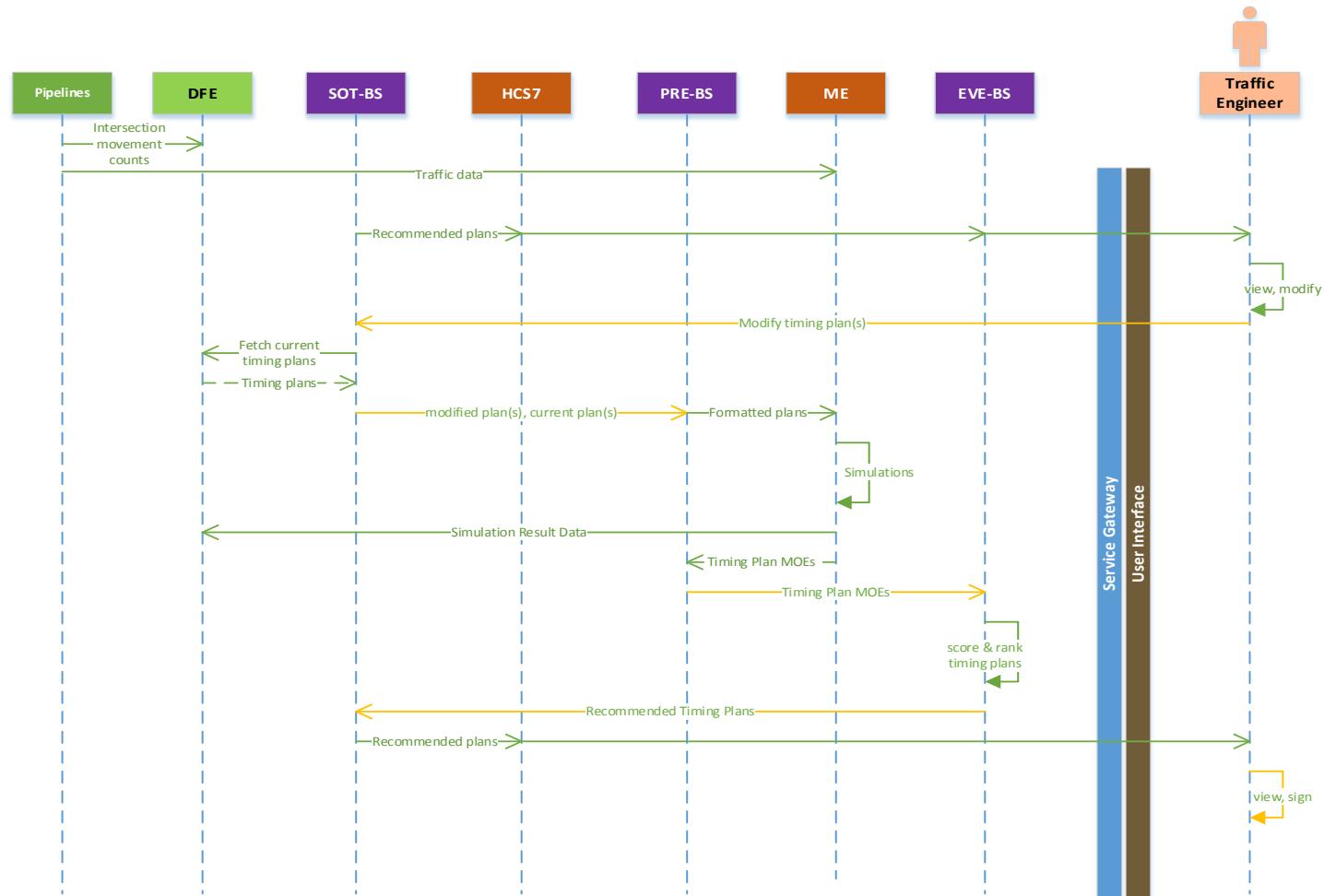


Figure 51 - SOT Modified Timing Plan Sequence Diagram

3.7.4.4 Detailed: Create Corridor

Figure 53 – SOT Detailed SOT-DS Intersection Geometry Diagram shows the process of creating a corridor based on user configured set of intersections and activation times.

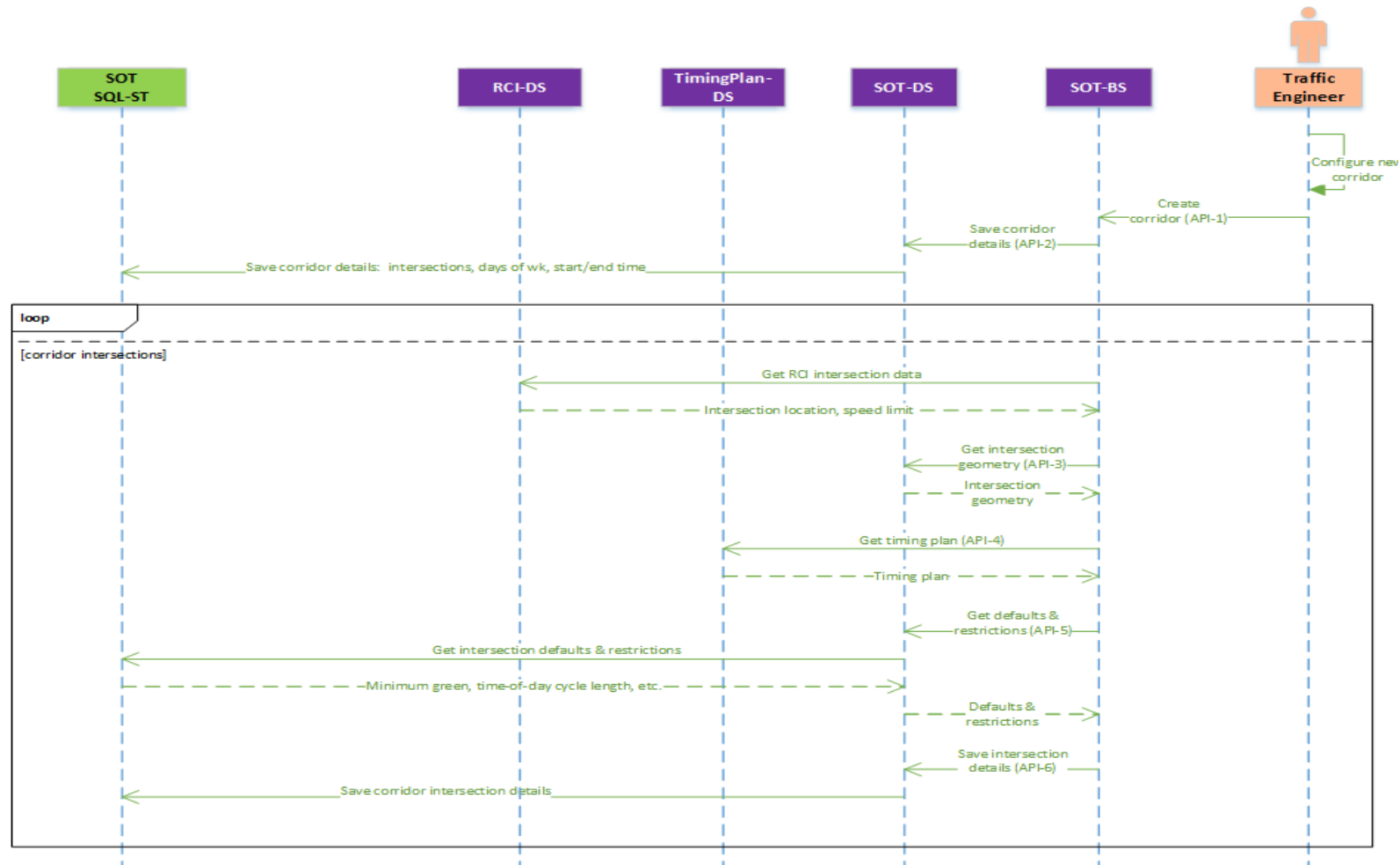


Figure 52 – SOT Detailed Create Corridor Sequence Diagram

3.7.4.5 Detailed: SOT-DS Intersection Geometry

Figure 53 – SOT Detailed SOT-DS Intersection Geometry Diagram shows the SOT data service retrieval of initial intersection data required by the SOT business service from various backing data stores.

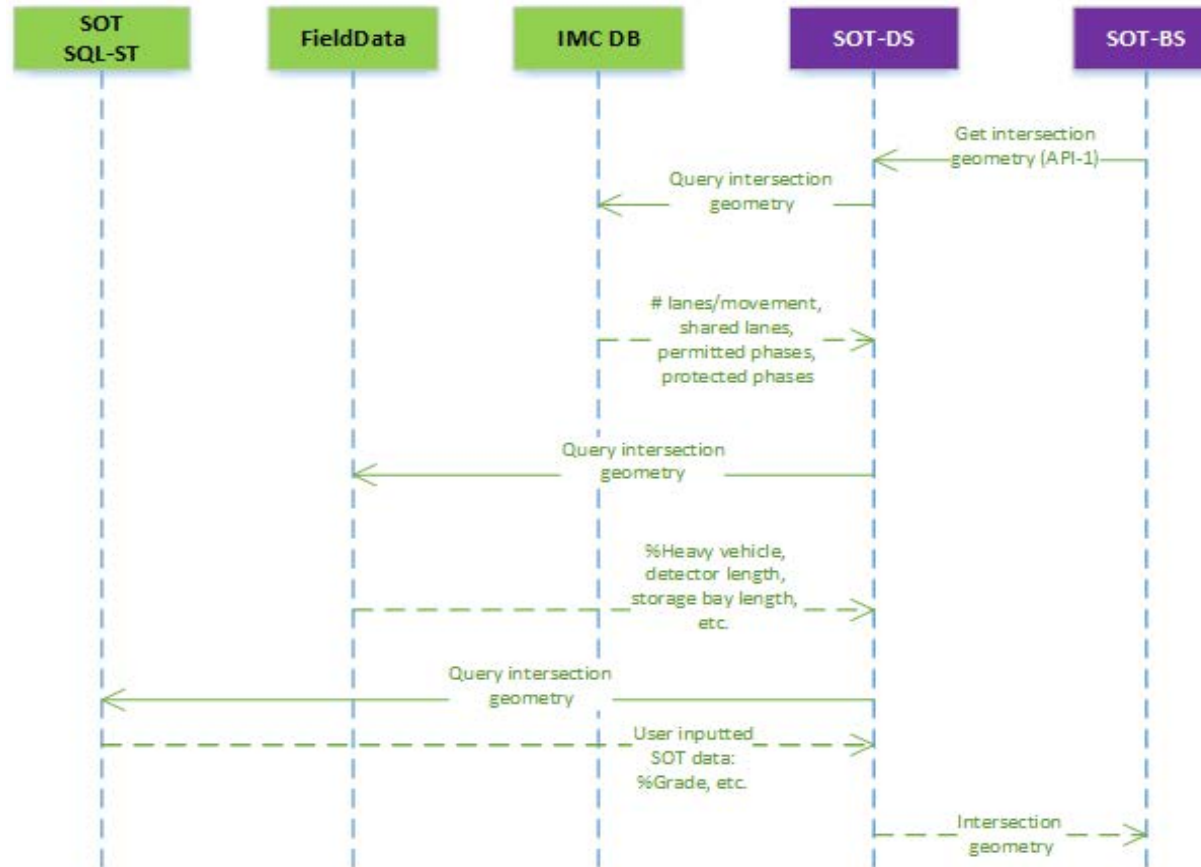


Figure 53 – SOT Detailed SOT-DS Intersection Geometry Diagram

3.7.4.6 Detailed: Modify Corridor

Figure 54 – SOT Detailed Modify Corridor Sequence Diagram shows the process of modifying a corridor based on user configured set of intersections and activation times.

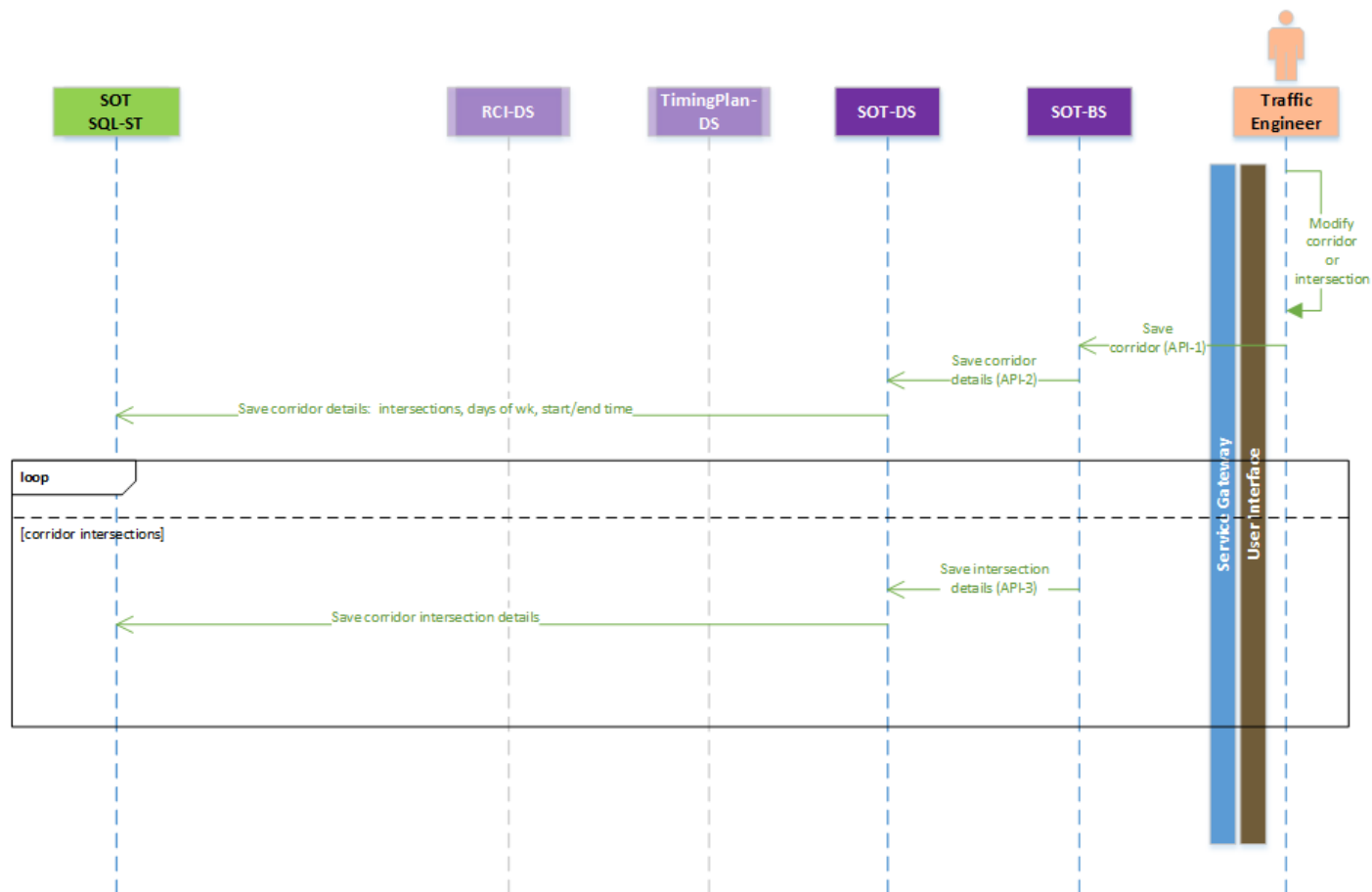


Figure 54 – SOT Detailed Modify Corridor Sequence Diagram

3.7.4.7 Detailed: Corridor Optimization 1

Figure 55 – SOT Detailed Corridor Optimization 1 shows the SOT corridor optimization process.

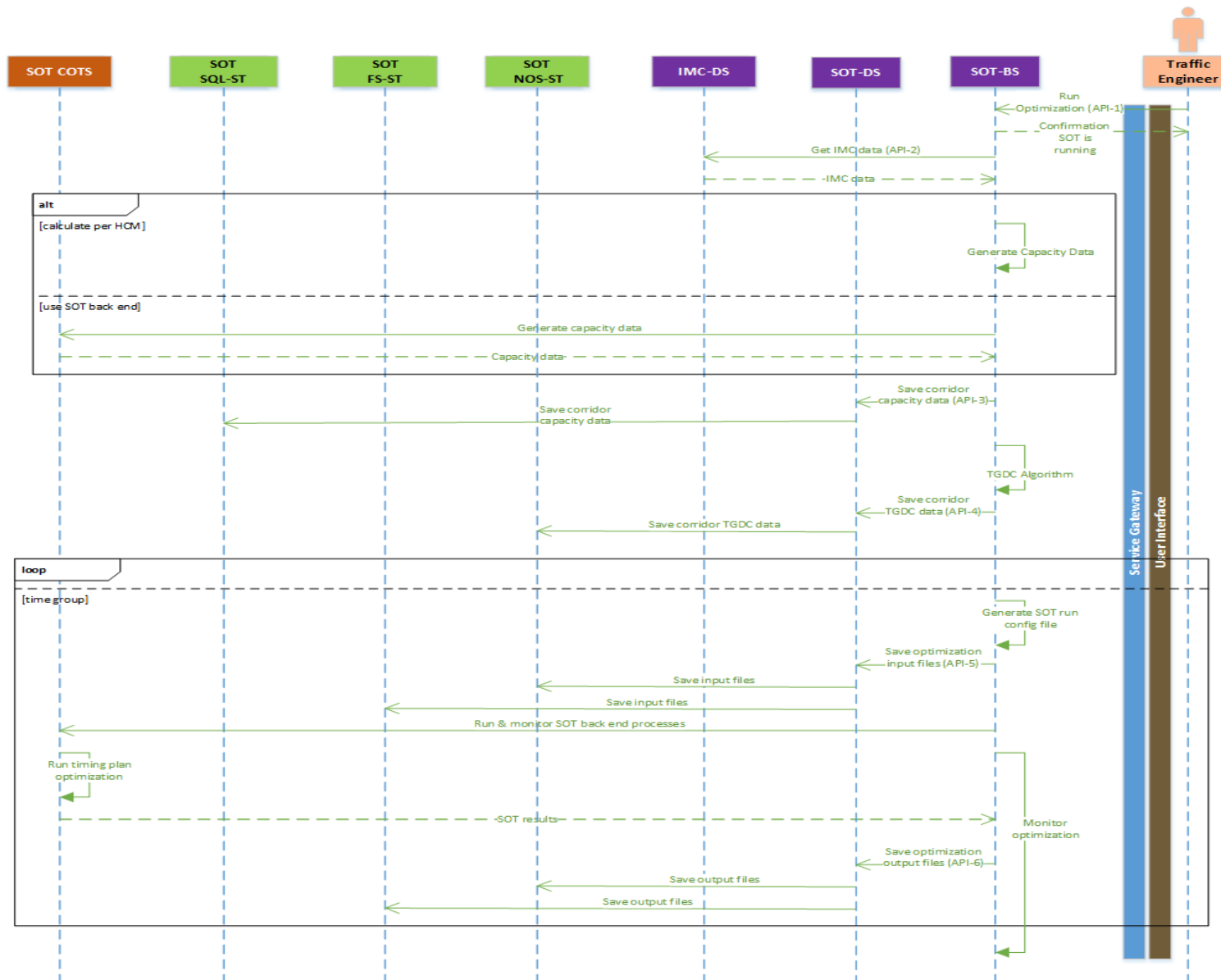


Figure 55 – SOT Detailed Corridor Optimization 1

3.7.4.8 Detailed: Corridor Optimization 2

Figure 56 – SOT Detailed Corridor Optimization 2, a continuation of Figure 55 – SOT Detailed Corridor Optimization 1, shows the extraction of timing plans from the output of a corridor optimization and user notification of the prepared results via the Alert business service.

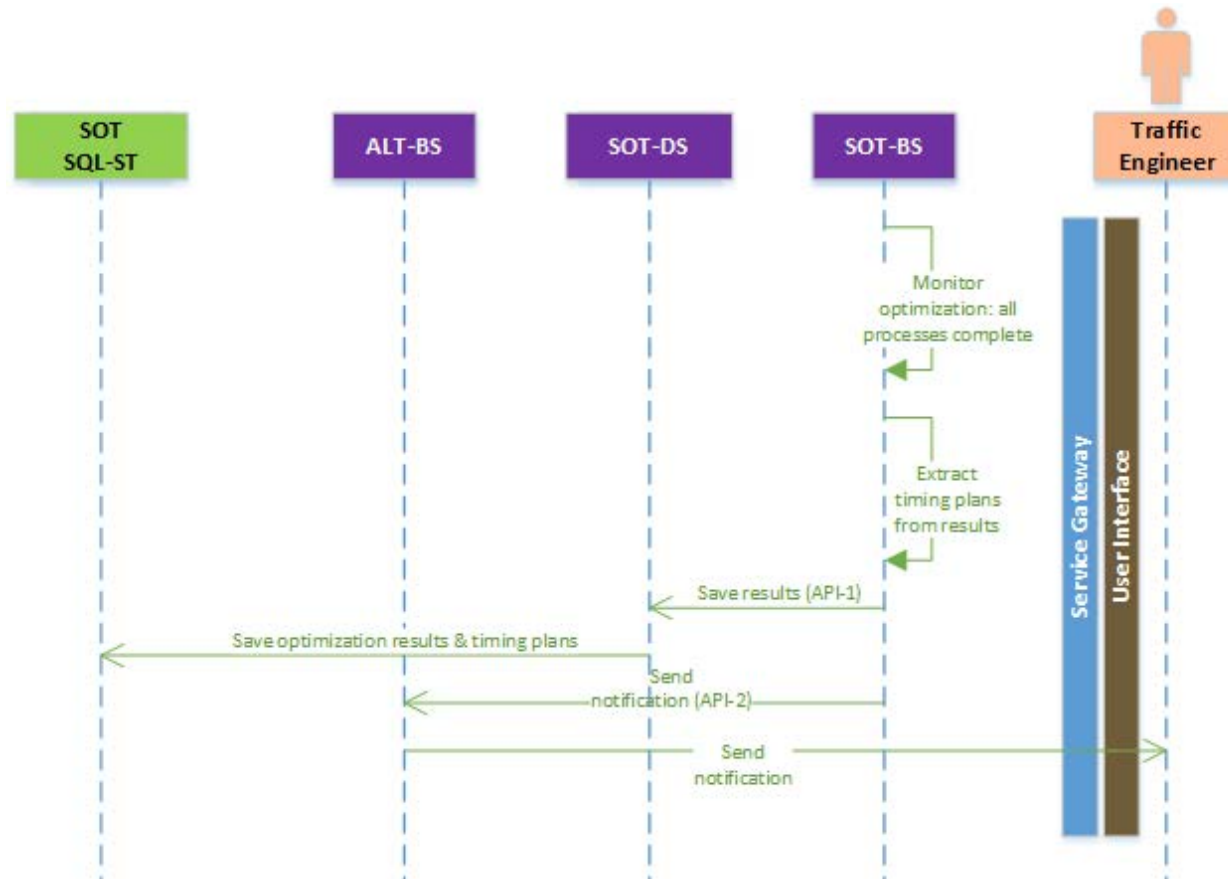


Figure 56 – SOT Detailed Corridor Optimization 2

3.7.4.9 Detailed: Deploy Corridor

Figure 57 – SOT Detailed Deploy Corridor shows a user marking a set of corridor timing plans as deployed, which triggers validation for conflicts of intersections in other corridors with overlapping activation times.

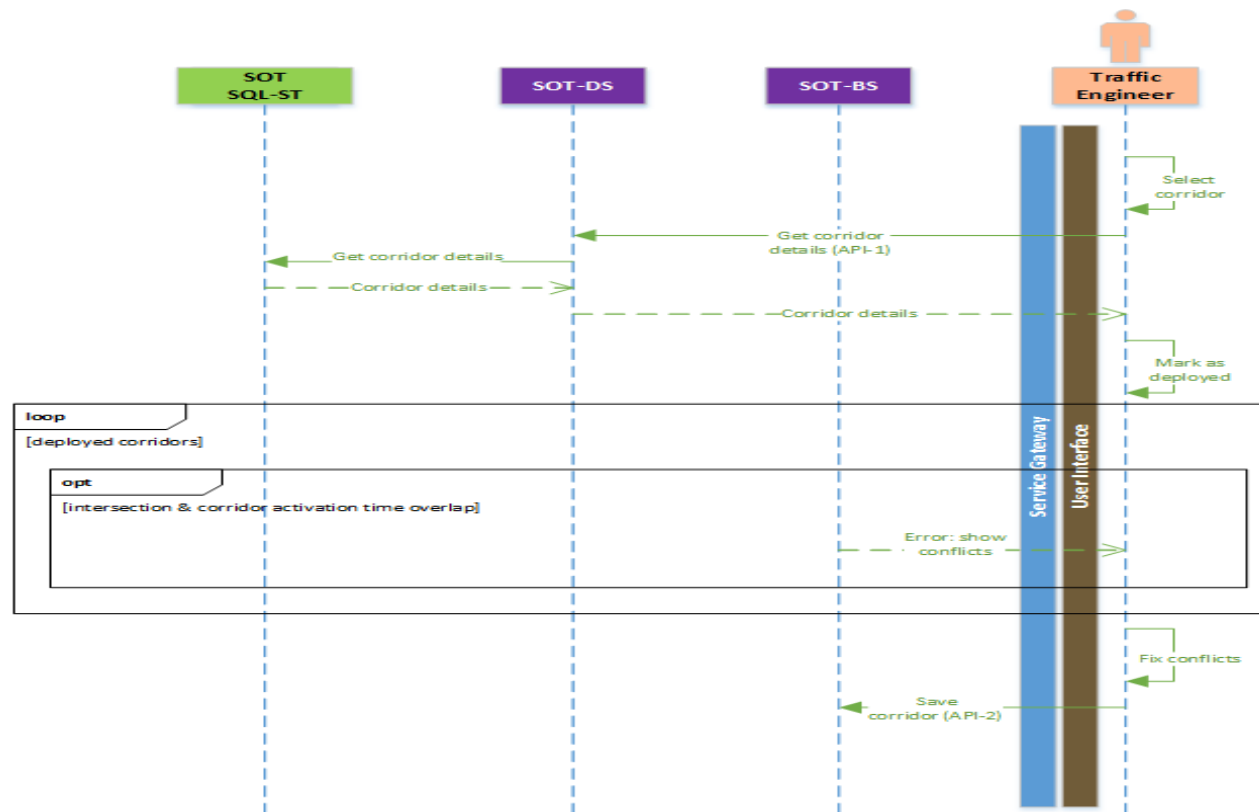


Figure 57 – SOT Detailed Deploy Corridor

3.7.4.10 Detailed: SOT-BS Logging

Figure 58 – SOT Detailed SOT-BS Logging shows interaction between the SOT business service and logging components. SOT-BS logs detailed information to files on the host system, which may be a container, virtual, or physical machine. A Beats agent (file beat) also runs on the host to monitor log files and send updates to the Kafka environment. Beats are small applications that collect and send data over the network to diverse services. A Logstash consumer group reads log information from Kafka topics and ingests it into Elasticsearch. Elasticsearch provides flexible and fast query capability for log data. Kafka is a distributed messaging system that provides information to multiple simultaneous consumers. Use of Kafka provides a buffer for the ingestion and indexing of log data by Logstash and Elasticsearch, which is a well-known bottleneck in this commonly used enterprise logging stack. The Kibana visualization tool, included with Elastic Stack, allows users to build dynamic queries and generate charts from data in Elasticsearch.

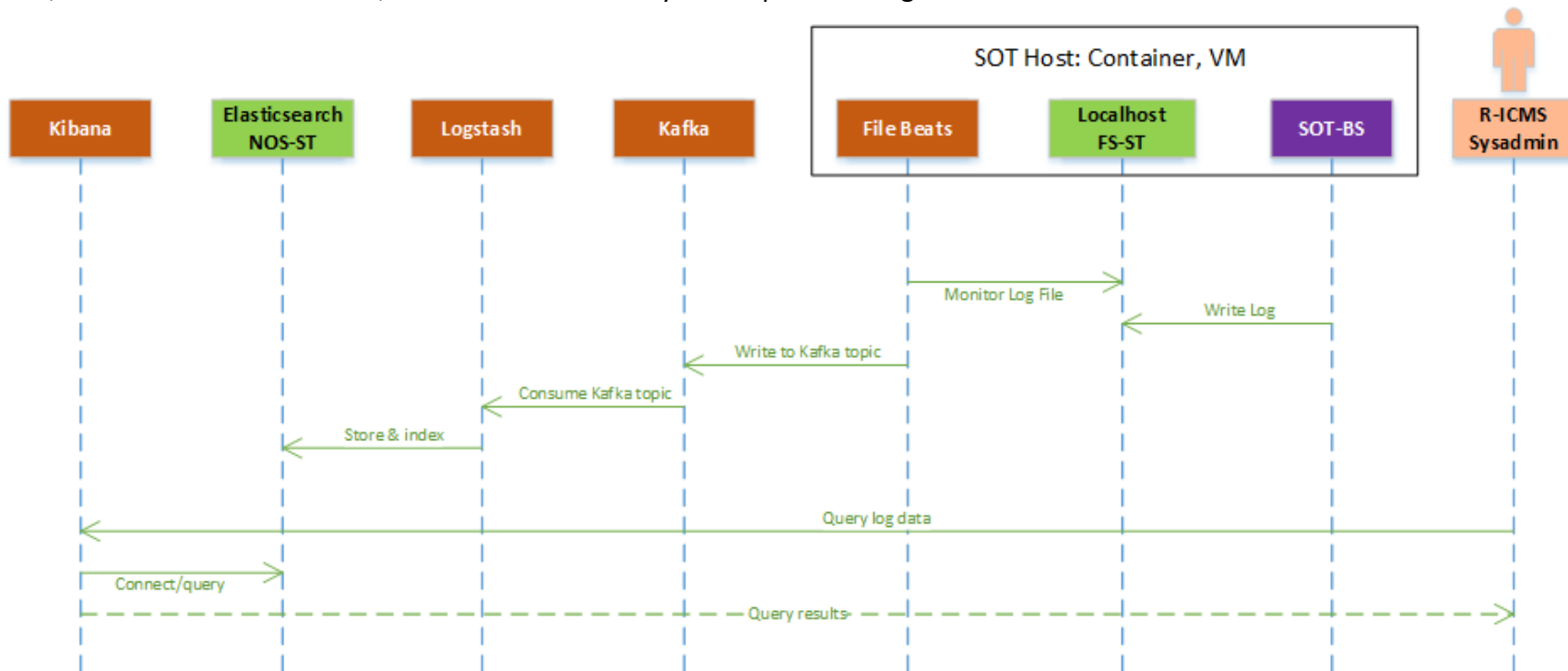


Figure 58 – SOT Detailed SOT-BS Logging

3.7.4.11 Iteration 1: Use Case Description

Figure 59 – SOT Use Cases illustrates the use cases associated the SOT functionality. Table 25 - Optimize Corridor Use Case Details provides the details for the Optimize Corridor use case.

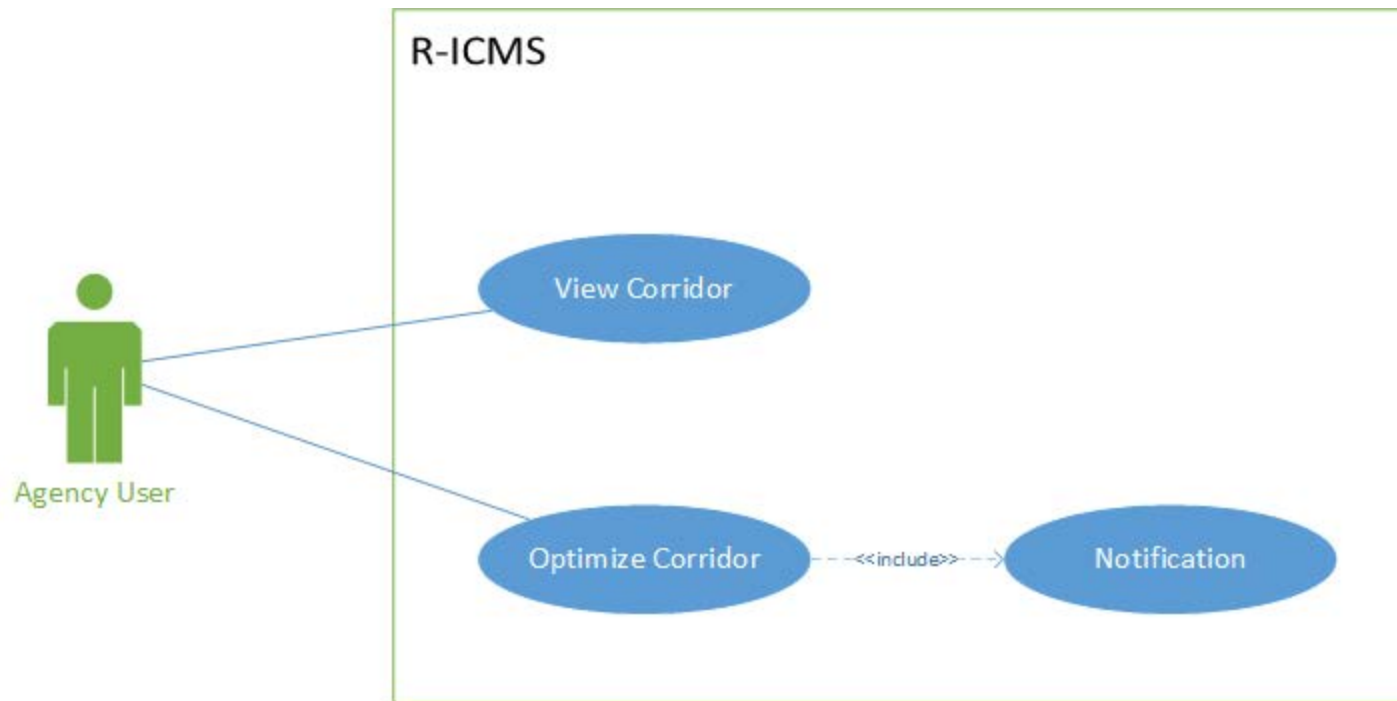


Figure 59 – SOT Use Cases

Table 25 - Optimize Corridor Use Case Details

Use Case Information	
Use Case ID	
Use Case Name	Optimize Corridor
Summary	User is able to trigger an optimization run of a test corridor
Actors	

Primary	System User
Secondary	None
Pre-conditions	The user must be logged into the system and have privileges to use the corridor feature. Test input data and corridor configuration must be prepared for the test corridor.
Main Scenario	
Trigger	User Interaction
Typical Events	1. The use case begins when the user clicks the button to trigger an optimization run 2. The system disables the button that triggers the optimization run for the duration that the optimization is running 3. The system runs the signal optimization routine 4. The system notifies the user that the optimization run has completed 5. The system re-enables the button to trigger an optimization run 6. Use case ends
Includes	Notification
Input	Test input data and corridor configuration for the test corridor
Output	The user is notified that the optimization run has completed
Alternate Scenarios	
Variants	None
Extends / Uses	None
Post-conditions	None
Frequency	User driven
Exceptions	User cancels the use case. The use case ends
Related Use Cases	None

Figure 60 – SOT Operational Database Diagram

3.7.5 Other Use Case Diagrams

Various other functionalities will be needed and included as part of the R-ICMS. The following diagrams depict these functionalities including external access to the DFE, alerting functionality, bulk loading, system monitoring and logging.

3.7.5.1 Other: External Access – Authentication

Figure 61 - External Access - Authentication Sequence Diagram shows the sequence for logging in external users. External users log in the same way as a regular user, but do so without the aid of the UI. They ultimately authenticate against Active Directory/LDAP and must have permissions assigned to them within the R-ICMS application. They receive a token to be used with all requests for data.

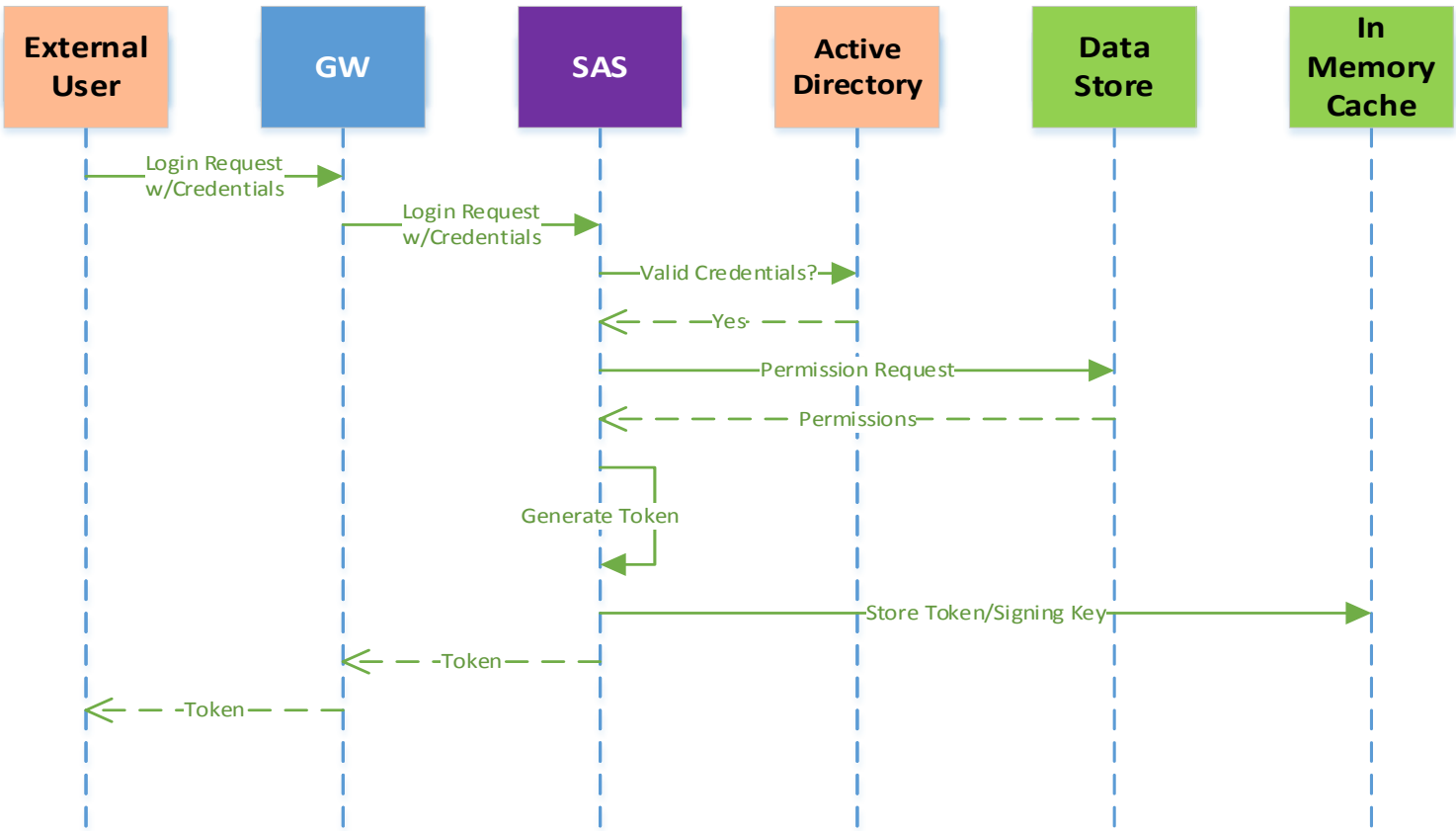


Figure 61 - External Access - Authentication Sequence Diagram

3.7.5.2 Other: External Access – Data Request

Figure 62 - External Access - Data Request Sequence Diagram shows the sequence used to satisfy external data requests. External users make data requests directly to the gateway without the aid of the UI. As with regular users, the token is passed in and authorization performed (not shown here).

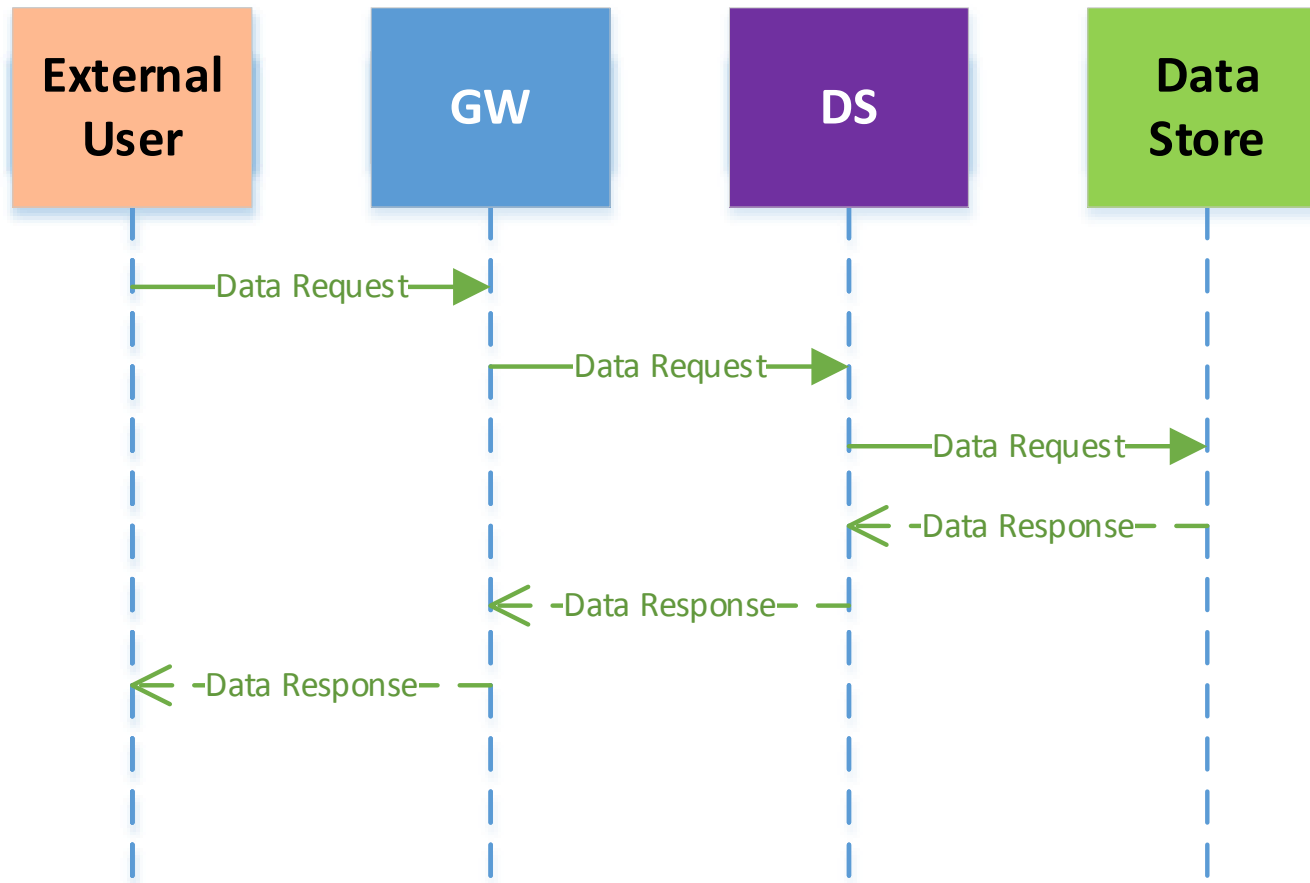


Figure 62 - External Access - Data Request Sequence Diagram

3.7.5.3 Other: External Access – Streaming

Figure 63 - External Access - Streaming Sequence Diagram shows the activities involved in accessing streaming data by an external user. The process of streaming data to an external user is similar to streaming to a regular user except that the request is made directly to the gateway. A subscription is requested and established, then updates are flowed down to the user as they come in.

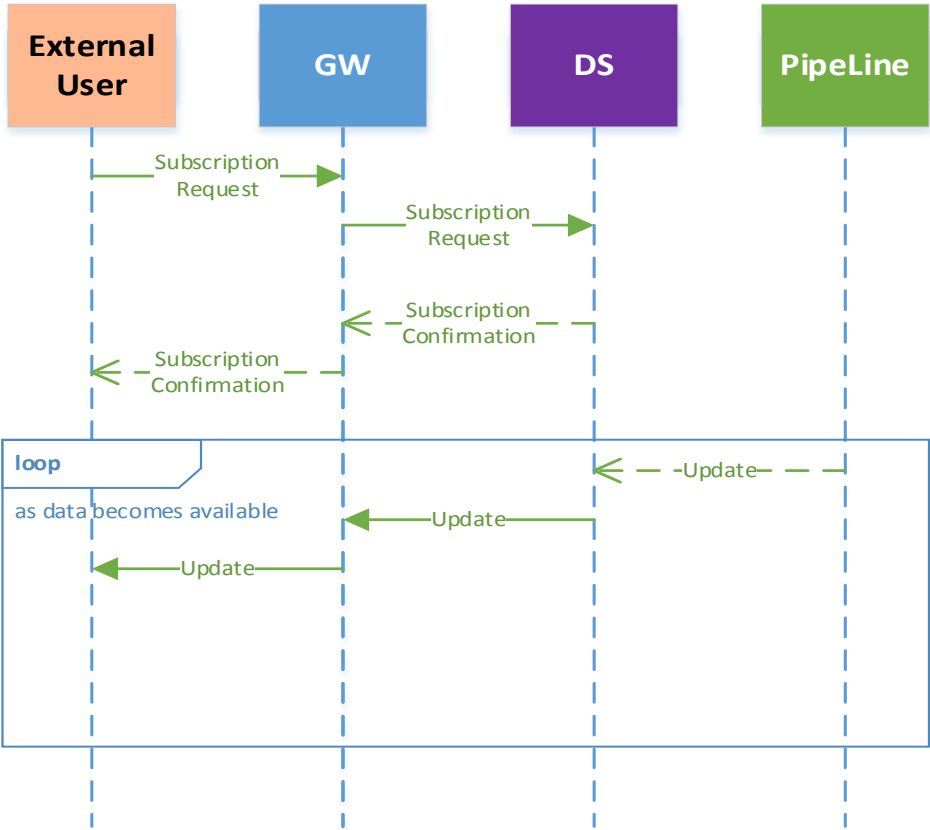


Figure 63 - External Access - Streaming Sequence Diagram

3.7.5.4 Other: External Access – Metadata

Figure 64 - External Access - Metadata Sequence Diagram shows the activities that occur when an external user requests metadata. A request for metadata is a plain data request that is made to the Metadata Data Service.

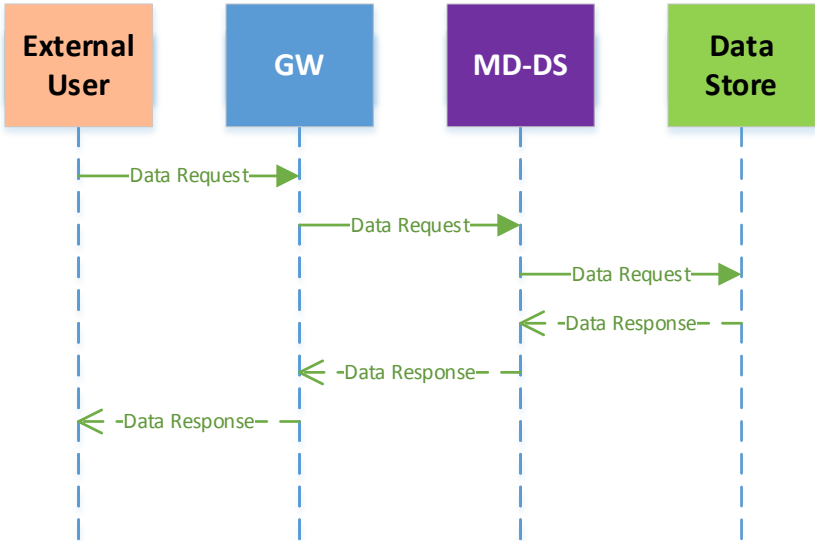


Figure 64 - External Access - Metadata Sequence Diagram

3.7.5.5 Other: Bulk Load – Business Rules

Figure 65 - Bulk Load – Business Rules Sequence Diagram shows that business rules are managed by the Business Rules Engine business service (BRE-BS). R-ICMS Managers may add and modify business rules via a configuration screen in the UI or by uploading files. A pre-determined file format for business rules definition will be used to validate files loaded from external systems. Rules may be activated/deactivated by the R-ICMS Manager.

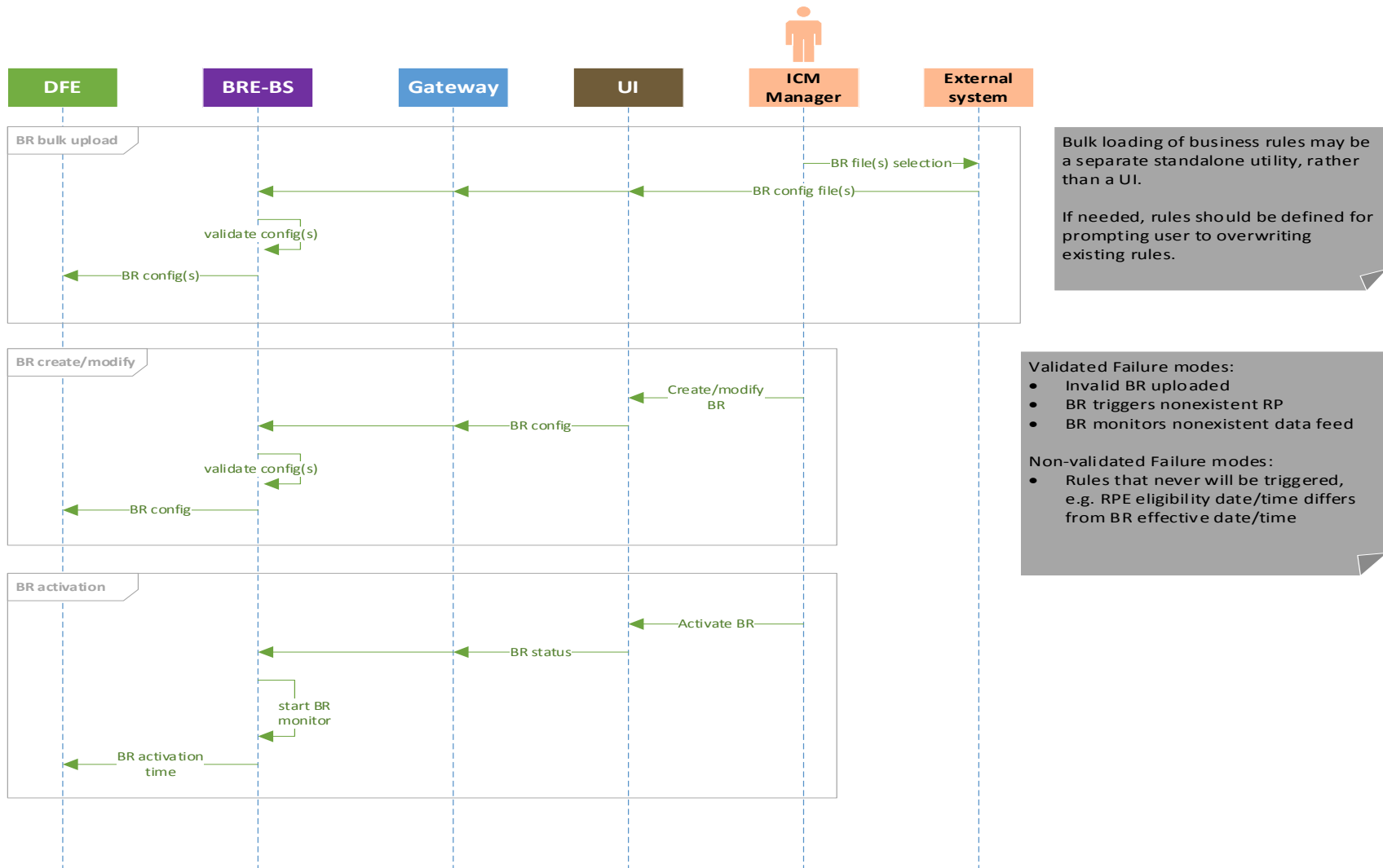


Figure 65 - Bulk Load – Business Rules Sequence Diagram

3.7.5.6 Other: Bulk Load – Response Plans

Figure 66 - Bulk Load - Response Plans Sequence Diagram shows how users add and modify RPs in R-ICMS via the UI form or by uploading files in bulk or singular. A **file format** will be defined for bulk uploaded RPs from external systems, which will be validated by the RP business service. Response plans are validated and stored in the DFE. Response plans **must include full details of signal timing plans**. These timing plans are used by the ME, and this version of the R-ICMS **assumes timing plan details are in sync with real world** timing plans with the same plan IDs.

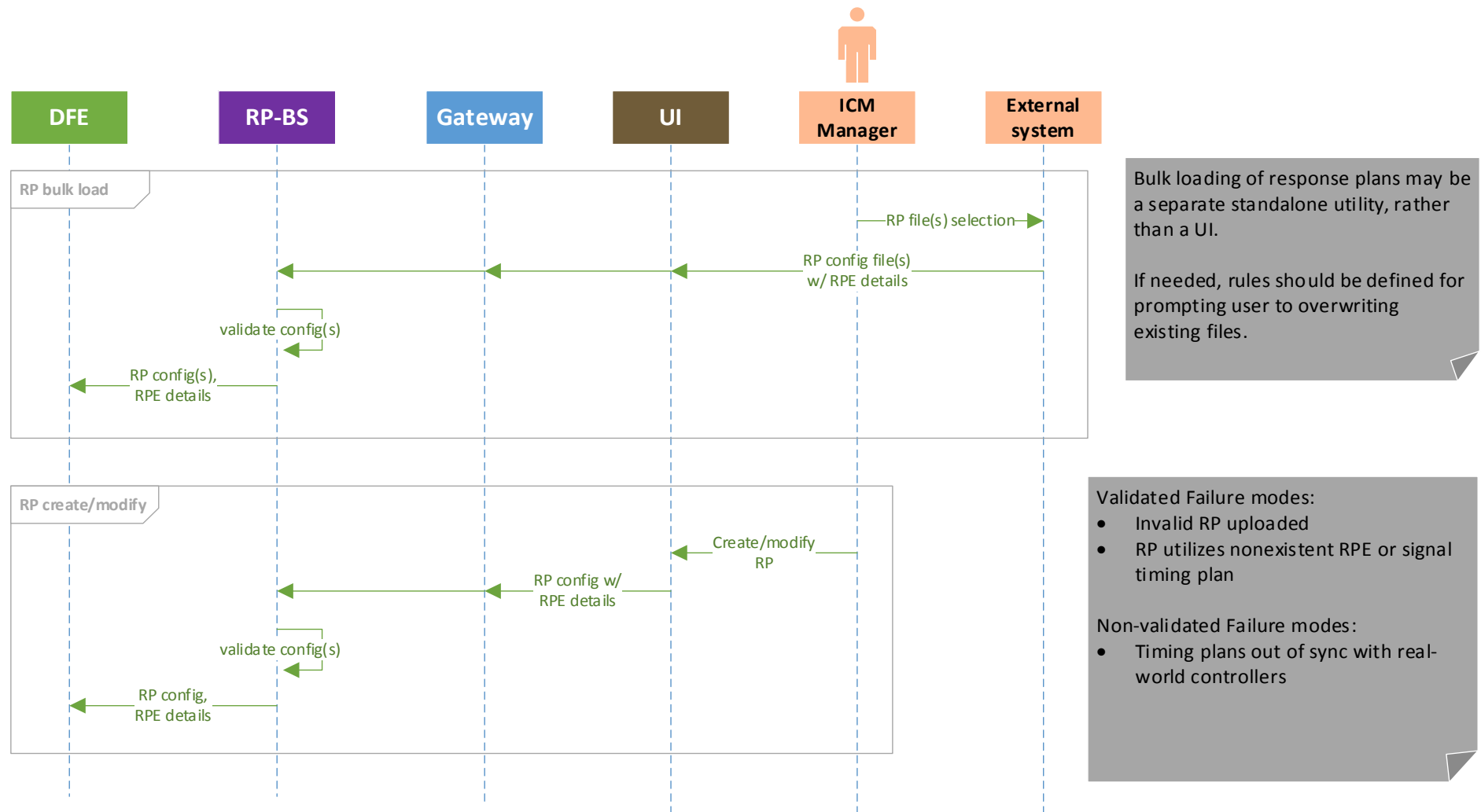


Figure 66 - Bulk Load - Response Plans Sequence Diagram

3.7.5.7 Other: Parking

Figure 67 - Parking Sequence Diagram shows the sequence of activities related to updating parking information. Pushing parking updates is similar to other update mechanisms in that an external system provides the data and it finds its way through the driver and pipelines to the relevant services, which then surface the changes via a push to an established subscription.

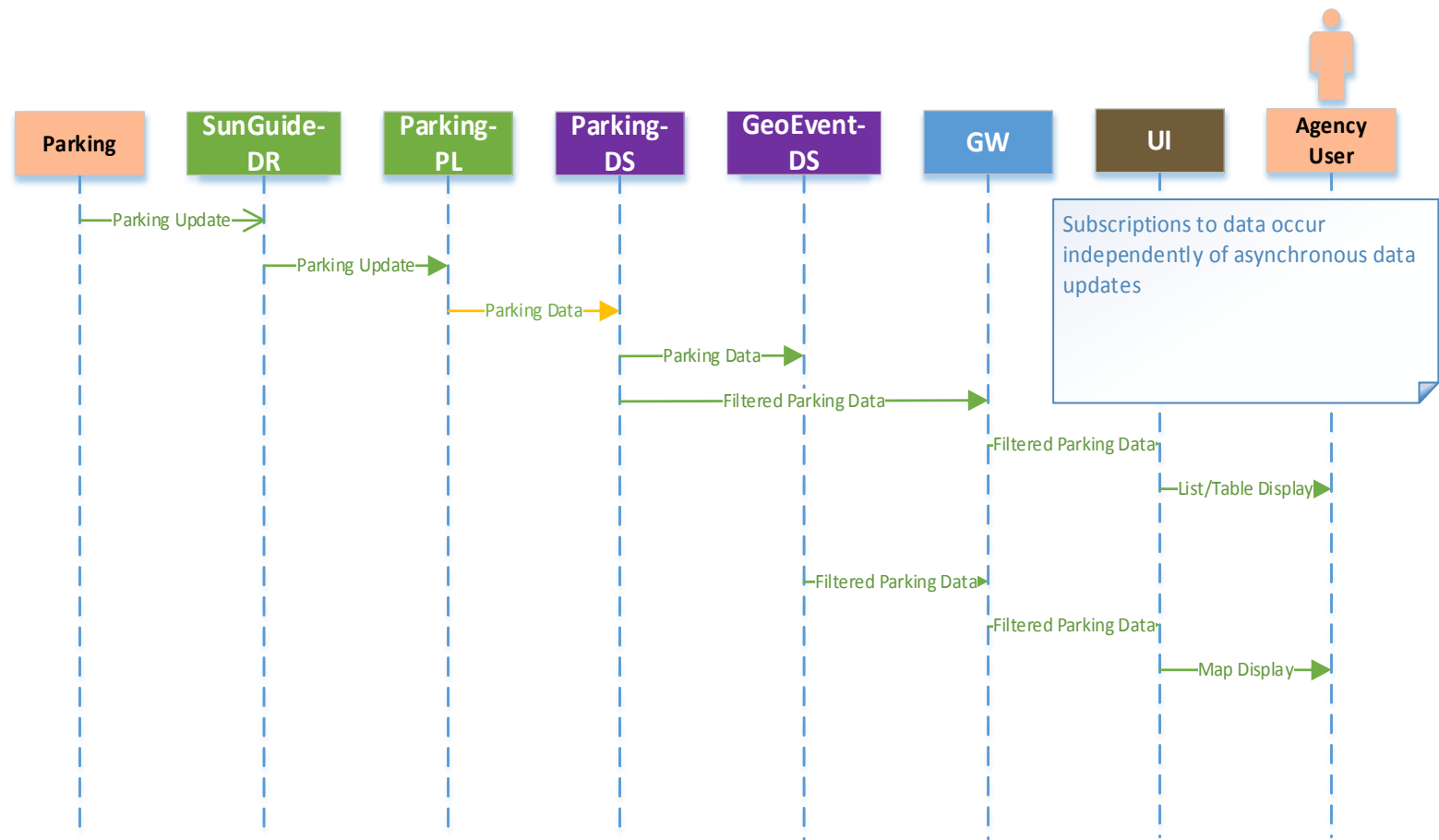


Figure 67 - Parking Sequence Diagram

3.7.5.8 Other: Alerts, Information, and Notification

The diagram in Figure 68 - Alerts, Information, and Notification Flow is a flow-chart type diagram that shows how the R-ICMS determines who approves RPEs and generates alerts for approval of an RPE using device and agency profiles.

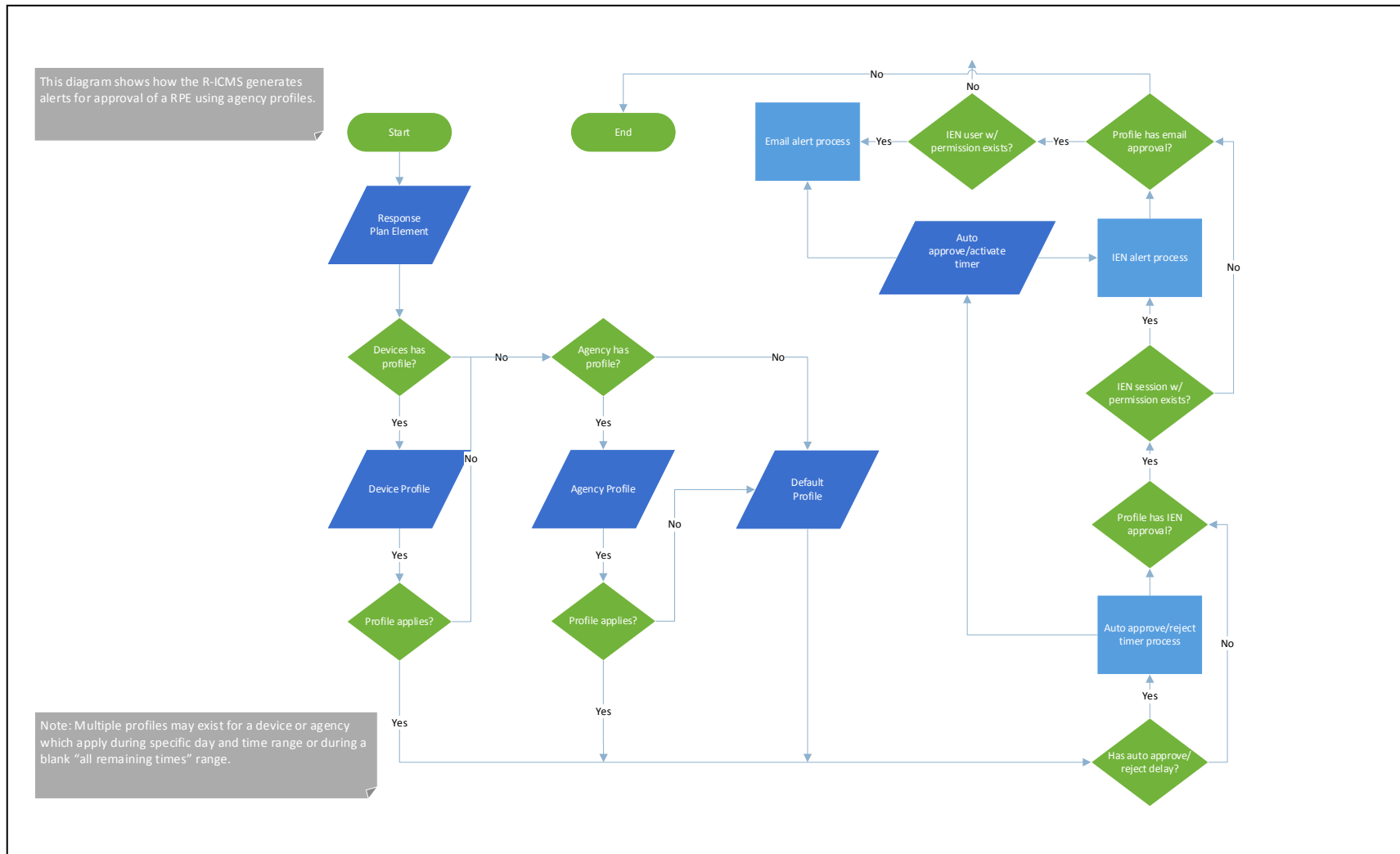


Figure 68 - Alerts, Information, and Notification Flow Chart Diagram

3.7.5.9 Other: Response Plan Notification

Figure 69 - Response Plan Notification Sequence Diagram shows a response plan approval notification process and depicts the following:

- Multiple users from the same agency may receive alerts to approve a RP
- Alerts may be received by IEN or email
- On login, users should see alerts for pending RPs (action required)
- Snoozing alerts only affects a single user
- A snoozed alert should always reappear, but if action is no longer required, it should be informational
- The status/info feed allows user to monitor the approval process

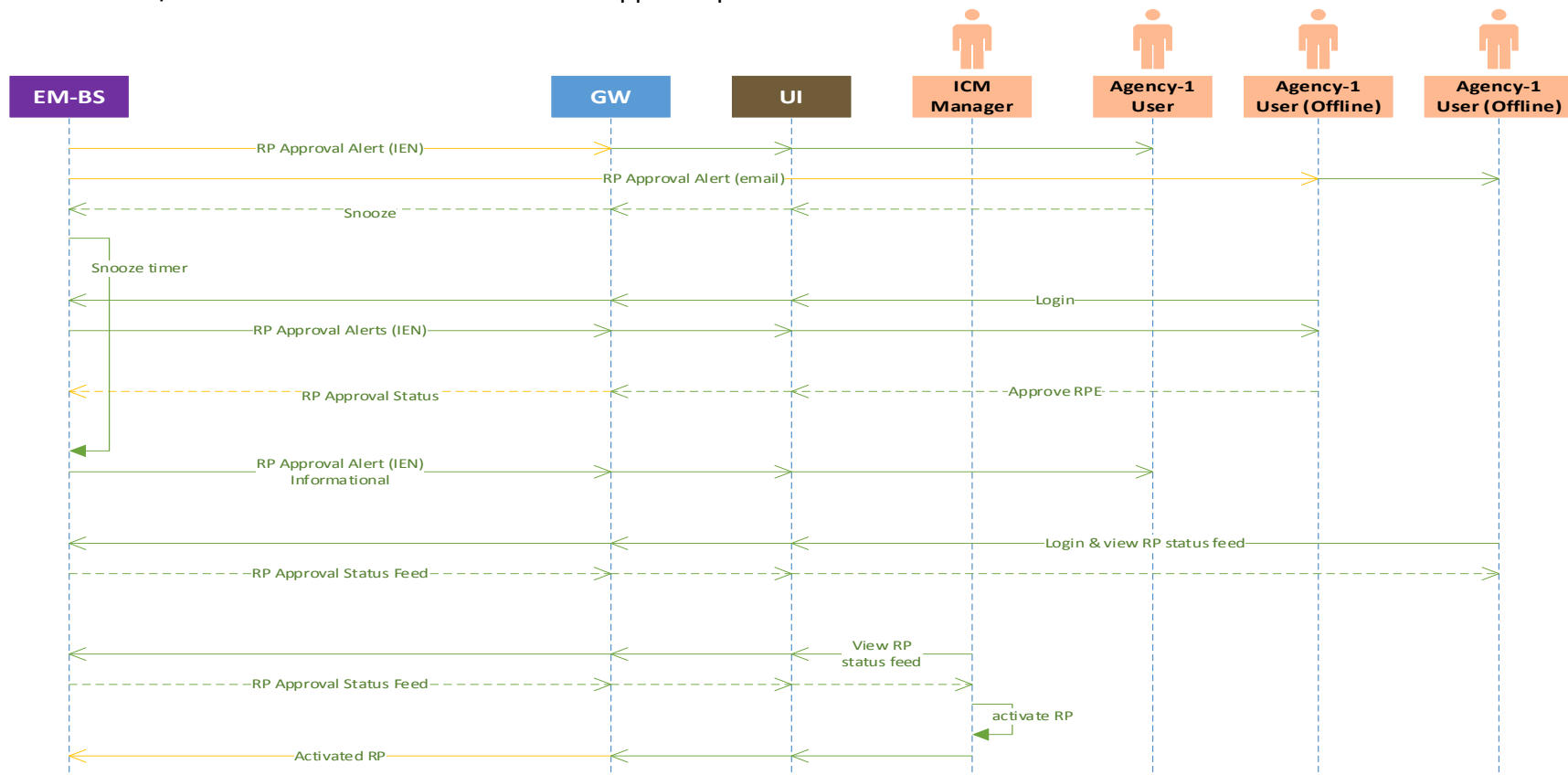


Figure 69 - Response Plan Notification Sequence Diagram

3.7.5.10 Other: Data Source Outage Alert

Figure 70 – Data Source Outage Alert Sequence Diagram shows a Data Source Outage alert notification process and depicts the following:

- Multiple users may subscribe to alerts for the Data Source Outage
- Users may also subscribe to offline, alerts
- Online alerts are received by IEN, offline alerts are received by email
- Snoozing alerts only affects a single user; a snoozed alert should always reappear

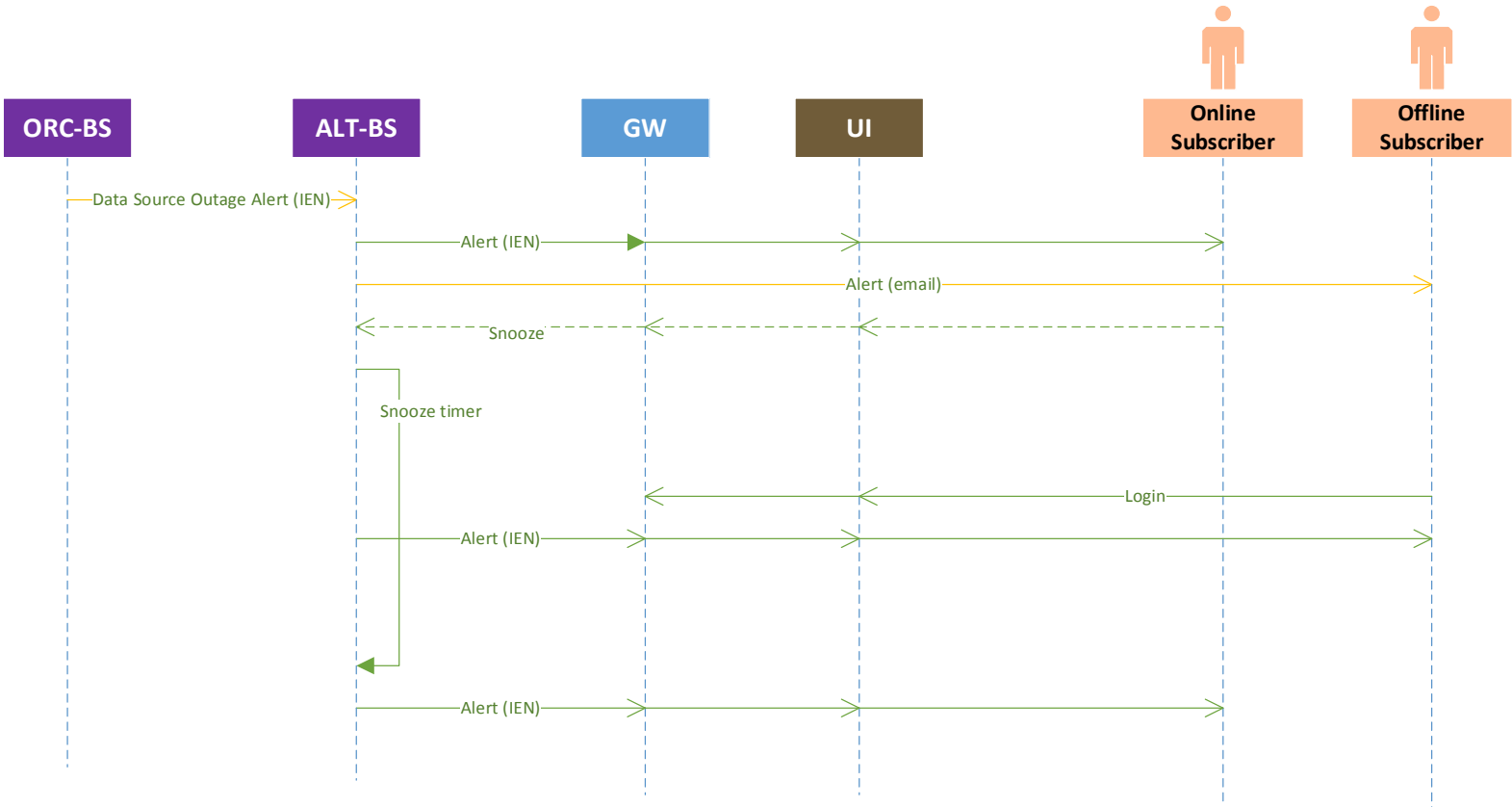


Figure 70 – Data Source Outage Alert Sequence Diagram

3.7.5.11 Other: Logging

Figure 71 - Logging Sequence Diagram shows an authenticated user making a request with examples of when logging occurs and the type of data it should contain. All logs will be saved for analysis and monitoring. Similar logging will be placed in all services.

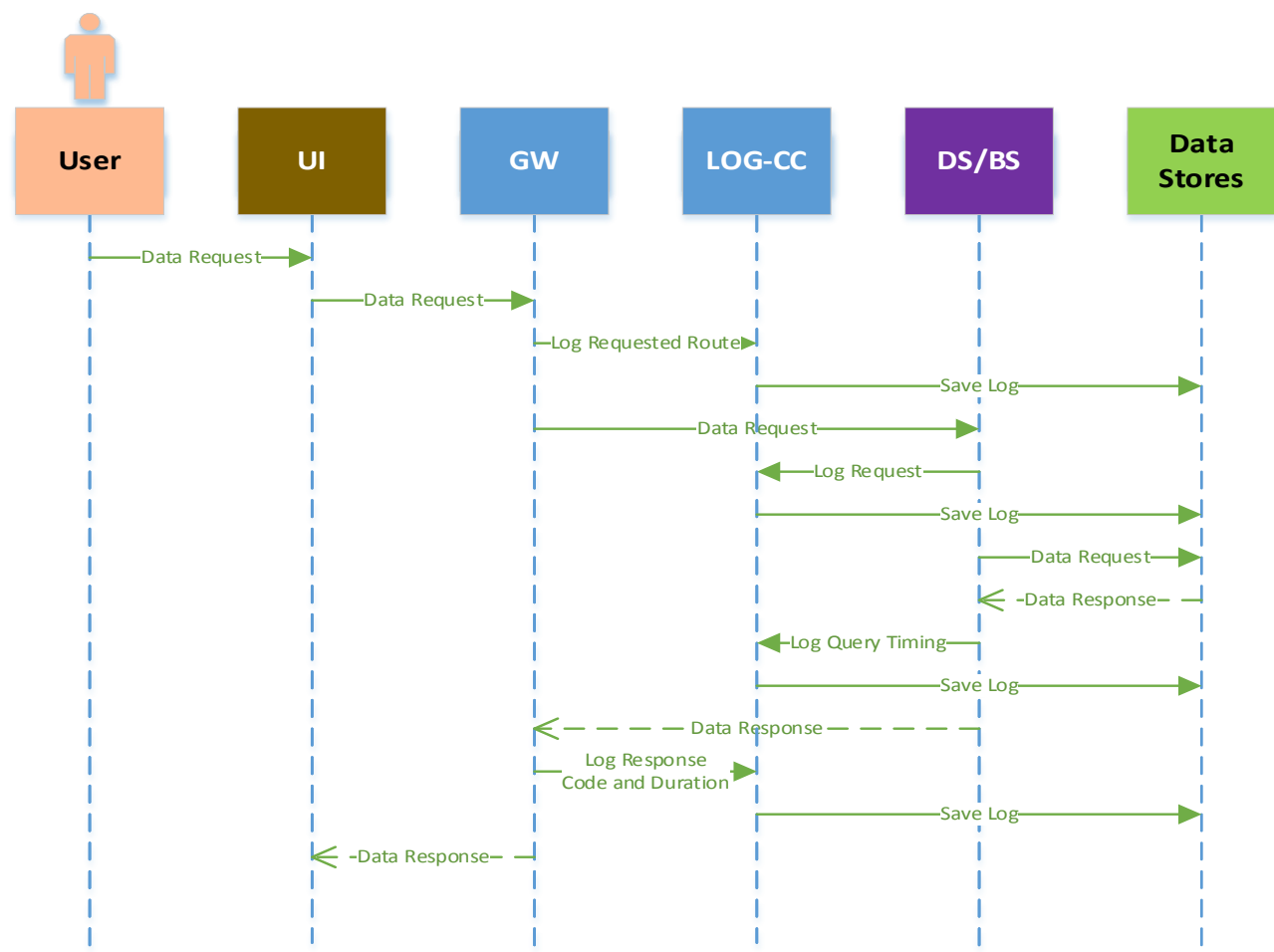


Figure 71 - Logging Sequence Diagram

3.7.5.12 Other: Monitoring

Figure 72 - Monitoring Sequence Diagram shows the activities that occur during system monitoring. The system monitor will periodically check the metrics gathered from logging and determine whether there's an issue with the system. When one is detected, the issue is logged, which saves the issue in a data store, and an alert is triggered.

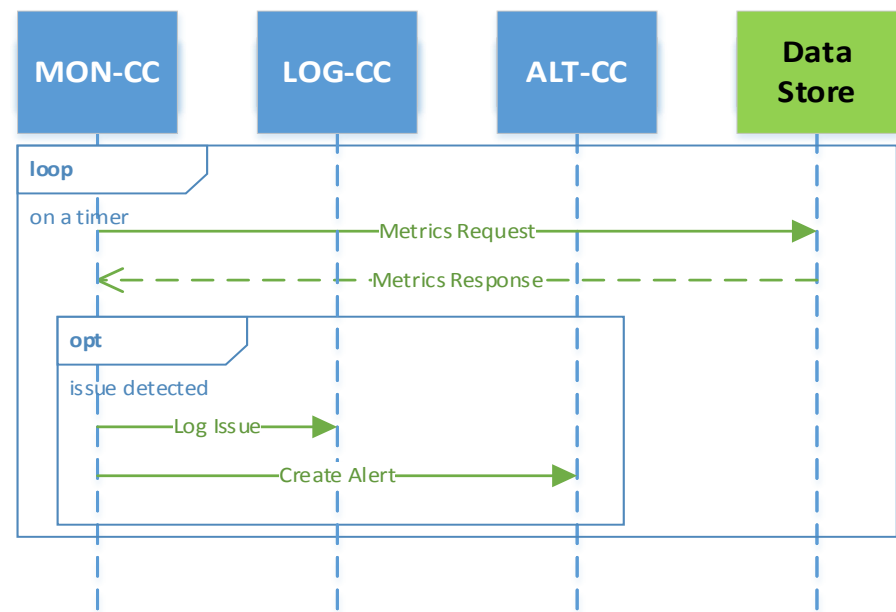


Figure 72 - Monitoring Sequence Diagram

3.7.5.13 Other: Timing Plan Data Service

Figure 73 – Timing Plan Data Service Diagram shows that an R-ICMS data service may combine information from multiple backing data stores or pipelines and provide it via a single interface for all internal services and end users. In this example, the TimingPlan-DS exposes current signal timing plans, which originate from SunGuide and various signal vendor ATMS interfaces, to the SOT business service and the modeling engine.

(API-1) GET /api/timingplan/ds/v1.0/plan/{id}

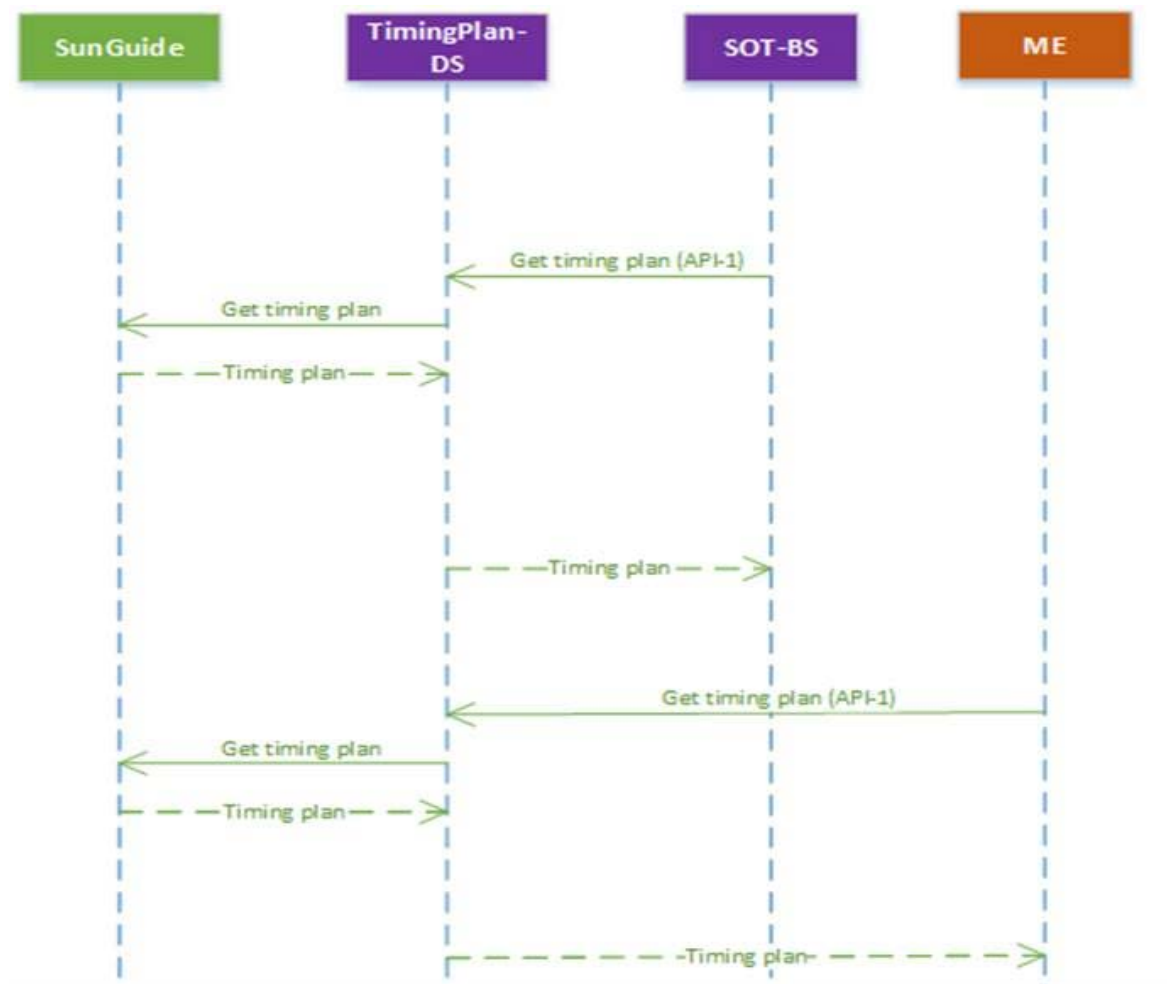


Figure 73 – Timing Plan Data Service Diagram

3.8 System Deployment

The section contains the diagrams and descriptions showing the deployment of physical and virtual components that comprise the R-ICMS system.

3.8.1 Service Host Deployment

Figure 74 – Service Host Deployment Diagram shows the interaction between various service components as deployed to physical servers hosting the R-ICMS. Lines between the boxes in Figure 74 – Service Host Deployment Diagram indicate that the components interact.

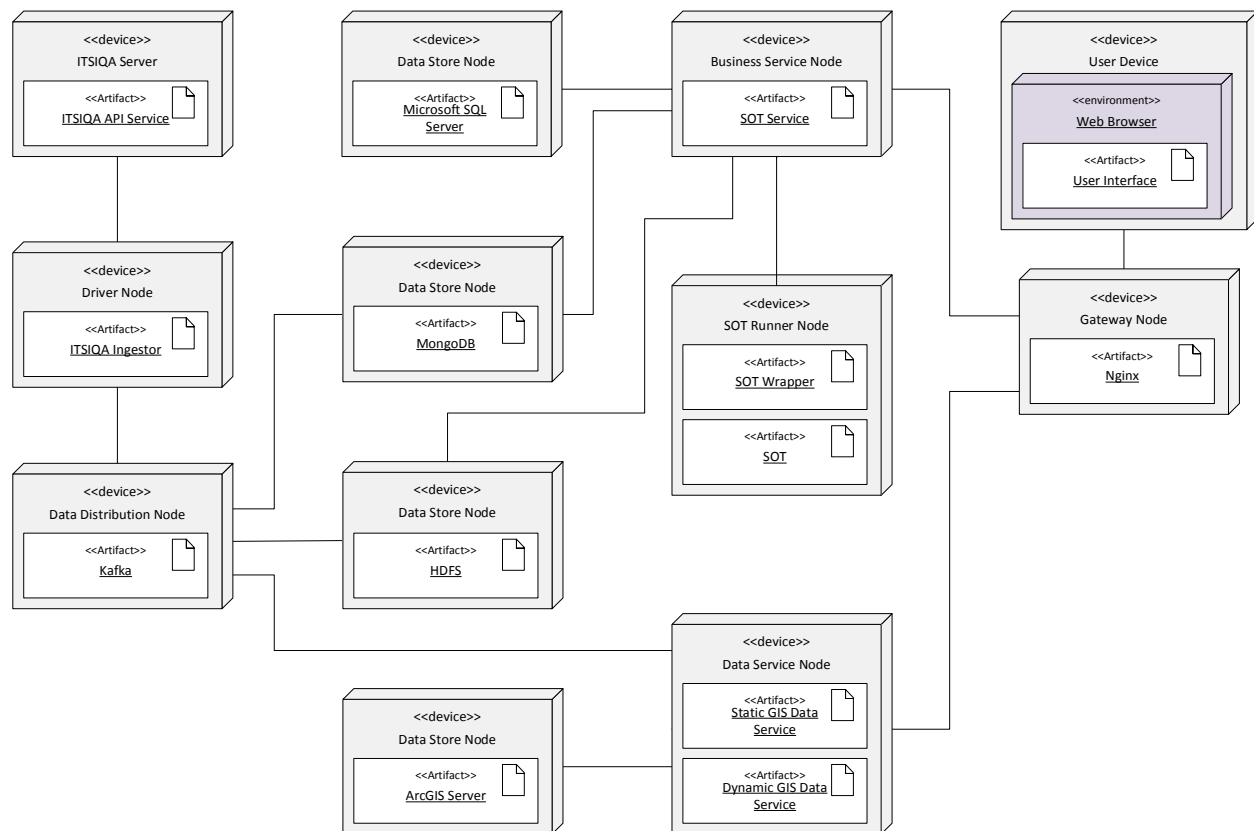


Figure 74 – Service Host Deployment Diagram

3.8.2 Containerized Service Orchestration

Figure 75 – Containerized Service Orchestration Diagram shows the scalable deployment and interaction between an internal R-ICMS service (Service X), the authentication and authorization service, and logging and monitoring services. Services are deployment onto host systems within Docker containers grouped into scalable pods by the Kubernetes service orchestration tool. This diagram assumes use Kubernetes as the orchestration tool and Elastic Stack for logging and monitoring with a Kafka buffer.

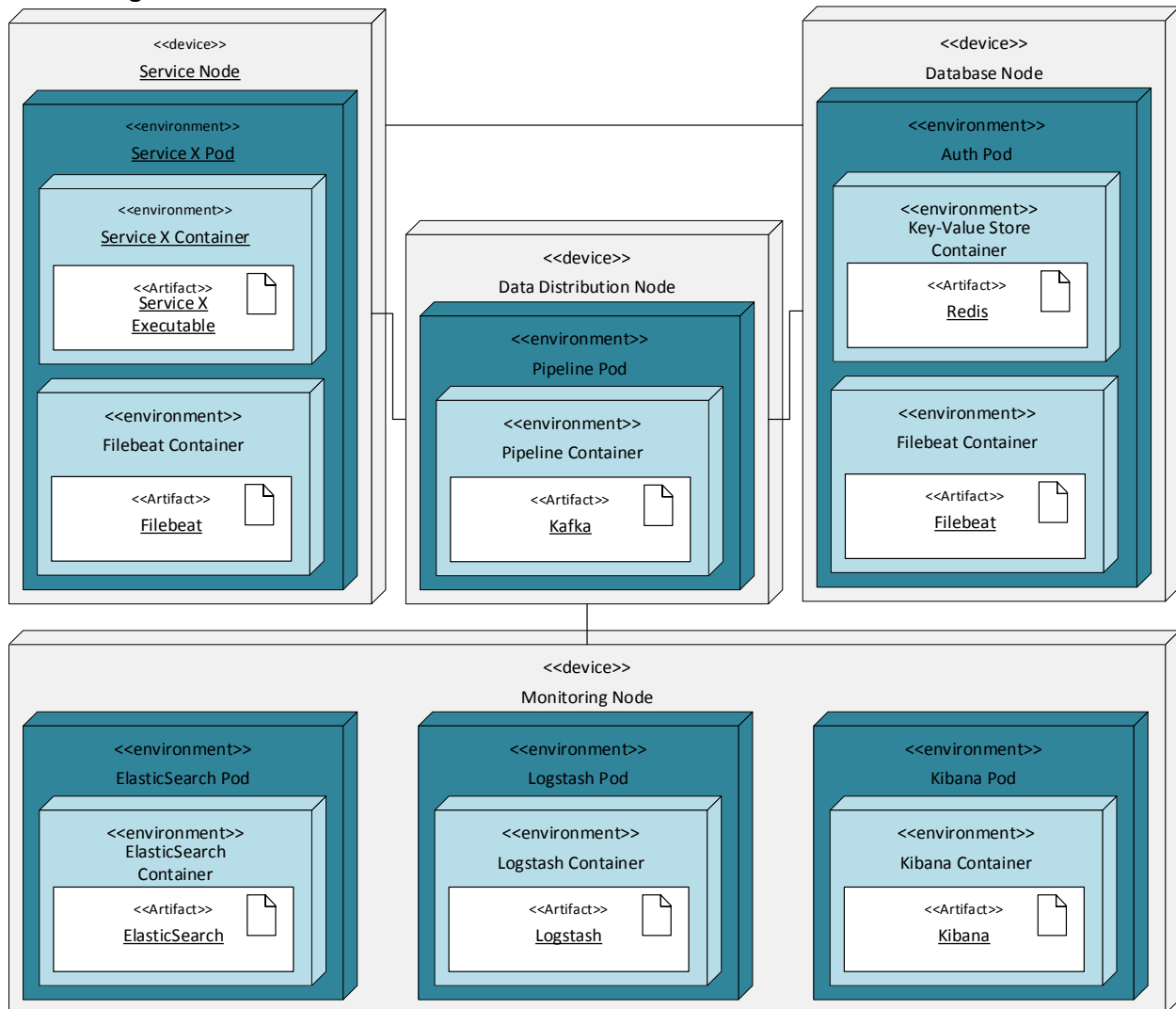


Figure 75 – Containerized Service Orchestration Diagram

4 Notes

4.1 Definitions

- Component – Generally, a part of some larger system or product
 1. Traffic system component – Part of the transportation system, such as a DMS or a traffic sensor
 2. Architectural components – Part of the designed implementation architecture, such as a service, a data store, a pipeline, or a user interface
- Corridor – Except where used in the title of this system, an acyclic set of coordinated neighboring signalized intersections
- Layer –
 1. A set of services/capabilities within the design architecture that provides similar capabilities and that interact in similar ways with other layers
 2. A set of viewable artifacts on a map interface that can be turned on or off (i.e., made visible or invisible to the user)
- Response Plan – A set of response plan elements which can be considered for implementation in response to traffic conditions
- Response Plan Element – A specific change that can be made to controllable elements of the traffic network; e.g., activation of a signal timing plan or putting a set of message elements on a DMS
- Special Event – A future planned event; e.g., parade, football game
- Subsystem – generally a system that is part of a larger system
 1. One of the three parts of the R-ICMS identified in the original Invitation to Negotiate (the IEN, DSS, and DFE)
 2. A part of SunGuide dealing with specific types of activities or data

Appendix A

Data Sources Table

R-ICMS Data Source	Data Source Detail (if decomposition is needed)	Data Source availability mechanism	Update Interval
ITSIQA	Traffic Conditions Data	ITSIQA	1 min
District 5 SunGuide System	Traffic Incident Data	SunGuide / Databus / EM	real-time
District 5 SunGuide System	CCTV Status	SunGuide / Databus / CCTV	real-time
District 5 SunGuide System	Ramp Meter Status	SunGuide / Databus / RMS	real-time
District 5 SunGuide System	Dynamic Message Signs Status	SunGuide / Databus / DMS	real-time
District 5 SunGuide System	Connected Vehicle Roadside unit status	SunGuide / Databus / CV	real-time
Signal Performance Measures	Volume for each movement	ATSPM D5 Deployment	1 min
Signal Performance Measures	Arrival timing per movement	ATSPM D5 Deployment	1 min
Transit GTFS-RT - SunRail			real-time
Transit GTFS-RT - Lynx Clever			real-time
Transit GTFS-RT - Lynx Trapeze			real-time
Transit GTFS-RT - Votran			real-time
Transit GTFS-RT - Lake Express			real-time
Transit GTFS-RT -SCAT (Space Coast Area Transit)			real-time
Transit GTFS-RT -SunTran (Marion County)			real-time
Transit GTFS - SunRail			
Transit GTFS - Lynx			
Transit GTFS - Votran			
Transit GTFS - Lake Express			
Transit GTFS -SCAT (Space Coast Area Transit)			
Transit GTFS -SunTran (Marion County)			
Weather	National Weather Service Watches and Warnings	NOAA; Is there capability to do weather radar for map overlay for IEN	real-time
Parking Data	Garages, surface lots, weigh stations, rest areas, beaches, on-street parking	SunGuide / Databus / TPS or Parking Subsystem	real-time
basemap "backdrop"		HERE Navstreets or ESRI ArcGIS system	quarterly

R-ICMS Data Source	Data Source Detail (if decomposition is needed)	Data Source availability mechanism	Update Interval
basemap links		FDOT (manually corrected version of HERE.com basemap)	3 months
Roadway Characteristics Inventory (RCI)	# lanes at intersection	FDOT RCI	
Predictive Engine Data		PRE	
Expert Rules Engine – Response Plans		ERE	
Evaluation Engine		EVE	
Intersection Geometry Data		TBD	
Intersection Plans and Schedules		TBD	
Intersection Movement Counts Data	turning movement counts	IMC	
Intersection Movement Counts Data	approach/direction	IMC	
Intersection Movement Counts Data	# lanes	IMC	
Intersection Movement Counts Data	saturation flow rates	IMC	
Intersection Movement Counts Data	vehicle detection	IMC	
Intersection Movement Counts Data	bike detection	IMC	
Intersection Movement Counts Data	pedestrian detection	IMC	
Origin Destination - SunGuide BlueTOAD	BlueTOAD	SunGuide D4 Enhancement	
Origin Destination - Tag Reads	BlueMAC	SunGuide	
Controller or ATMS	Signal Phasing	Signal ATMS Software or SunGuide	
Controller or ATMS	Detector Status Data (Vehicular and Pedestrian)	Signal ATMS Software or SunGuide	
Controller or ATMS	Controller Data	Signal ATMS Software or SunGuide	
Controller or ATMS	Available Controller Timing Patterns with timing plan details	Signal ATMS Software or SunGuide	
Controller or ATMS	Controller Timing pattern status	Signal ATMS Software or SunGuide	
Controller or ATMS	Corridor Plan	Signal ATMS Software or SunGuide	
Special Event Information		AAM Dashboard	irregular

R-ICMS Data Source	Data Source Detail (if decomposition is needed)	Data Source availability mechanism	Update Interval
School locations		FDOT provided GIS layer	
School zones		FDOT provided GIS layer	
School schedules		FDOT provided data	
Express lanes status	status, current price, current pricing model, if the system has any dynamic features such as changeable toll rates, changeable lane configurations	SunGuide - Future	
Emergency Responder Locations	Hospitals, Police stations, Fire stations	TBD (GIS layer possibly)	
Railroad Signaling System	PTC Insaldo ATMS (SunRail)	TBD	
Video - iVEDDS			